



Correspondence of Local Features for Wide-Baseline Matching, Image Retrieval, Stitching, 3D reconstruction and more ...

Jiří Matas, Ondra Chum, Jan Čech, Dmytro Mishkin
Center for Machine Perception,
Czech Technical University Prague

Includes slides by:

- Darya Frolova, Denis Simakov, The Weizmann Institute of Science
- Martin Urban, Stepan Obdrzalek, Ondra Chum, Filip Radenovic, CTU CMP Prague
- Matthew Brown, David Lowe, University of British Columbia
- Daniel DeTone, Paul-Eduard Sarlin, MagicLeap
- Jerome Revaud, Naver Labs Europe

- Reconstruction/Retrieval – this lecture, later.
- Detection & recognition.
- Tracking
- Segmentation
- OCR, VQA, VLM

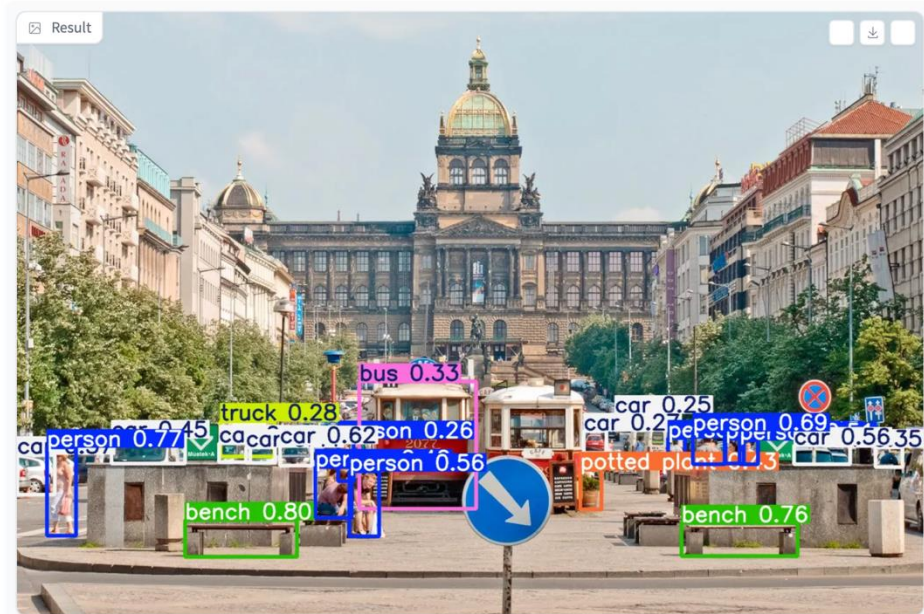
We don't cover here, but there are other courses:

- Generation – images, 3D assets, etc. -> Selected topics in AI
<https://cw.fel.cvut.cz/b252/courses/becm36stai/start>)
- Non-standard sensors – event cameras, multispectral, drone control, black hole reconstruction

Object Detection: YOLO11

Ultralytics Gradio Application 🚀

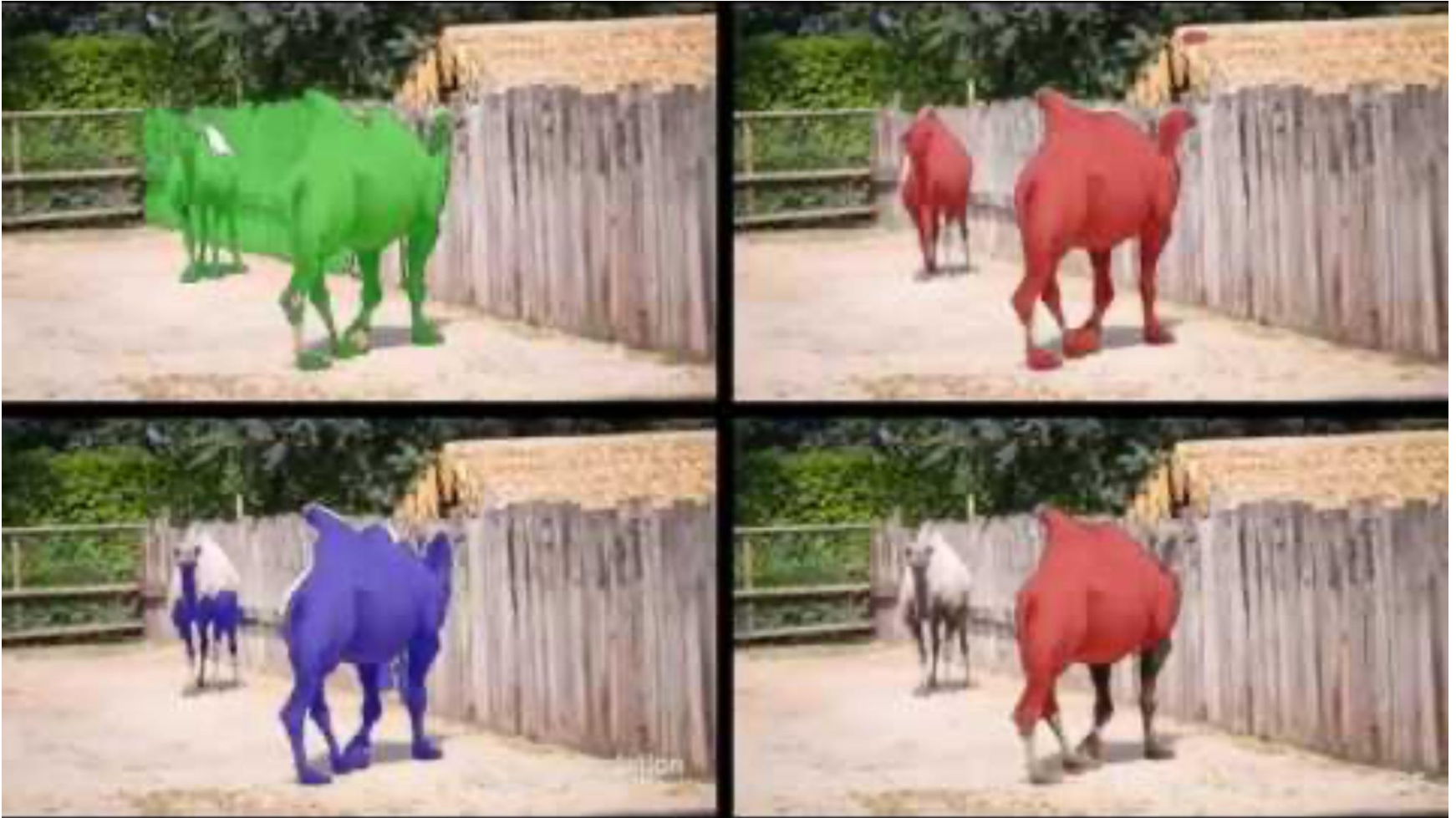
Upload images for inference. The Ultralytics YOLO11n model is used by default.



Tracking: SAM3

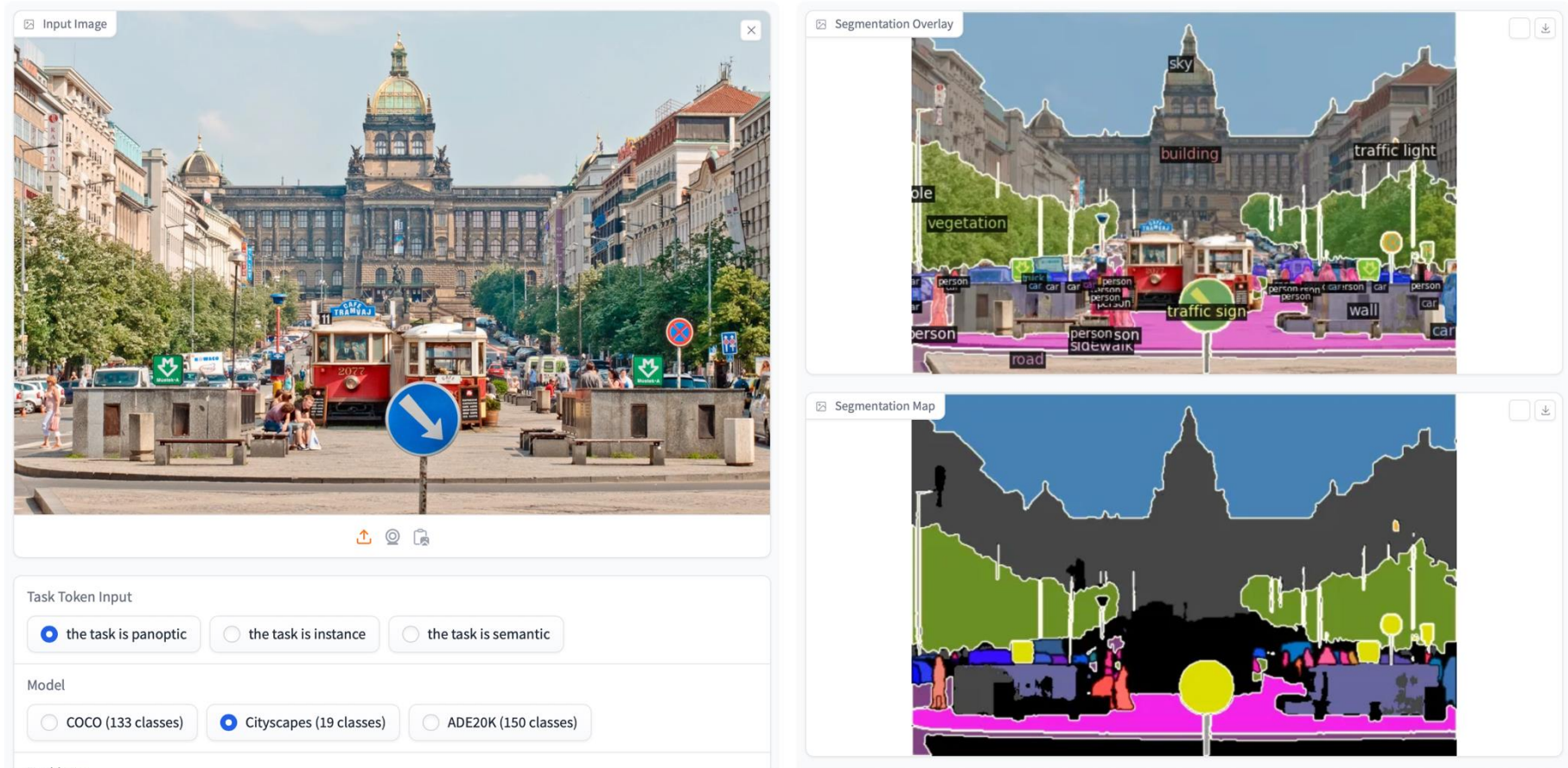


Tracking: SAM3



Segmentation: OneFormer

processing time may take up to 2 minutes. On a single RTX 3090 or RTX 4090, OneFormer needs to process images with size 1024x1024.



<image>\nDescribe this image in detail.



A green plastic container labeled for food storage purposes. The lid is yellow and appears slightly worn out.

On the front label:

- At the top left corner, some Chinese characters.
- In bold black letters near the center-top reads "6月香" (June Fragrance).
- Below that, smaller white font states "鲜甜咸香" which translates roughly to "Fresh Sweet Salty Fragrant."
- Dominating the middle part of the label is large lettering stating "豆瓣酱," meaning "Bean Paste Sauce."

Additional information includes:

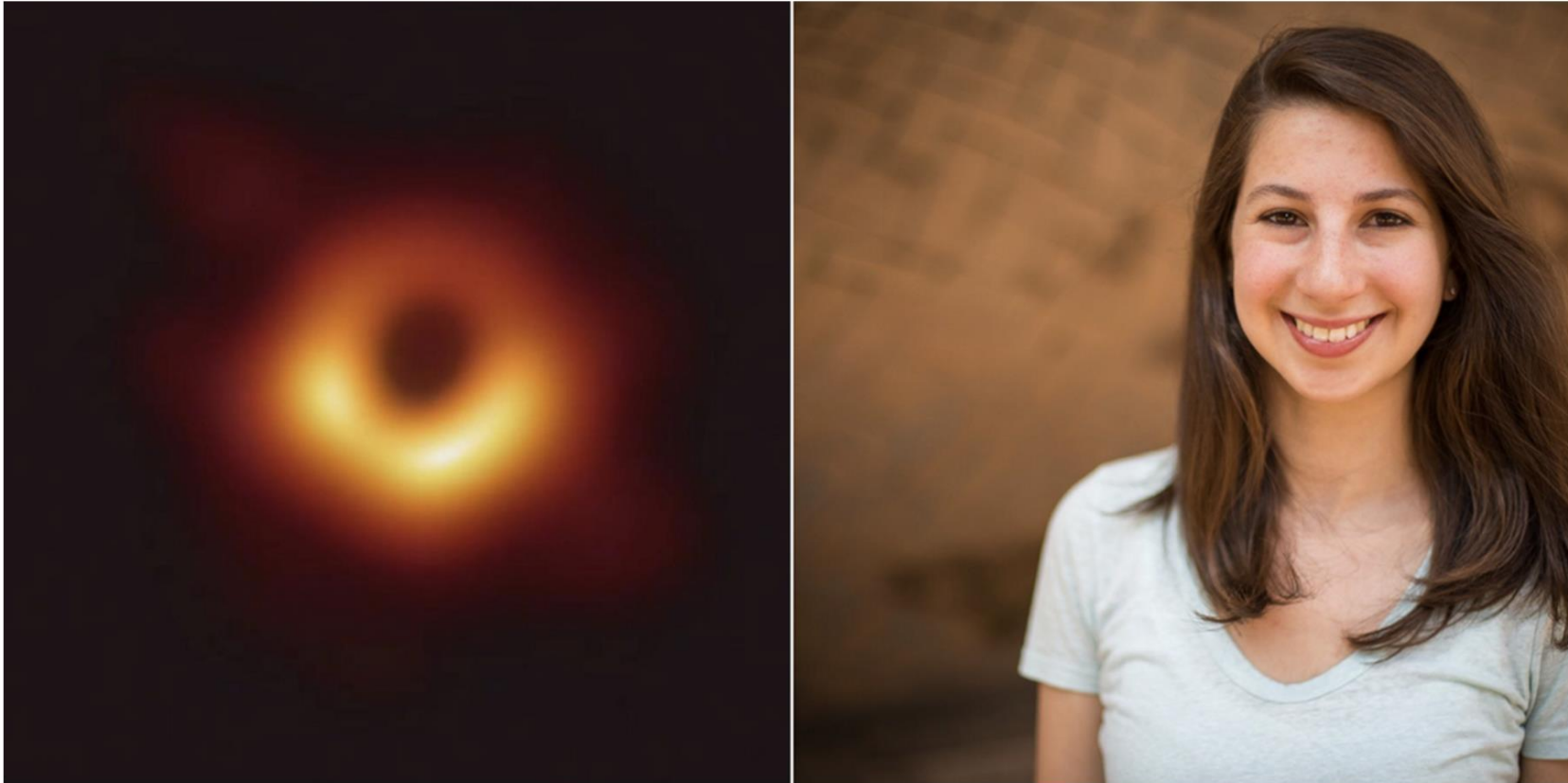
- A small circular logo indicating quality assurance ("优质品").
- An illustration showing what seems like a dish made using bean paste sauce.
- Text below the main title says "专注25年, 只做原酿酱" translating to "Focusing since 1998, only making authentic fermented bean paste."
- On the bottom right side, additional product specifications include "净含量: 500g".

Overall Style/Type:

It looks like commercial packaging designed specifically for preserving food items within the jar. It has clear labeling intended for easy identification and understanding among consumers who read Mandarin.

Note: There might be slight variations due to handwriting styles but overall maintains readability regarding brand identity and contents description.

Black hole image reconstruction



As the first ever picture of a supermassive black hole circled the Internet on April 10, 2019 – Dr. Katie Bouman now has a **Wikipedia page**. She is notable because her computer science work helped contribute to the imaging of the **black hole** for the first time – a feat previously thought impossible.

Image Generation



Prompt:

Create a hyper-realistic, ultra-sharp, full-color large-format image featuring a massive group of celebrities from different eras, all standing together in a single wide cinematic frame. The image must look like a perfectly photographed editorial cover with impeccable lighting, lifelike skin texture, micro-details of hair, pores, reflections, and fabric fibers.

GENERAL STYLE & MOOD:

Photorealistic, 8k, shallow depth of field, soft natural fill light + strong golden rim light. High dynamic range, calibrated color grading. Skin tones perfectly accurate. Crisp fabric detail with individual threads visible. Balanced composition, slightly wide-angle lens (35mm), center-weighted. All celebrities interacting naturally, smiling, posing, or conversing. Minimal background noise, but with enough world-building to feel real.

THE ENVIRONMENT: A luxurious open-air rooftop terrace at sunset overlooking a modern city skyline. Elements include: Warm golden light wrapping around silhouettes. Polished marble.



Prompt:

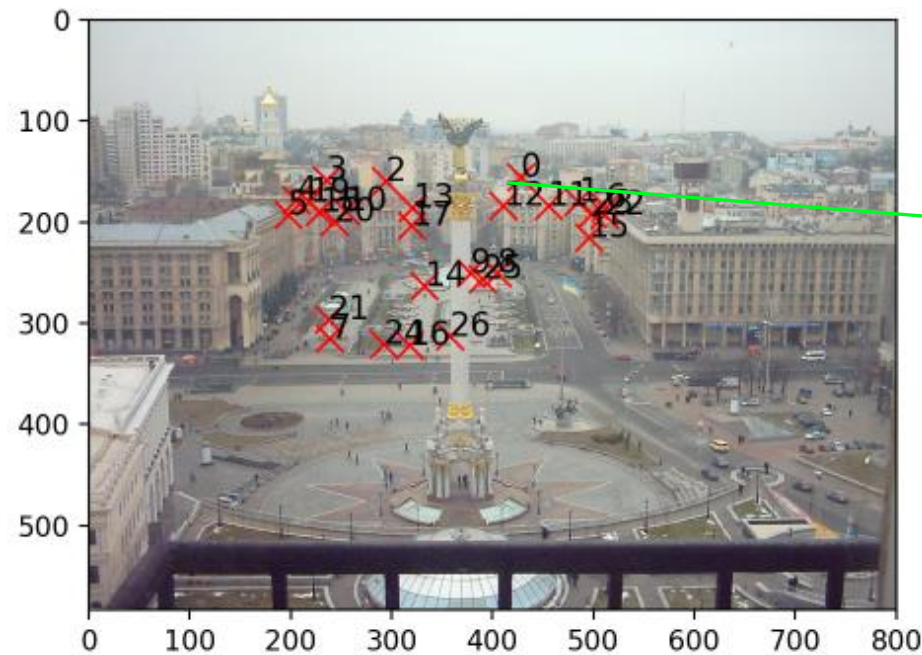
A daily snapshot taken with a low-quality disposable camera.
A clumsy photo taken by a Japanese high school student.
(Aspect ratio 3:2 is recommended)

nature

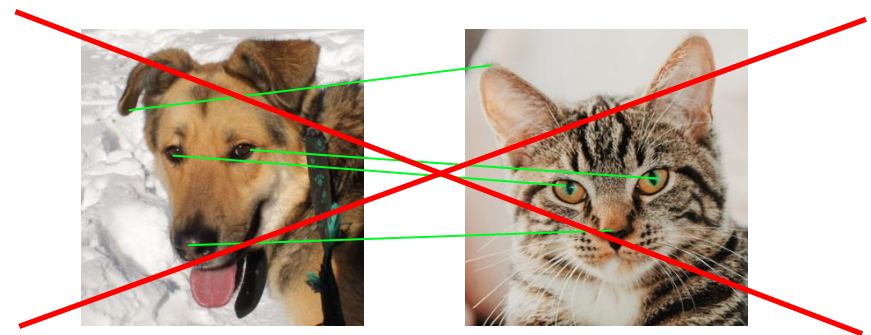


The *Geometric* Correspondence Problem

- We will be interested in **local geometric correspondence**

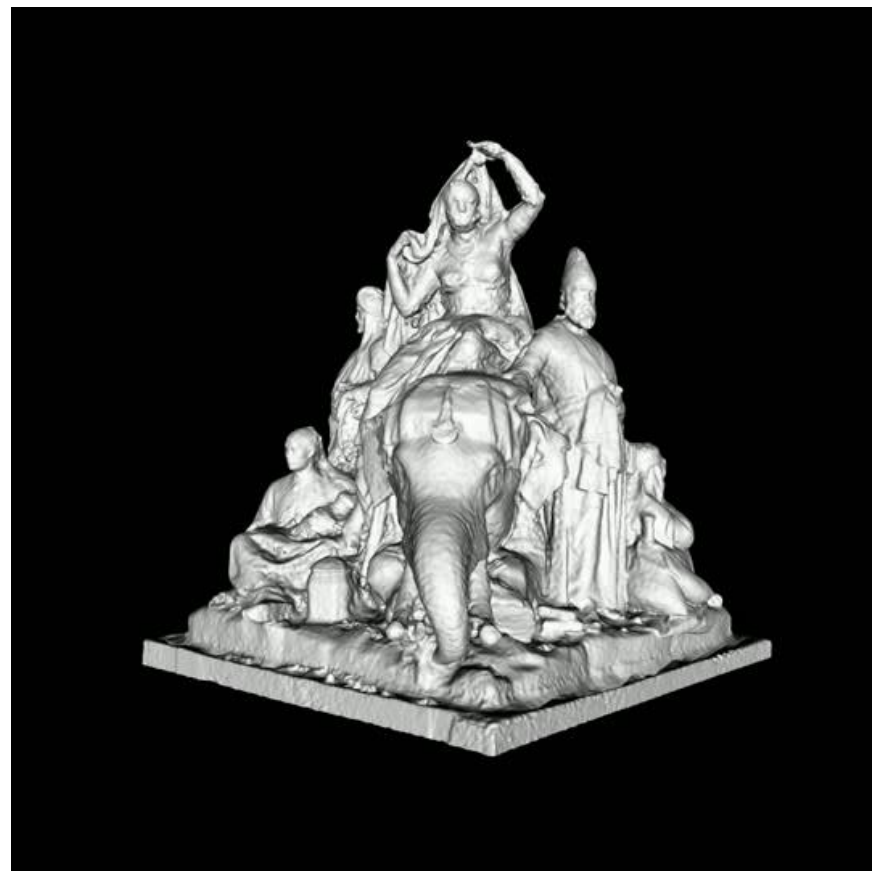


- **Not semantic correspondence**



Correspondences are useful for 3D reconstruction

3D reconstruction – camera pose estimation



Correspondences are useful for 3D reconstruction

Large scale 3D reconstruction – “Microsoft Photo Tourism”

57,845 downloaded images, 11,868 registered images.

The Old City of Dubrovnik



SLAM: simultaneous localization and mapping



R. Mur-Artal, and J. D. Tardós. ORB-SLAM2: an Open-Source SLAM System for Monocular, Stereo and RGB-D Cameras, arXiv 2016

Not only buildings: medical image registration

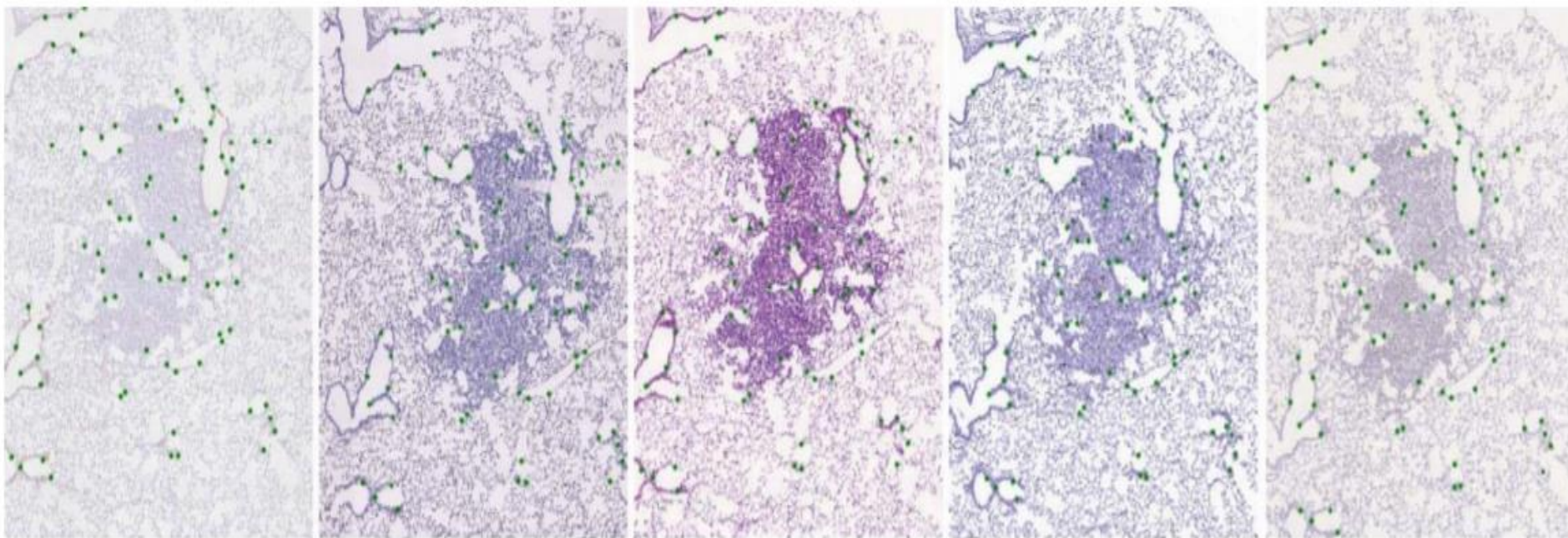


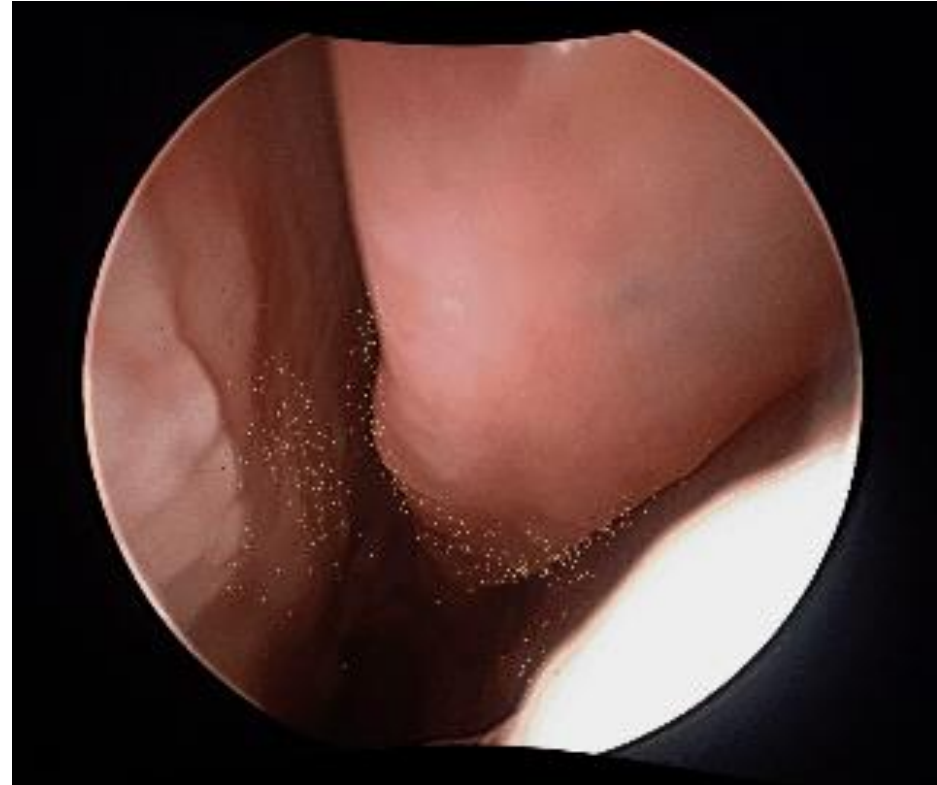
Fig. 2. An example of a set of lung lesion tissue images using Cc10, CD31, H&E, Ki67 and proSPC stains (from left to right) with landmarks shown as green dots.

ANHIR: Automatic Non-Rigid Histological Image Registration Challenge. Borovec et al.
IEEE TRANSACTIONS ON MEDICAL IMAGING, 2020

Endoscopy



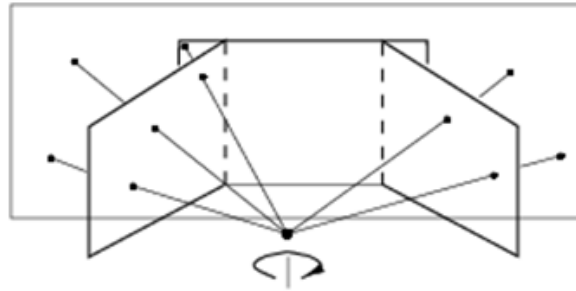
Learned dense descriptor



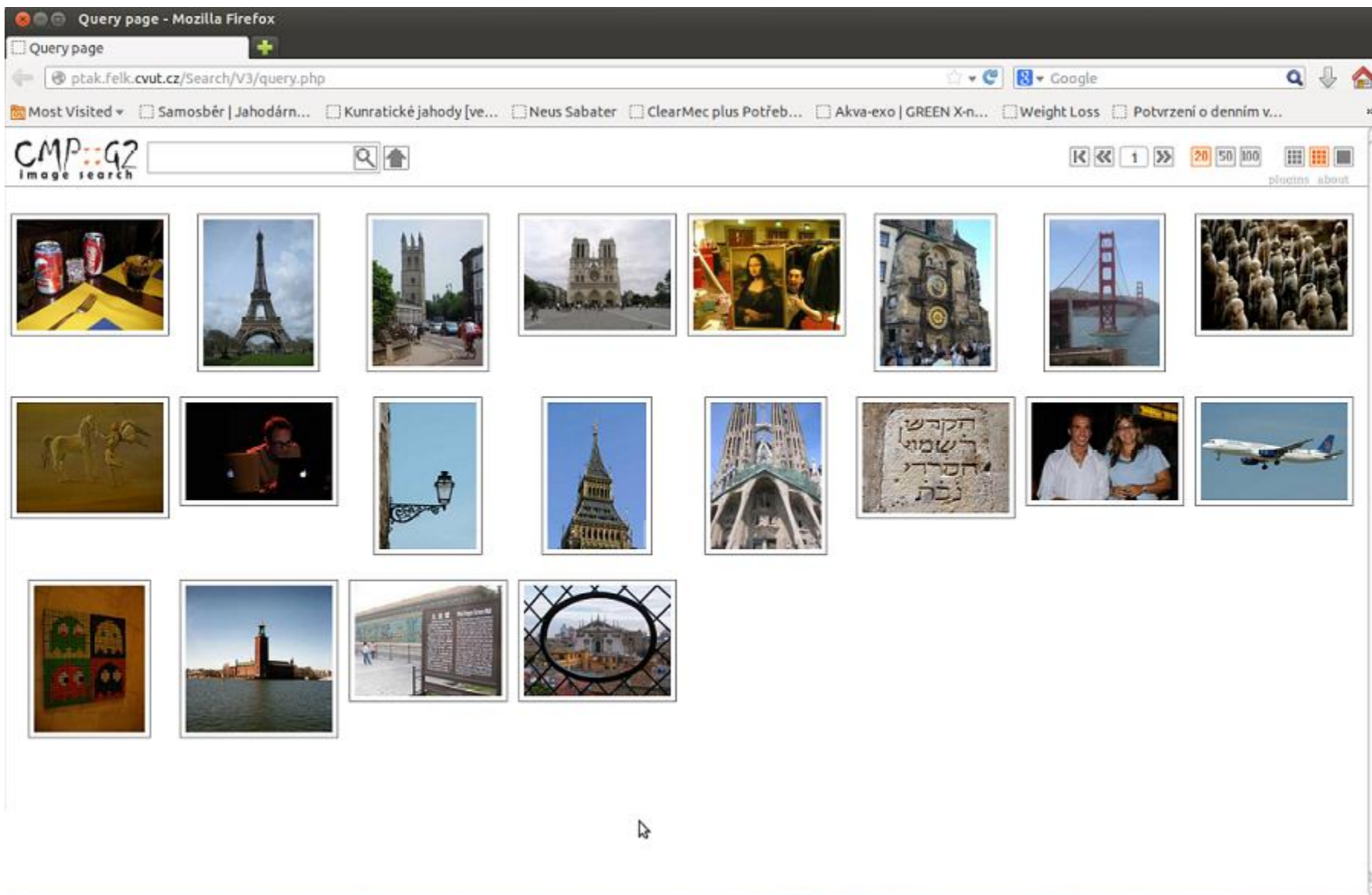
SIFT

Extremely Dense Point Correspondences using a Learned Feature Descriptor. Liu et.al CVPR 2020

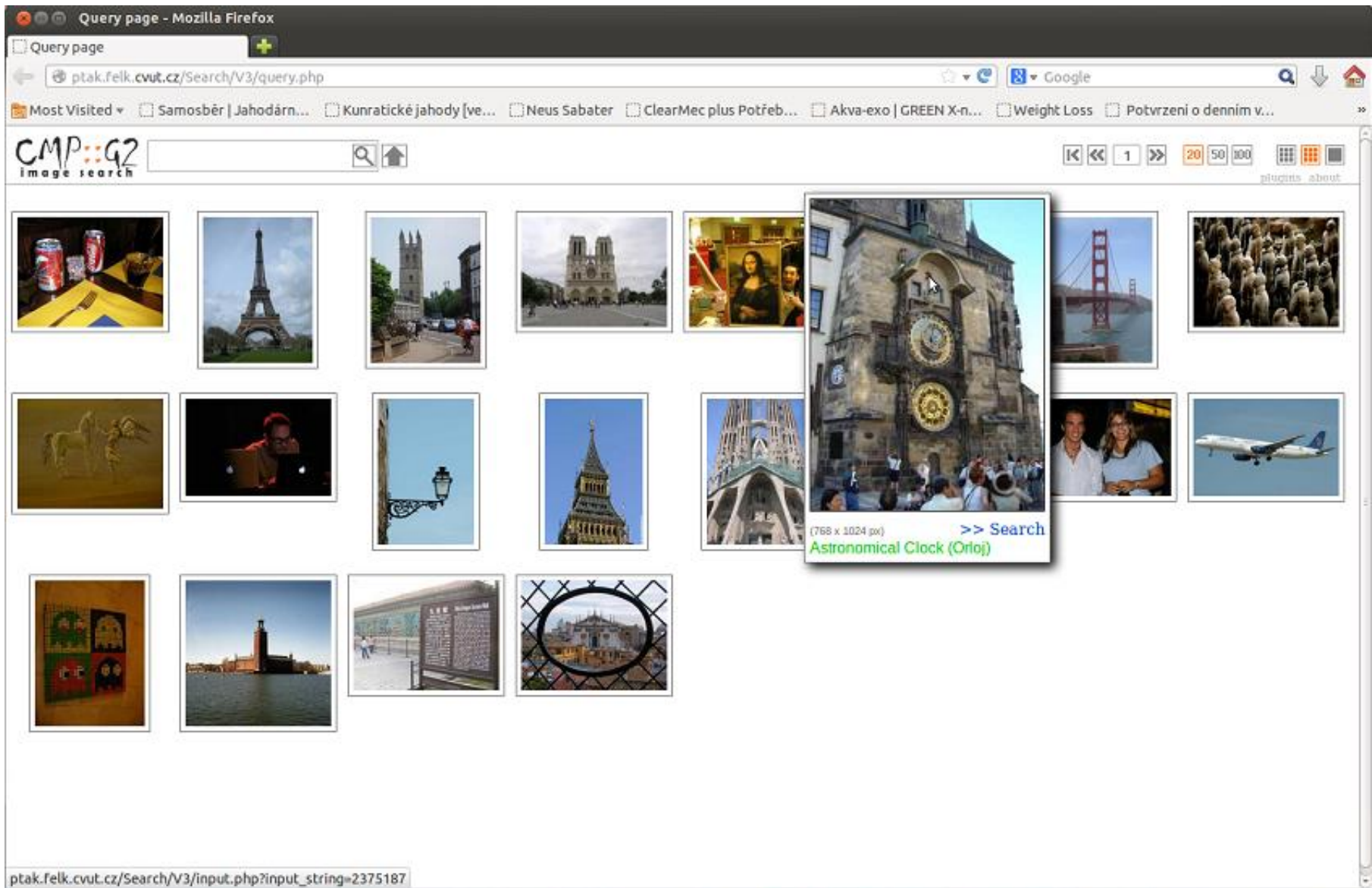
How does it work? Panorama stitching



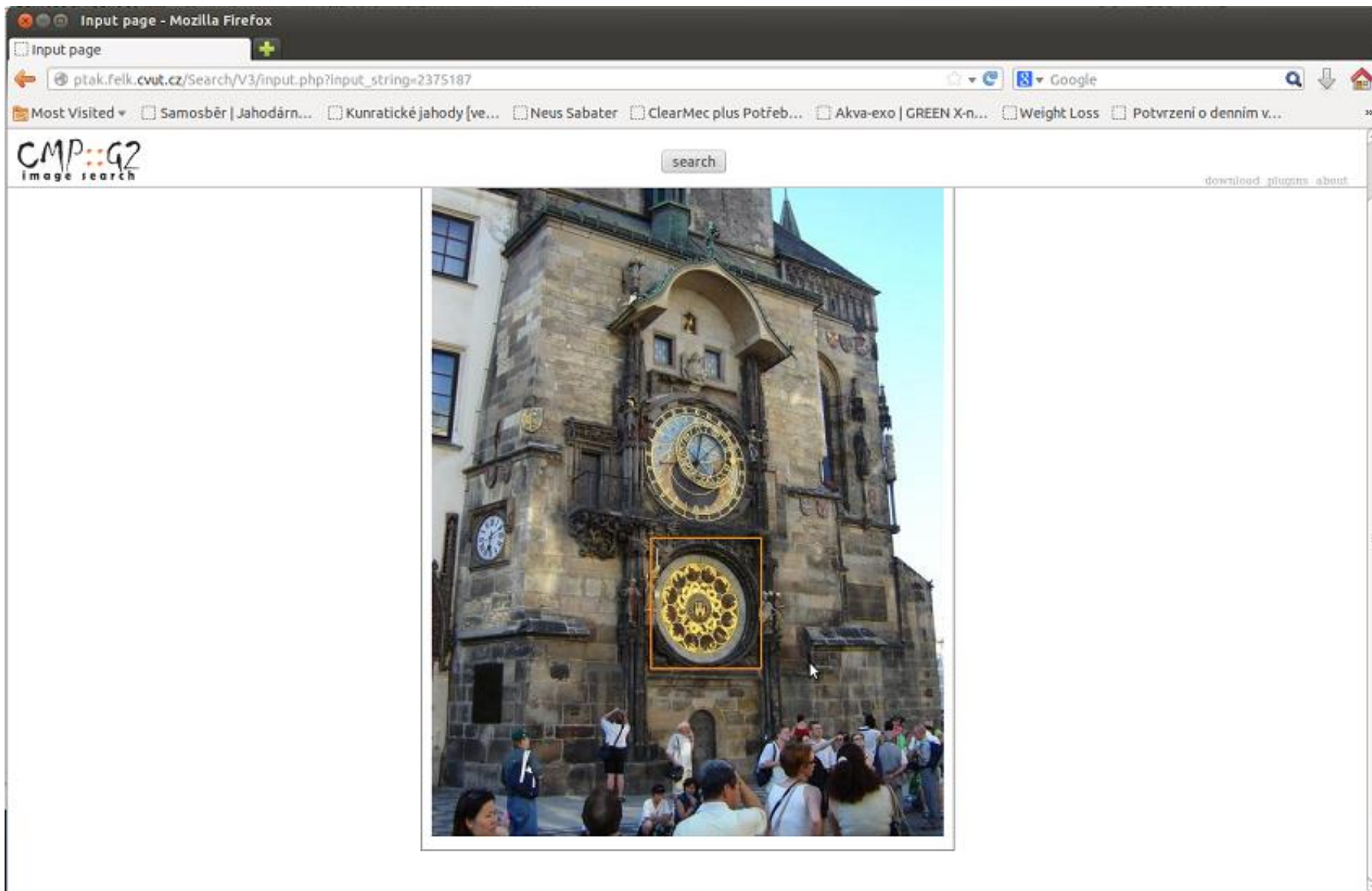
Correspondences in Action (2): Image Retrieval



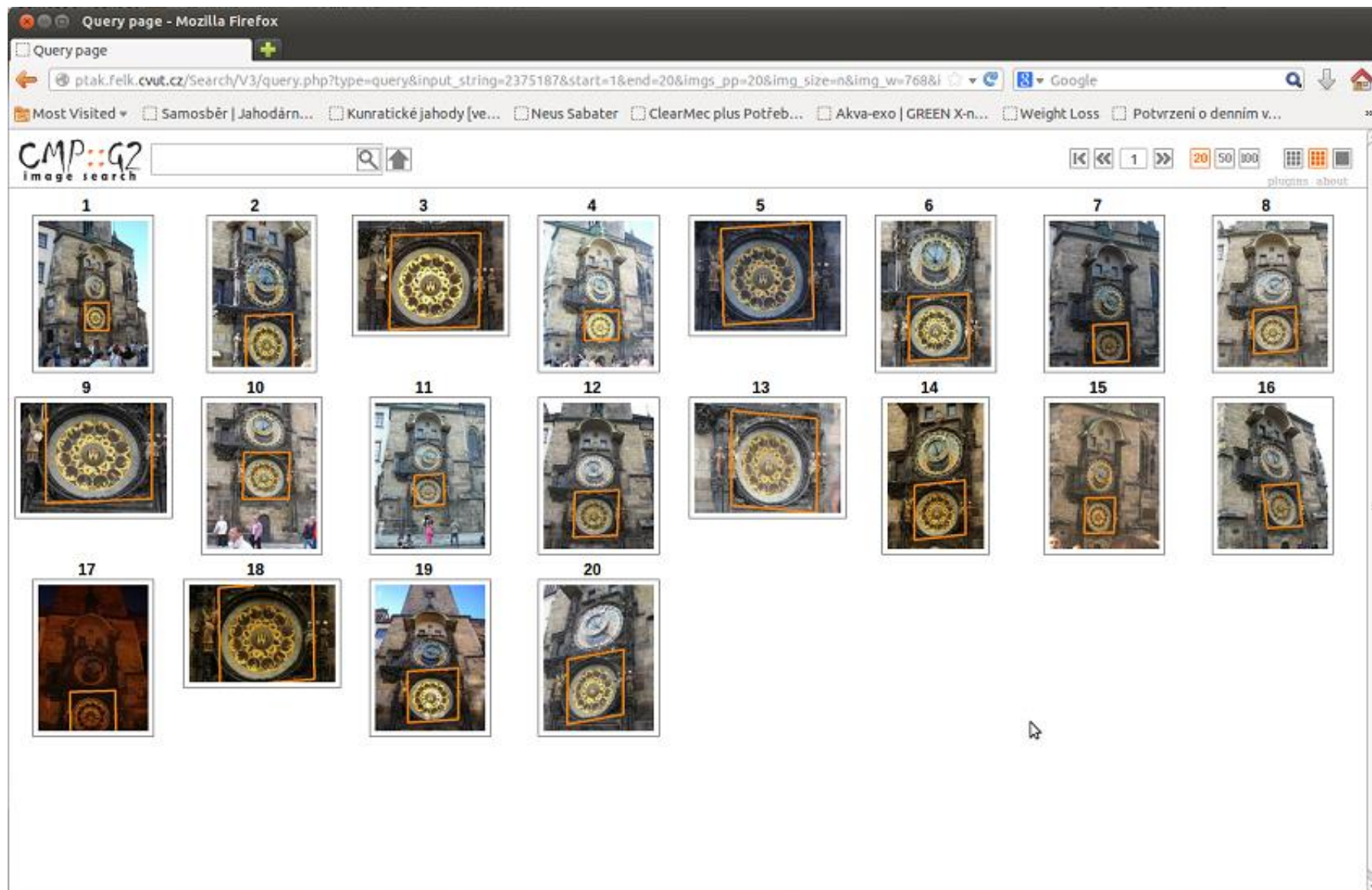
Correspondences in Action (2): Image Retrieval



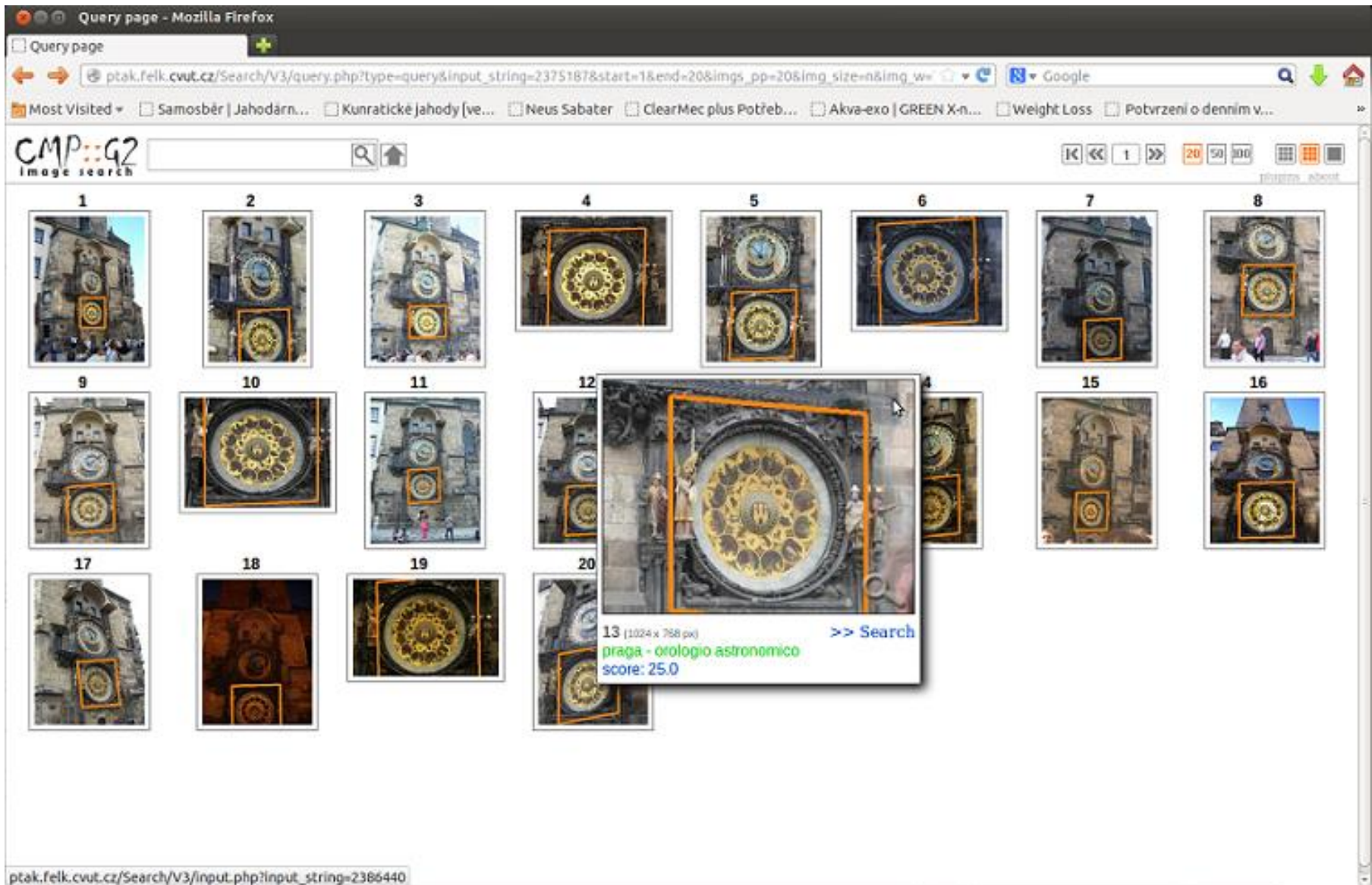
Correspondences in Action (2): Image Retrieval



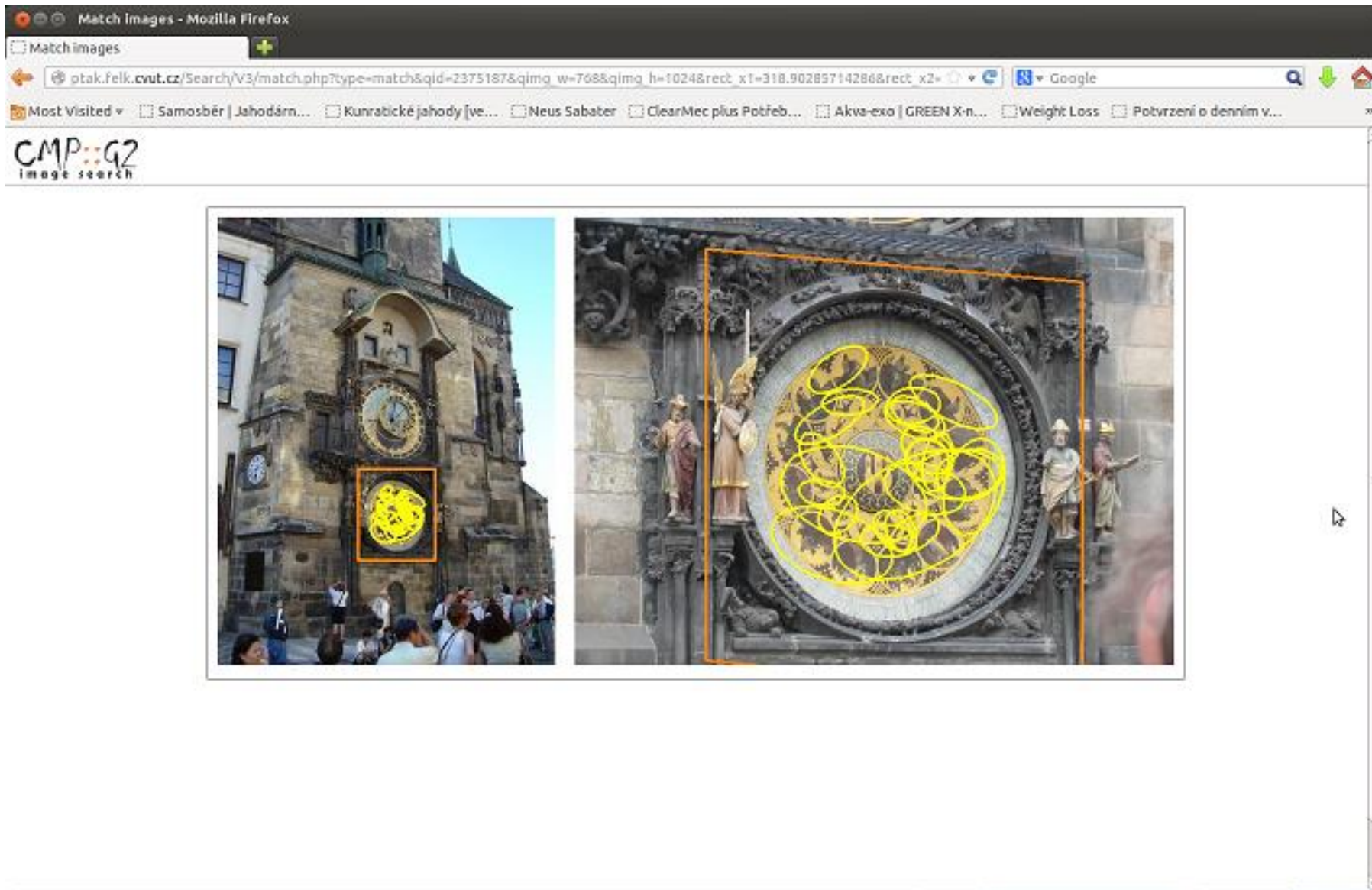
Correspondences in Action (2): Image Retrieval



Correspondences in Action (2): Image Retrieval

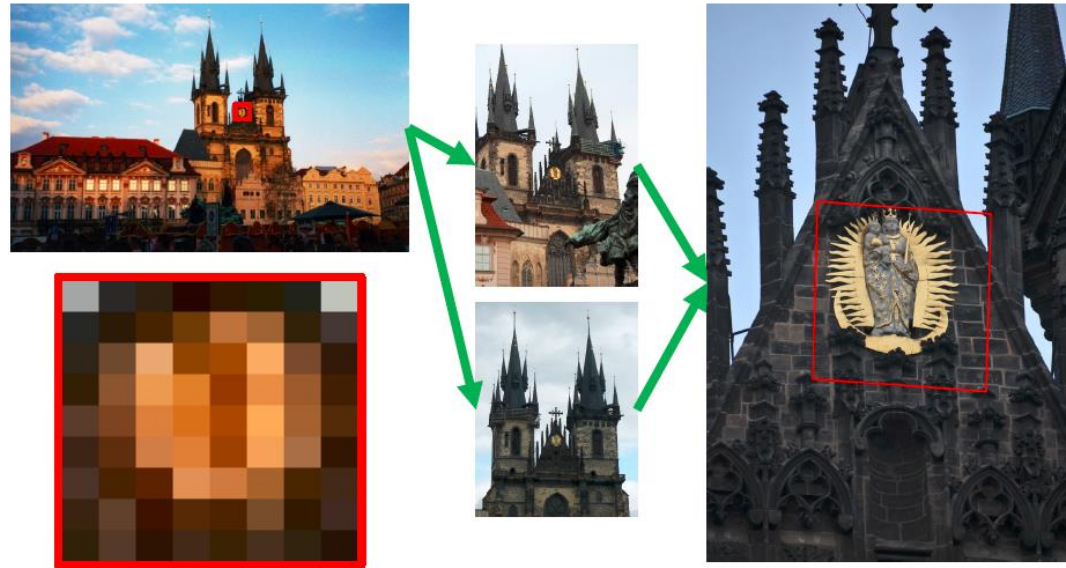


Local Features in Action (2): Image Retrieval

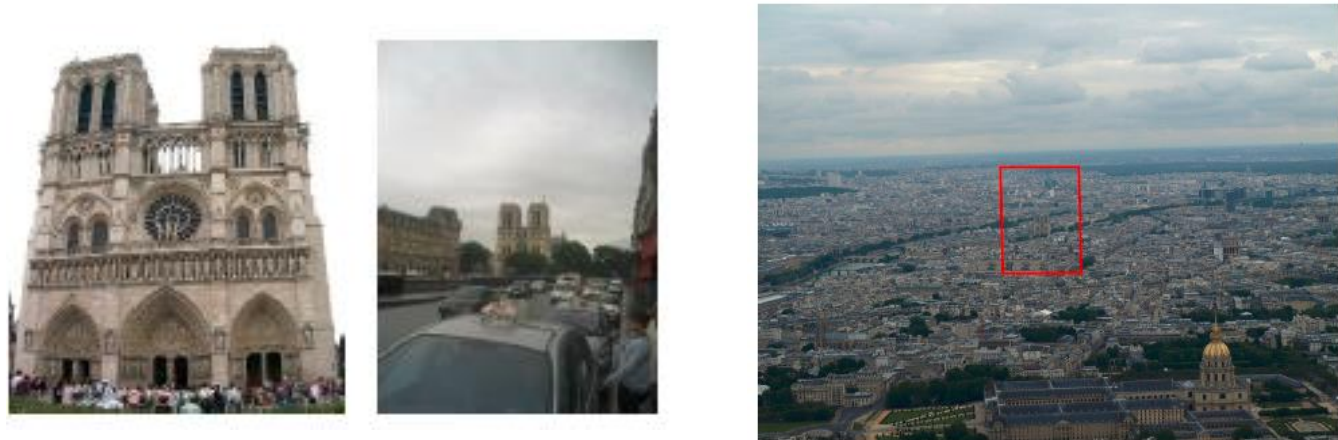


Correspondences in Action (2): Image Retrieval

“Zoom in”

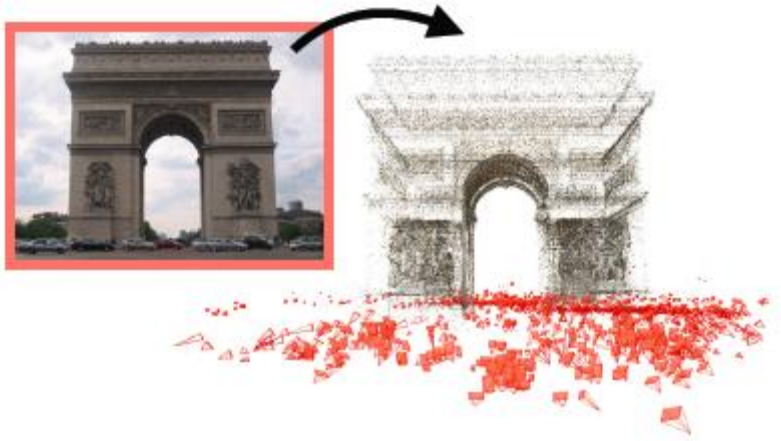


“Zoom out”

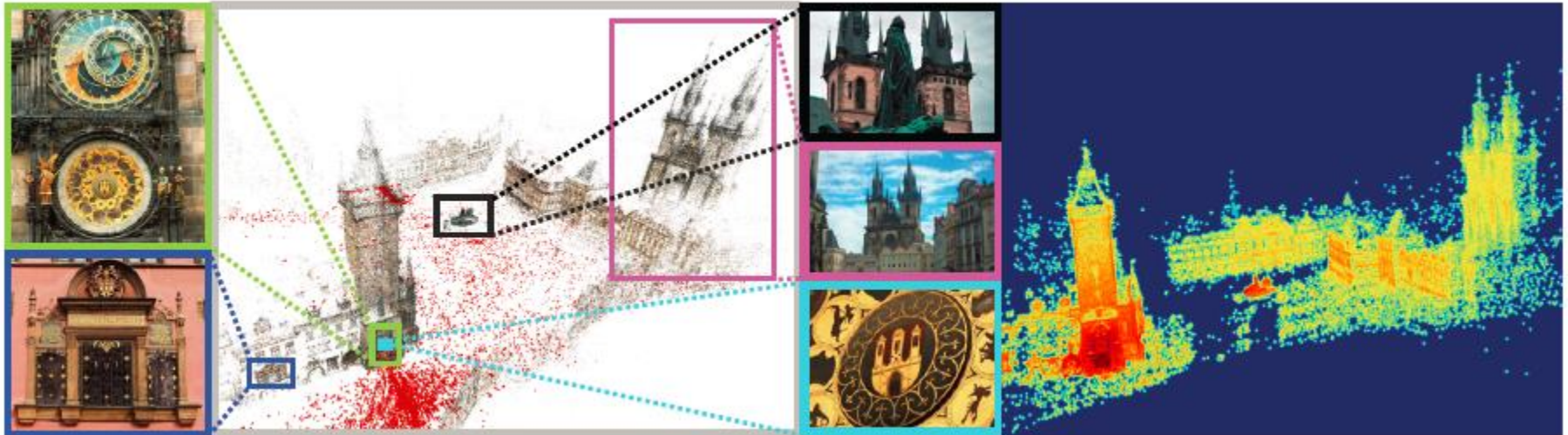


Schonberger J, Radenovic F, Chum O, Matas J. From Single Image Query to Detailed 3D Reconstruction. CVPR, 2015.

Correspondences in Action (2): Image Retrieval



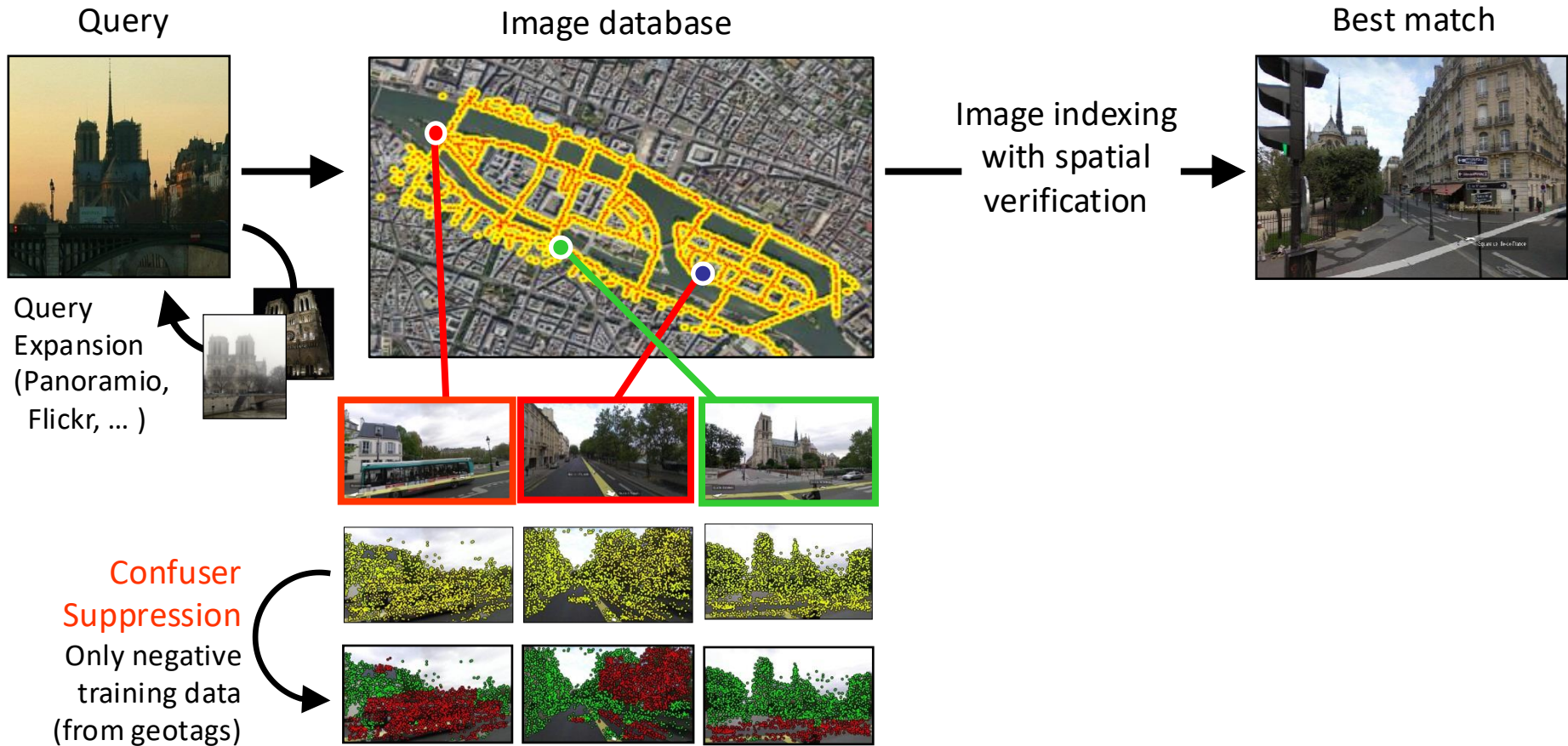
<https://youtu.be/Dlv1aGKqSIk>



Schonberger J, Radenovic F, Chum O, Matas J. From Single Image Query to Detailed 3D Reconstruction. CVPR, 2015.

Correspondences in action(3): Localization and Mapping

■ Place recognition - retrieval in a structured (on a map) database



[Knopp, Sivic, Pajdla, ECCV 2010] <http://www.di.ens.fr/willow/research/confusers/>

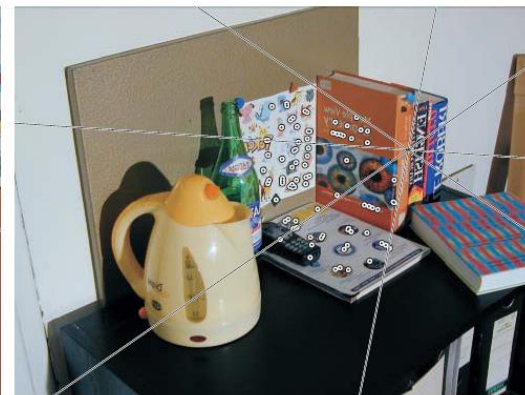
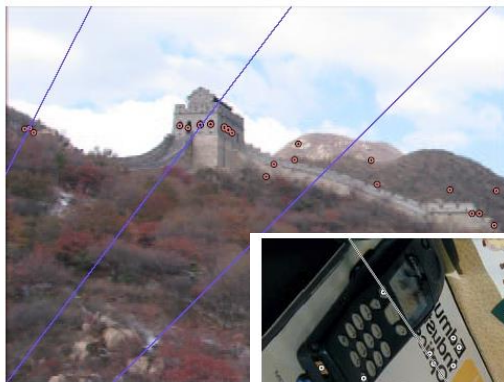
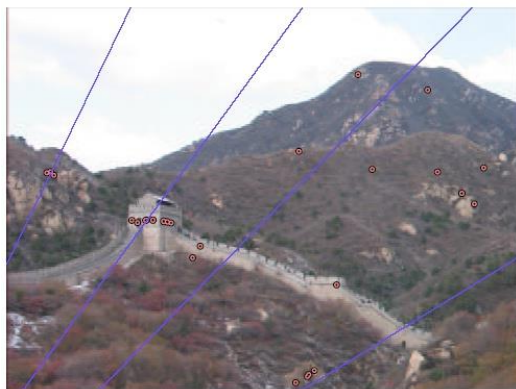
Challenges in the Correspondence Problem

Why is Establishing Correspondence Difficult?

Widening of baseline or zooming in/out

- local deformation is well modelled by affine or similarity transformations
- how can the “local feature” concept be generalized?

The set of ellipses is closed under affine tr., but it's too big to be tested window scanning approach becomes computationally difficult.



Finding correspondences is not easy

due to large viewpoint
change (including scale)

=>

**the wide-baseline
stereo problem**



Applications:

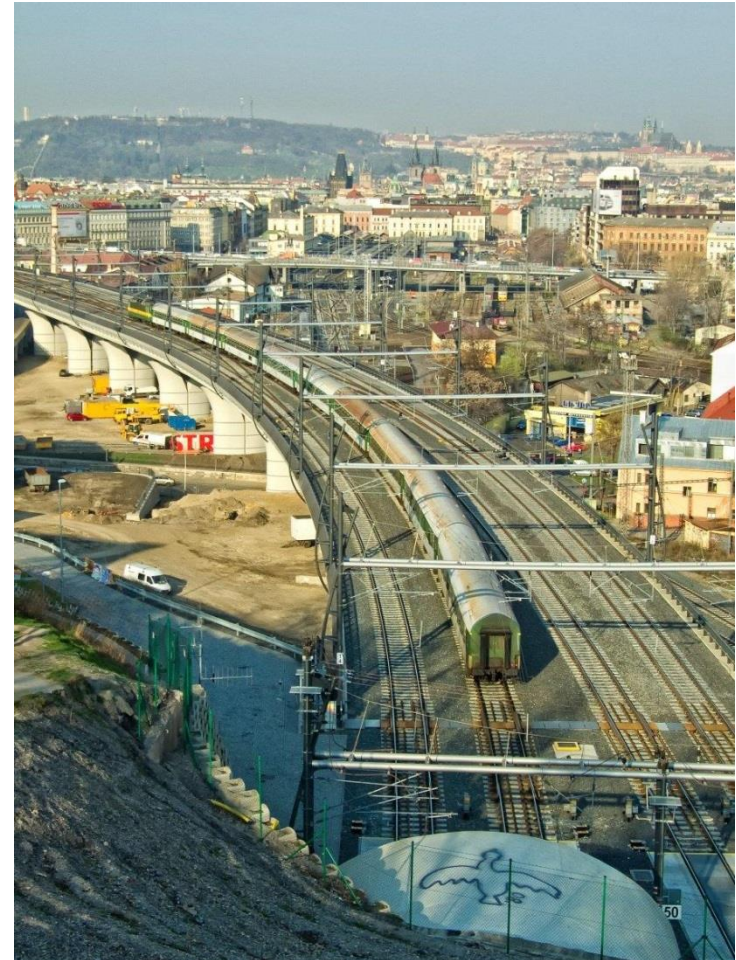
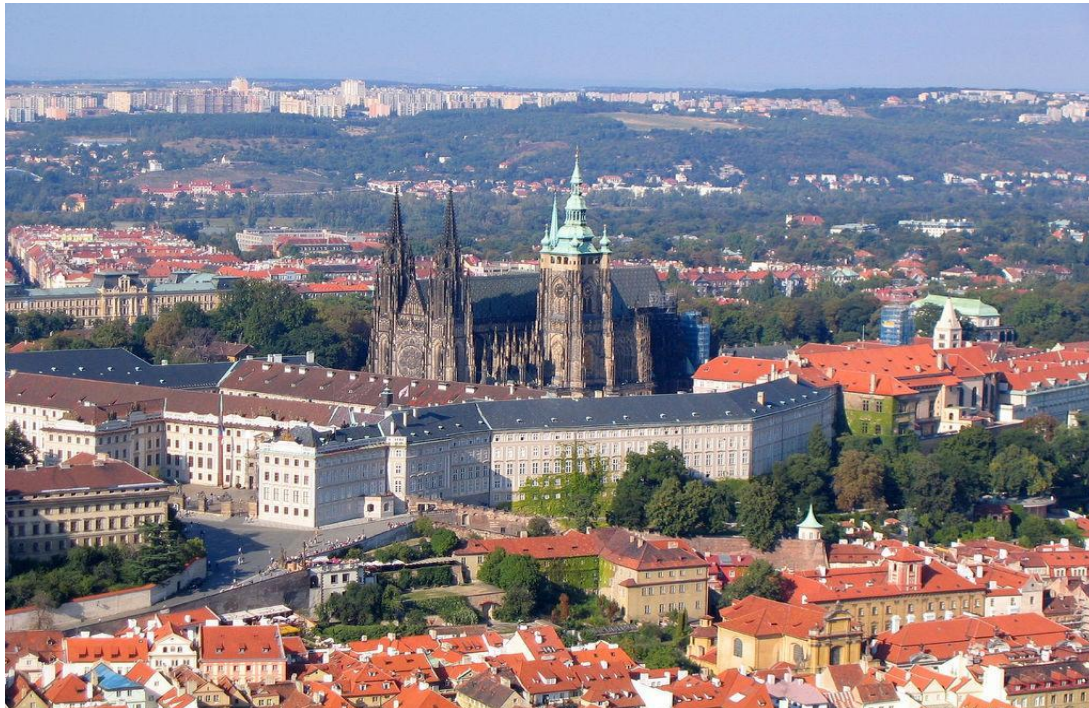
- pose estimation
- 3D reconstruction
- location recognition

Finding correspondences is not easy

due to large viewpoint change
(including scale)

=>

**the wide-baseline (WBS)
stereo problem**



Finding correspondences is not easy

due to large
illumination change

=>

wide “illumination-baseline”
stereo problem

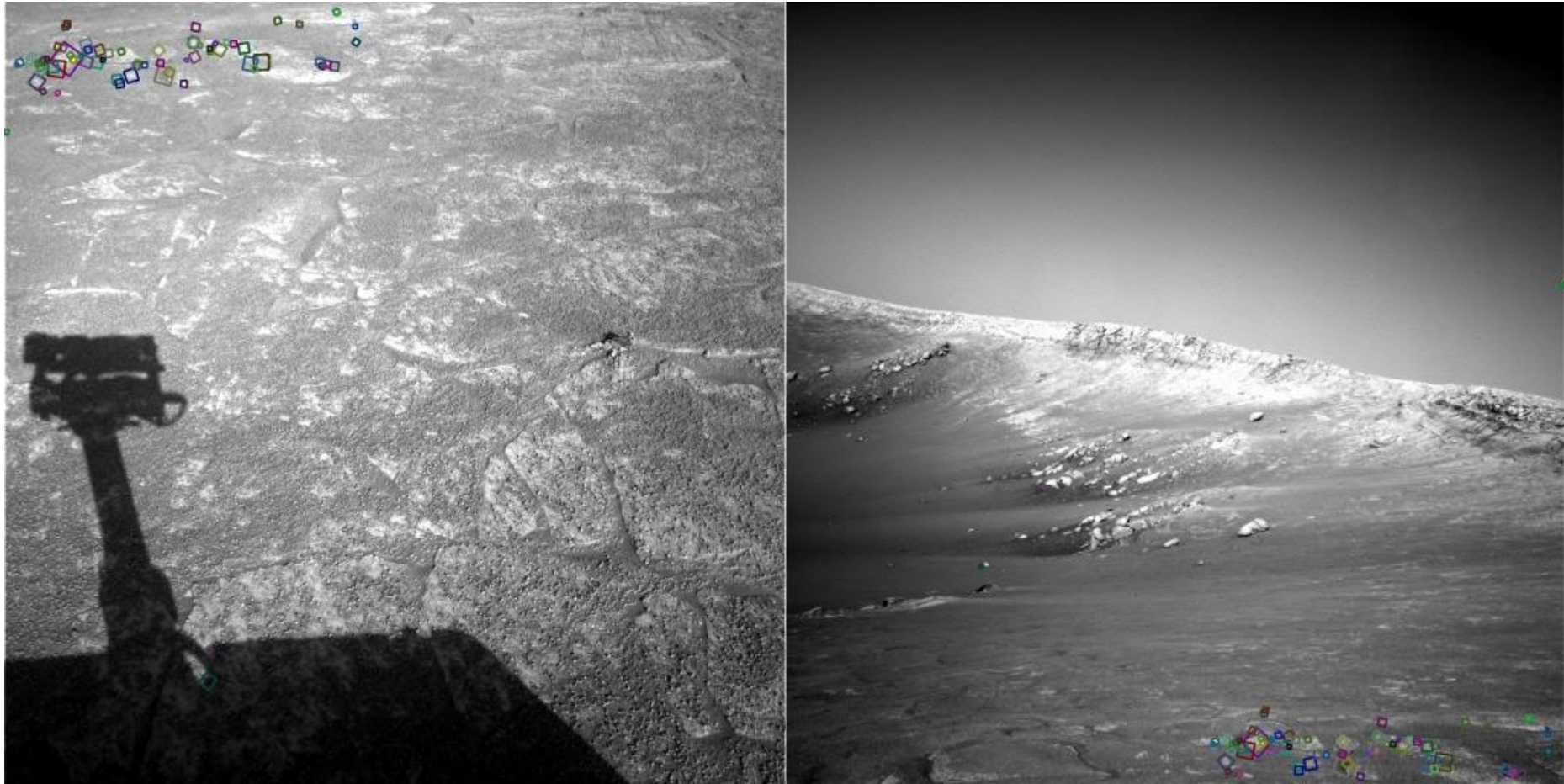


Applications:

- location recognition
- summarization of image collections

Fernando Zarur - fzarur@gmail.com

Find the matches (look for tiny colored squares...)



NASA Mars Rover images with SIFT feature matches
Figure by Noah Snavely

Finding correspondences is not easy

due to large
time difference

=>

wide temporal-baseline
stereo problem



Applications:

- historical reconstruction
- location recognition
- photographer recognition
- camera type recognition

Finding correspondences is not easy

due to occlusion



Applications:

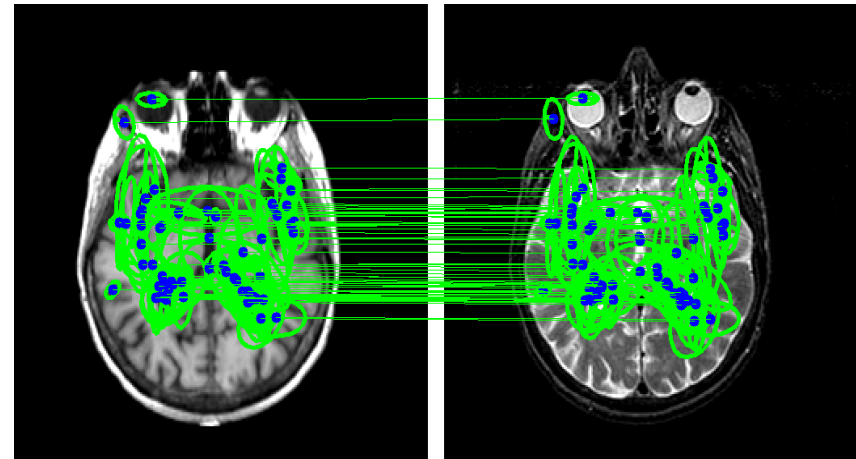
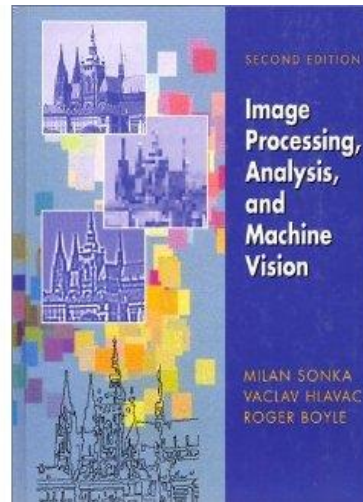
- pose estimation
- inpainting

Finding correspondences is not easy

change of
imaging modality

Applications:

- medical imaging

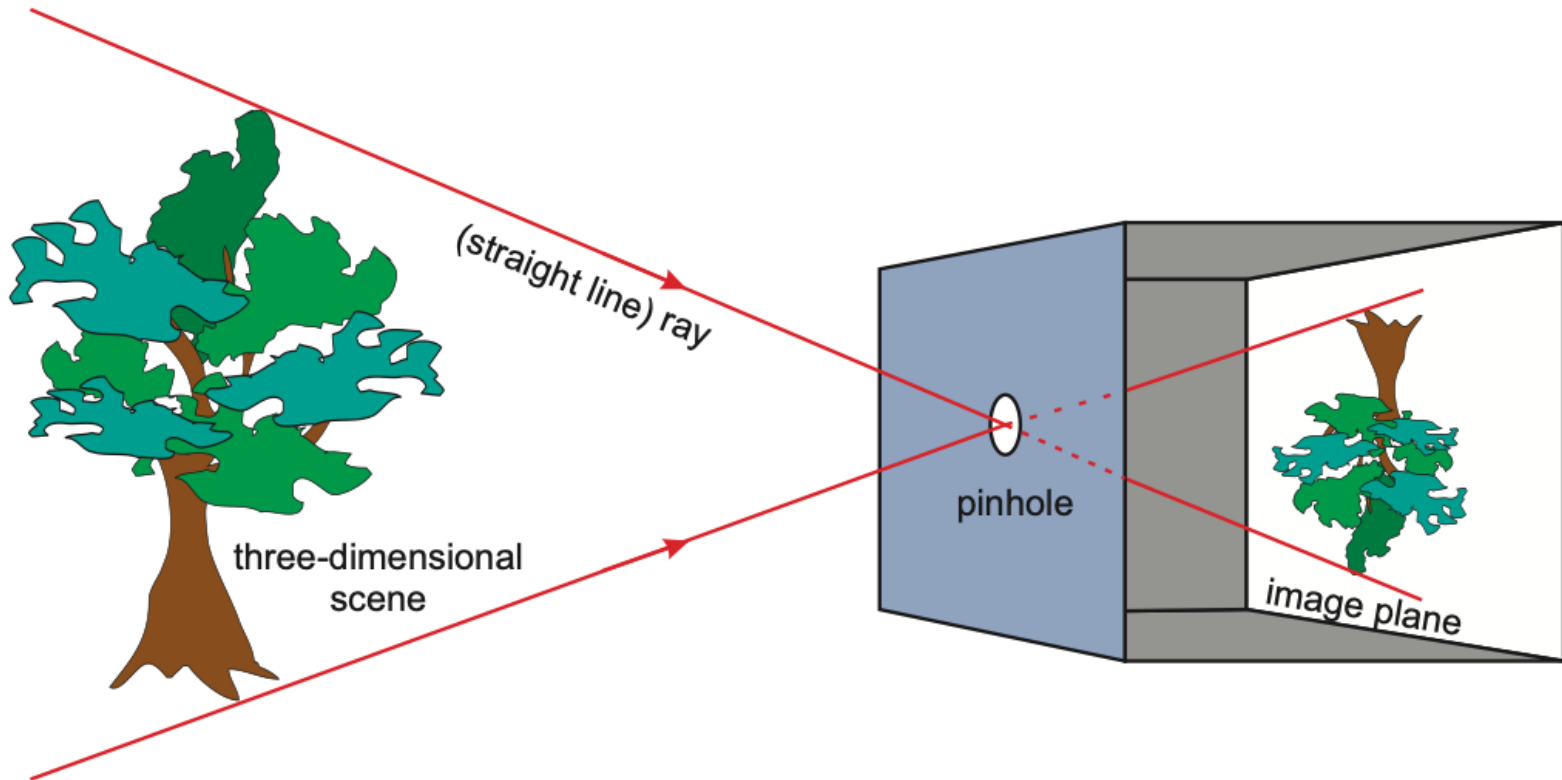


Finding image correspondences in practice

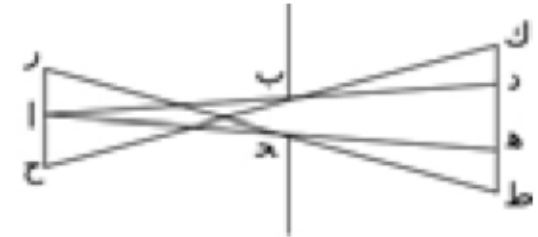
Image formation

And low-level image processing

- The **image** is created on the retina or light sensitive chip in a camera, . . .
- We assume a **perspective projection** corresponding to the pinhole model.



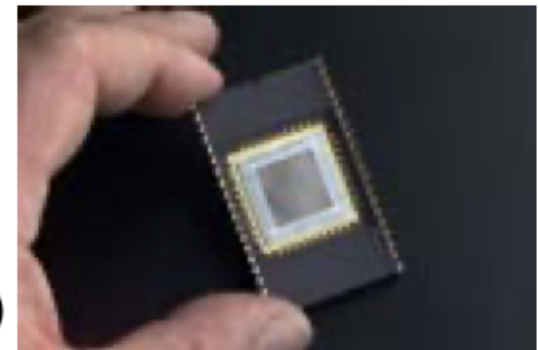
- **Pinhole model:** Mozi (470-390 BCE), Aristotle (384-322 BCE)
- **Principles of optics (including lenses):** Alhacen (965-1039 CE)
- **Camera obscura:** Leonardo da Vinci (1452-1519), Johann Zahn (1631-1707)
- **First photo:** Joseph Nicéphore Niépce (1822)
- **Daguerreotypes** (1839)
- **Photographic film** (Eastman, 1889)
- **Cinema** (Lumière Brothers, 1895)
- **Color Photography** (Lumière Brothers, 1908)
- **Television** (Baird, Farnsworth, Zworykin, 1920s)
- **First consumer camera with CCD:** Sony Mavica (1981)
- **First fully digital camera:** Kodak DCS100 (1990)



Alhacen's notes



Niepce, "La Table Servie," 1822



CCD chip

Slide credit: L. Lazebnik

- **Image** (continuous) is formed on the sensor surface (retina, film, CCD, CMOS). This is an **image function** $f(x, y)$
 - $f(x, y, t)$ in the case of an **image sequence**, t is time.
- $f(x, y)$ – **value of the image function**, depending on:
 - light sources, the scene, materials, etc.
 - and lens & optics, but we ignore them in this lecture
- After discretization, a **digital image** is a matrix of pixels.
 (x_i, y_i) are spatial coordinates of a pixel, with intensity $I(x_i, y_i)$.
 x_i, y_i, I are integer-valued.
- **2D images, not formed by a perspective camera**:
a thin specimen in the optical microscope, a scanned image of a letter (character) on a piece of paper, a fingerprint, one slice in the tomograph, etc.

Physical pixel in camera (Bayer filter)

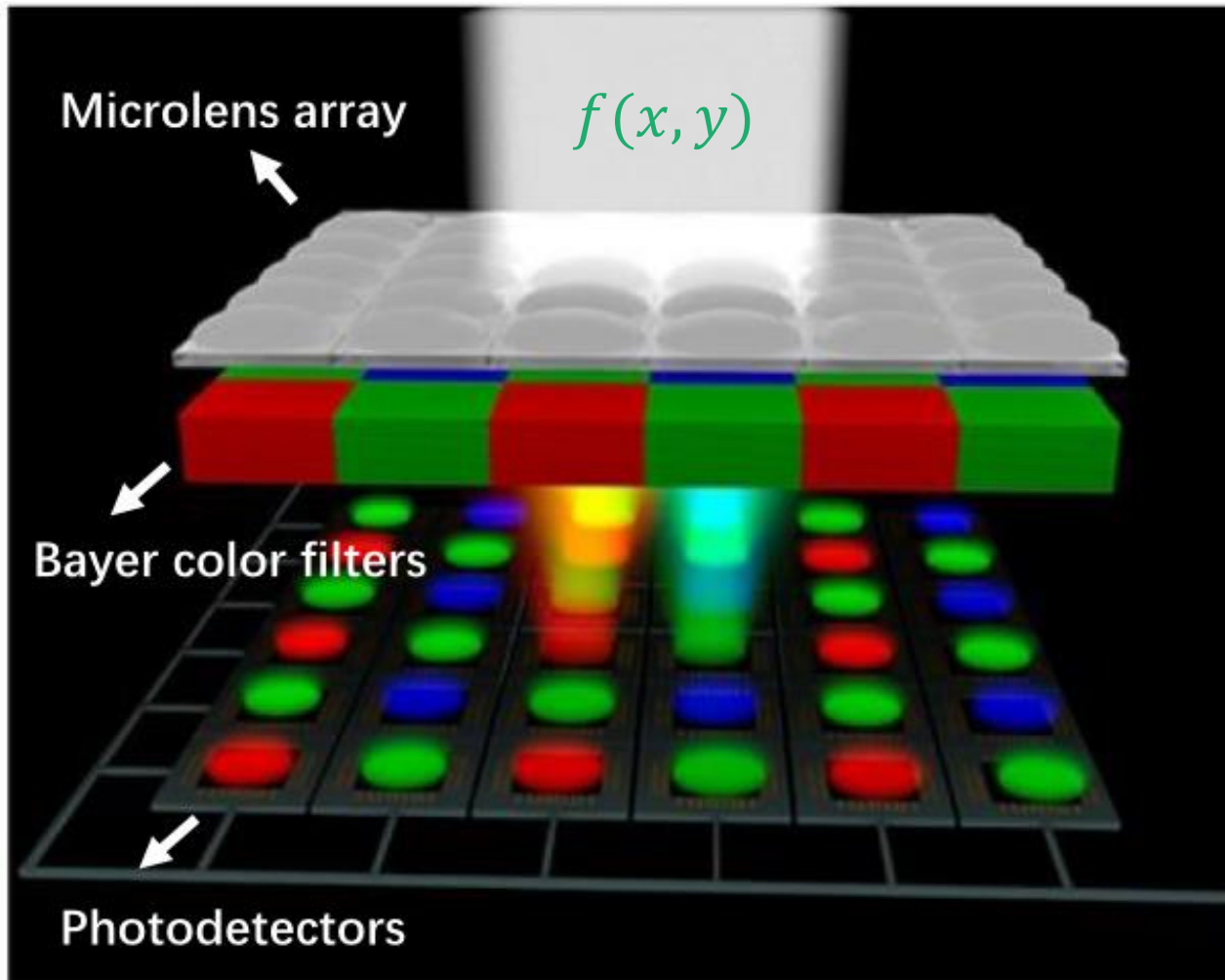


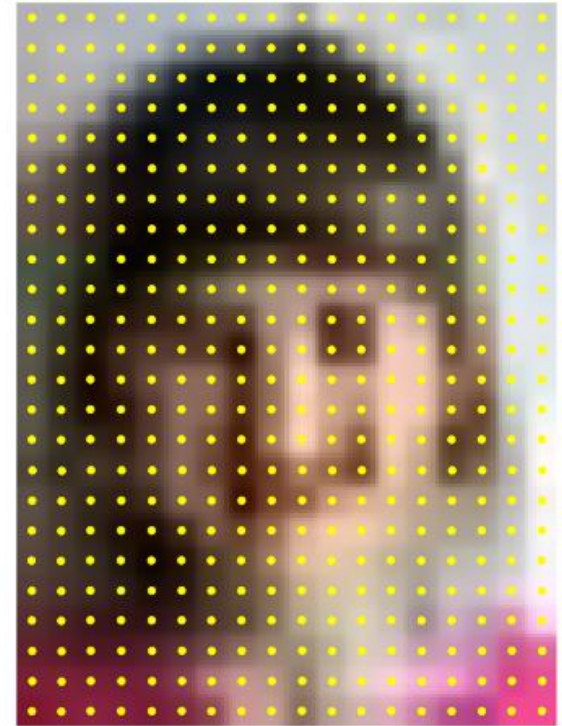
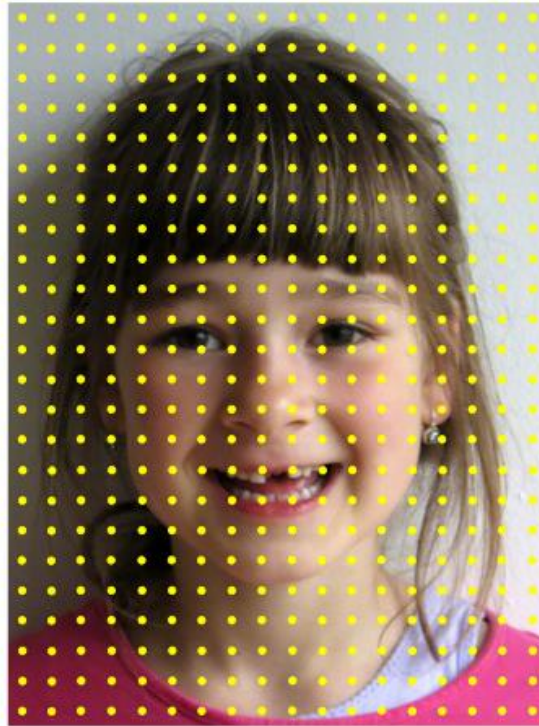
Image sources: <https://www.nature.com/articles/s41467-022-31019-7>

The continuous image is:

- not sampled at a single point
- not square-tiled

$I(x_i, y_i)$
after A-D conversion

Pixel as we display



Pixel – spatial discrete sample $I(x_i, y_i)$ of the continuous image function $f(x, y)$.

For displaying, we often model pixel as a constant-colour square. Depending on the goal, pixel can be modelled as

- a square
- an infinitely small (point) sample
- point sample after blurring the image, etc

- Discretization = quantizing of the image function value (called also intensity).
 - Quantizing divides a real value of the sample into a finite number of values (also bins). It can be 256 gray-level values for grayscale image. That is what you get in RAW.



gray wedge



6 gray-level bins

- Lots of additional processing is usually done: gamma-correction, de-Bayering, white-balancing.

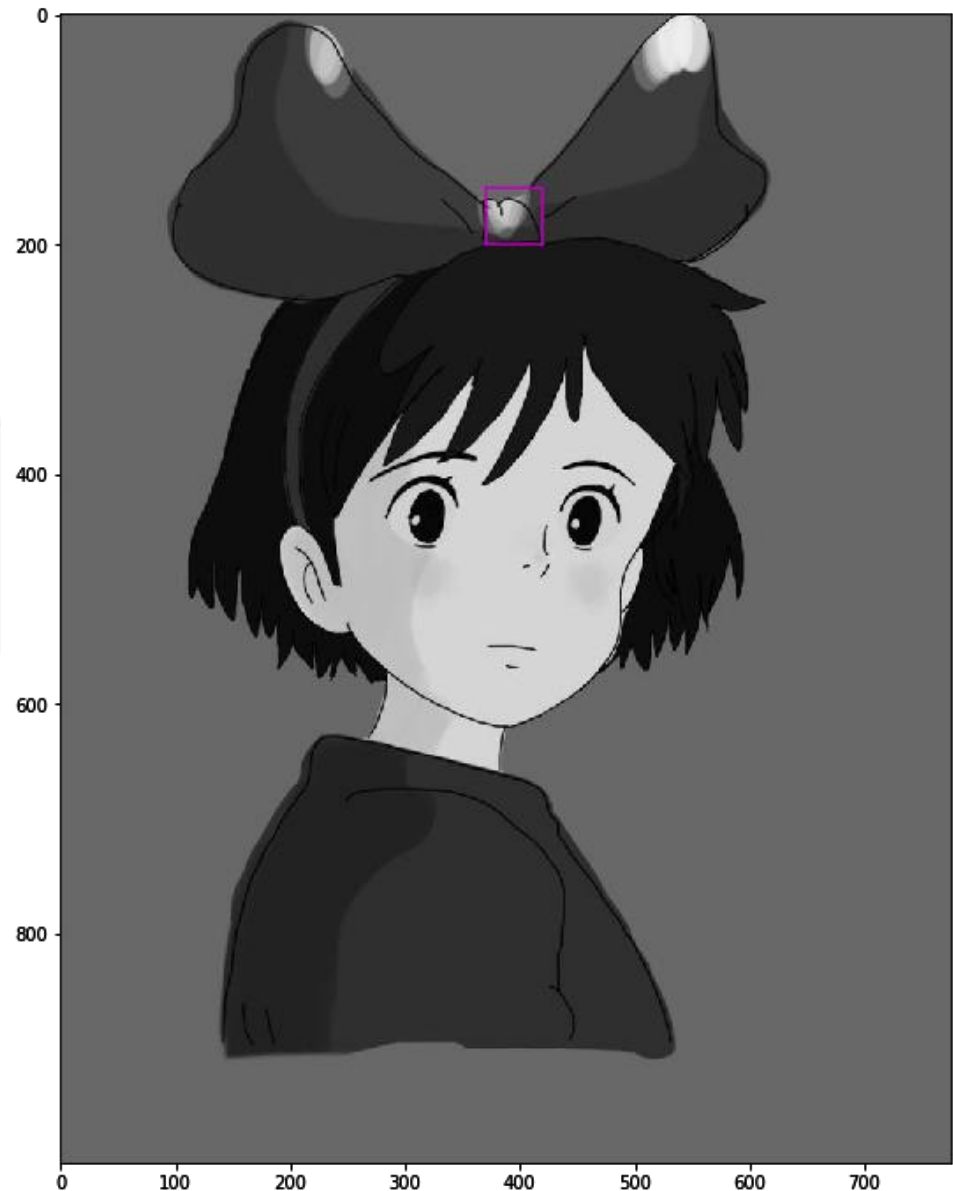
Reading & displaying images in Python with OpenCV and Matplotlib

```
▶ fname = 'kiki.jpg'  
img_gray = cv2.imread(fname, 0)  
plt.figure(figsize=(16,12))  
plt.imshow(img_gray, cmap='gray')
```

```
▶ patch_x, patch_y = 370, 150  
patch_w, patch_h = 50, 50  
patch = img_gray[patch_y:patch_y+patch_h,  
                  patch_x:patch_x+patch_w]  
print (patch)
```

```
[[102 102 102 ... 60 60 59]  
 [102 102 102 ... 60 60 59]  
 [102 102 102 ... 60 59 59]  
 ...  
 [ 48 44 41 ... 12 32 43]  
 [ 48 48 46 ... 7 26 32]  
 [ 44 47 45 ... 9 23 21]]
```

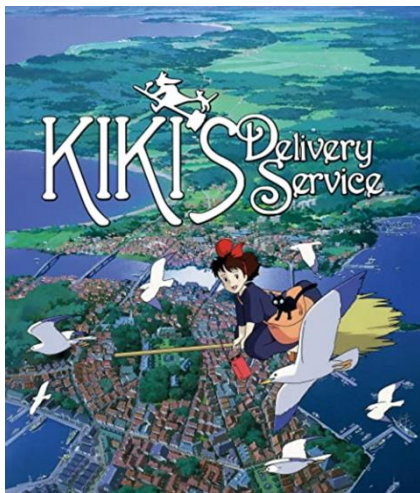
You can run yourself [here](#)



Kiki's Delivery Service

Color (RGB) image

```
▶ fname = 'kiki.jpg'  
img = cv2.imread(fname)  
plt.figure(figsize=(16,12))  
plt.imshow(img)
```



Oops!

```
▶ img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)  
plt.figure(figsize=(16,12))  
plt.imshow(img)
```



Color (RGB) image

```
print (img.shape)
```

```
(1000, 776, 3)
```

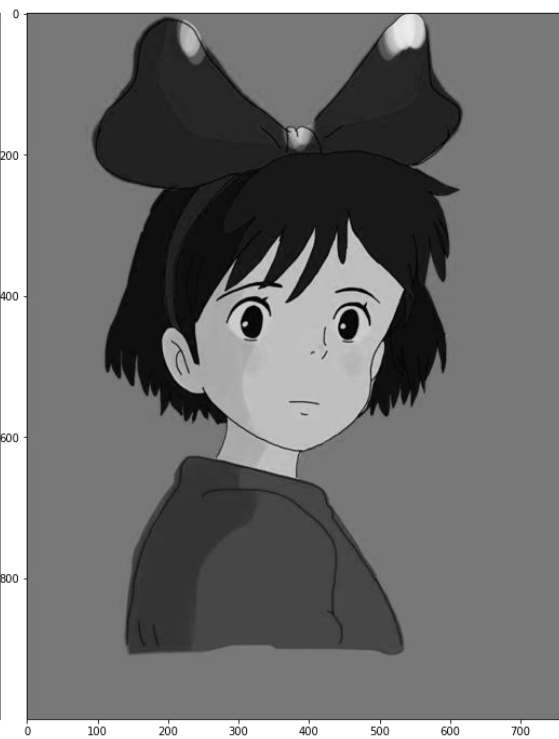
```
▶ for i in range(3):  
    plt.figure(figsize=(16,12))  
    plt.imshow(img[...,i], cmap='gray')
```



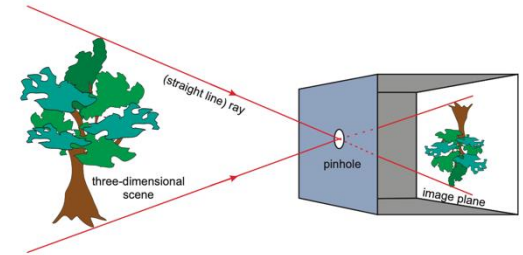
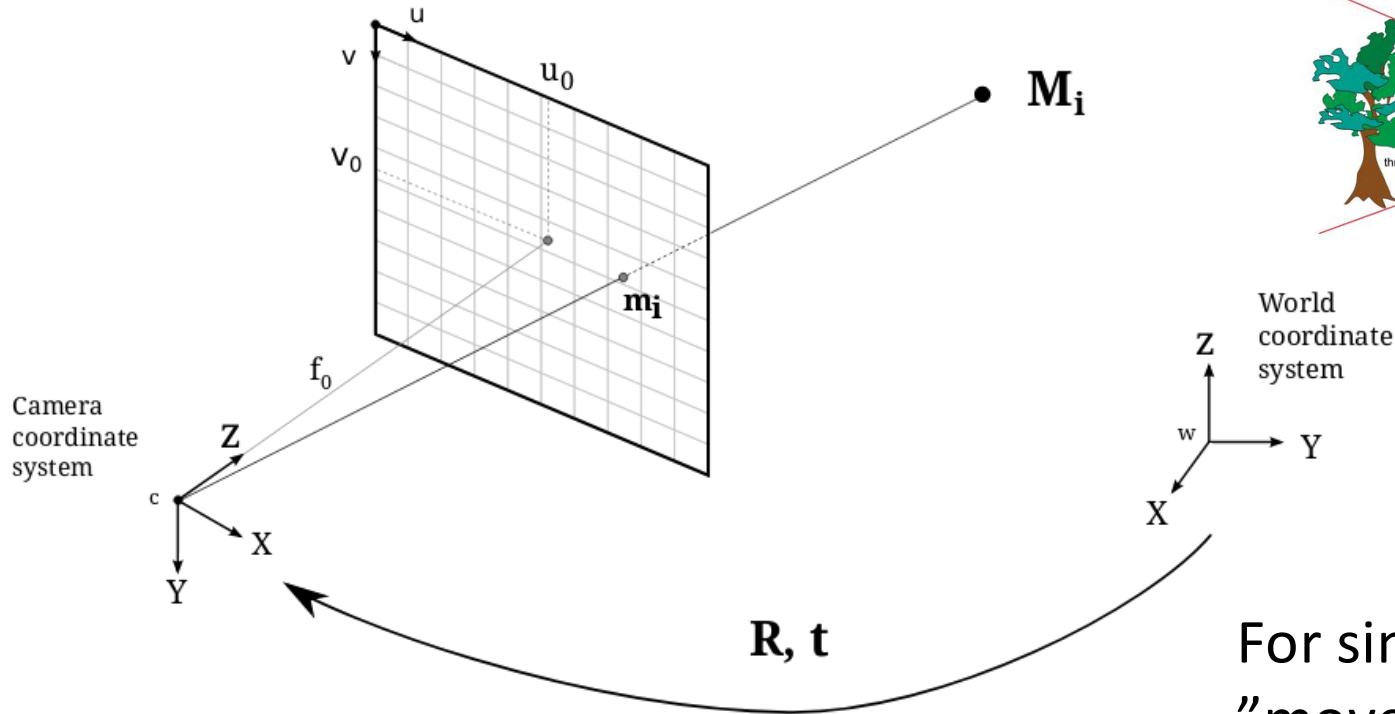
Red

Green

Blue



Mathematical Model of Pinhole Camera



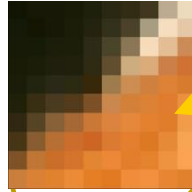
For simplicity, we "move" image to the front of the pinhole and not upside-down

https://en.wikipedia.org/wiki/Camera_matrix

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} f_x & 0 & u_0 \\ 0 & f_y & v_0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_1 \\ r_{21} & r_{22} & r_{23} & t_2 \\ r_{31} & r_{32} & r_{33} & t_3 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

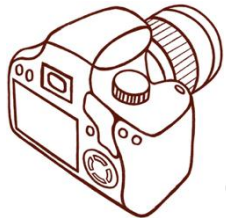
The Geometric Correspondence Problem

$O_{1,k} = \{x, d\}$ – observation
 $x_{1,k} = (2290, 780)$ – spatial info
 $d_{1,k} = (17, 21, 4)$ – descriptor



View

$$V_1 = \left\{ \bigcup_{k=1}^n O_{1,k}, \Theta_1 \right\}$$



$$\Theta_1 = \left\{ \begin{array}{l} (R_1, t_1), K_1, \\ 10.12.2021, \\ \text{Xiaomi Mi 9,} \\ \text{daylight,} \\ \text{deep field of view} \end{array} \right\}$$

Sensor information

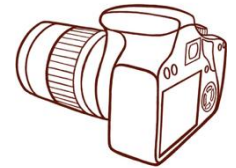
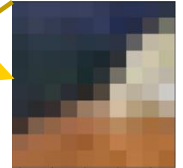


$M: f(V, O, C, \Omega) < \epsilon$
 World model

$$x_k^T F_{12} x_l < \epsilon_{\text{epi}}$$

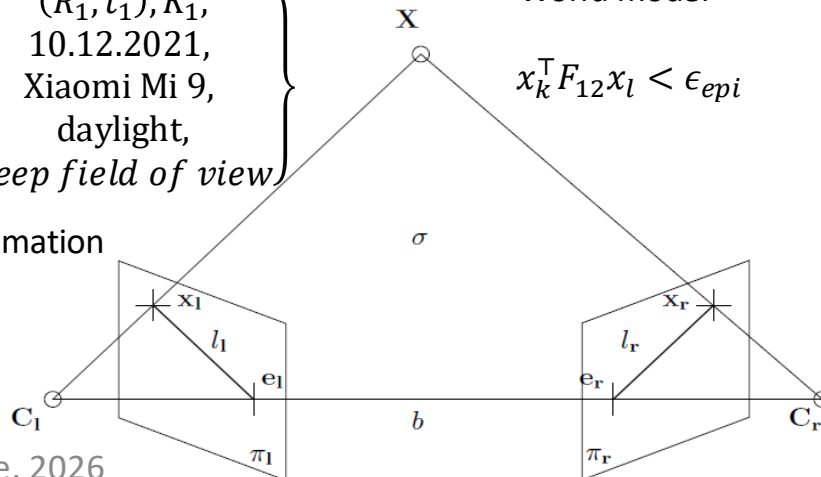
$C_{1,k,2,l}$ – correspondence

$O_{2,l}$



V_2

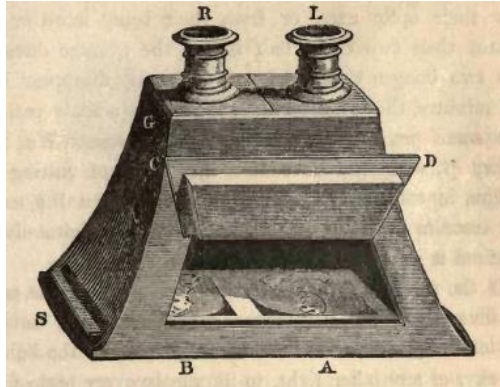
$$\Theta_2 = \left\{ \begin{array}{l} (R_2, t_2), K_2, \\ 11.12.2021, \\ \text{iPhone 11,} \\ \text{fairly lights,} \\ \text{narrow field of view} \end{array} \right\}$$



Finding image correspondences in practice

Narrow-baseline stereo has a long history

Narrow-baseline stereo

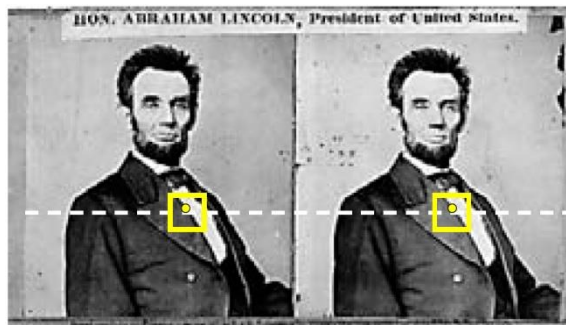


Brewster
Stereoscope, 1856

A “photo” for both
eyes

Local Feature (Region) = a rectangular “window”

- robust to occlusion, translation invariant
- windows matched by correlation, assuming small displacement
- successful in narrow-baseline stereo matching



Correspondences in Action (1): Building a Panorama

- We need to match and align images = find (dense) correspondences
- Note: this can be done only if both images were taken from the same viewpoint



■ Problem 1:

- Detect the *same* feature *independently* in both images^{*}
- Note that the set of “features” may be rather sparse



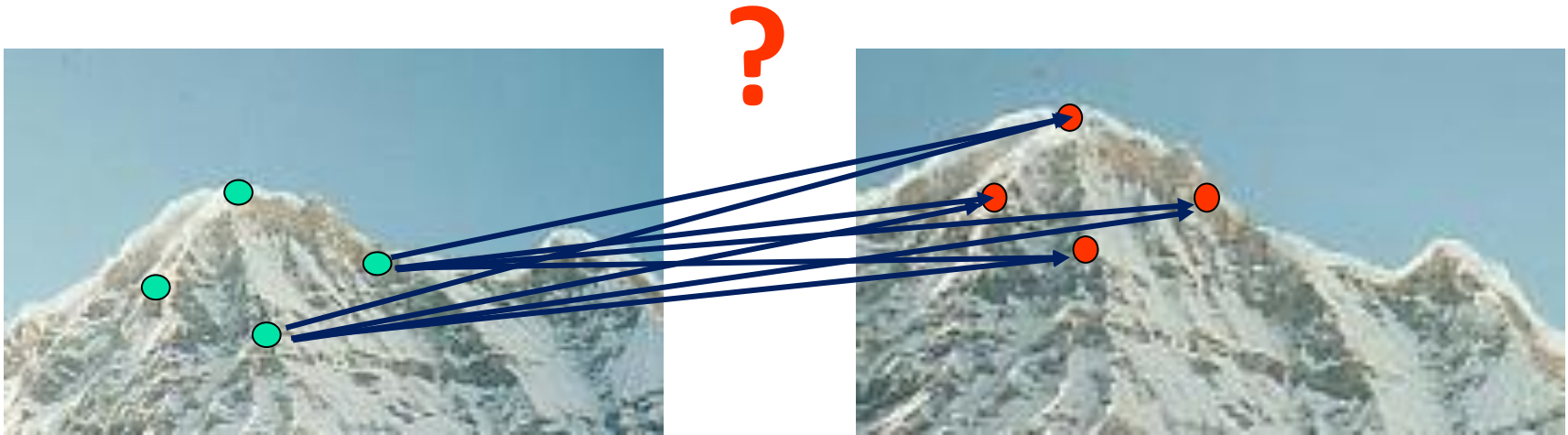
no chance to match!

A repeatable detector needed.

^{*} Some existing methods operate on both images simultaneously

■ Problem 2:

- how to correctly recognize the corresponding features?

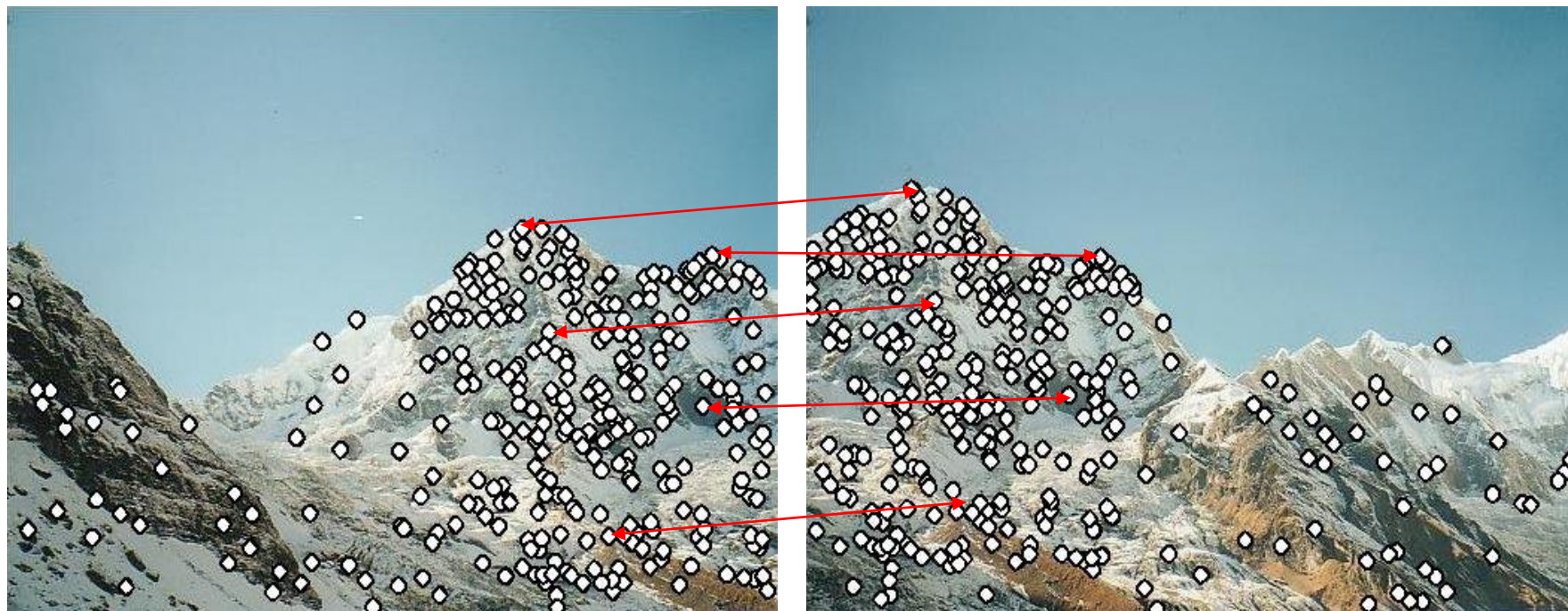


Solution:

1. Find a discriminative and stable descriptor
2. Solve the matching problem

Possible approach:

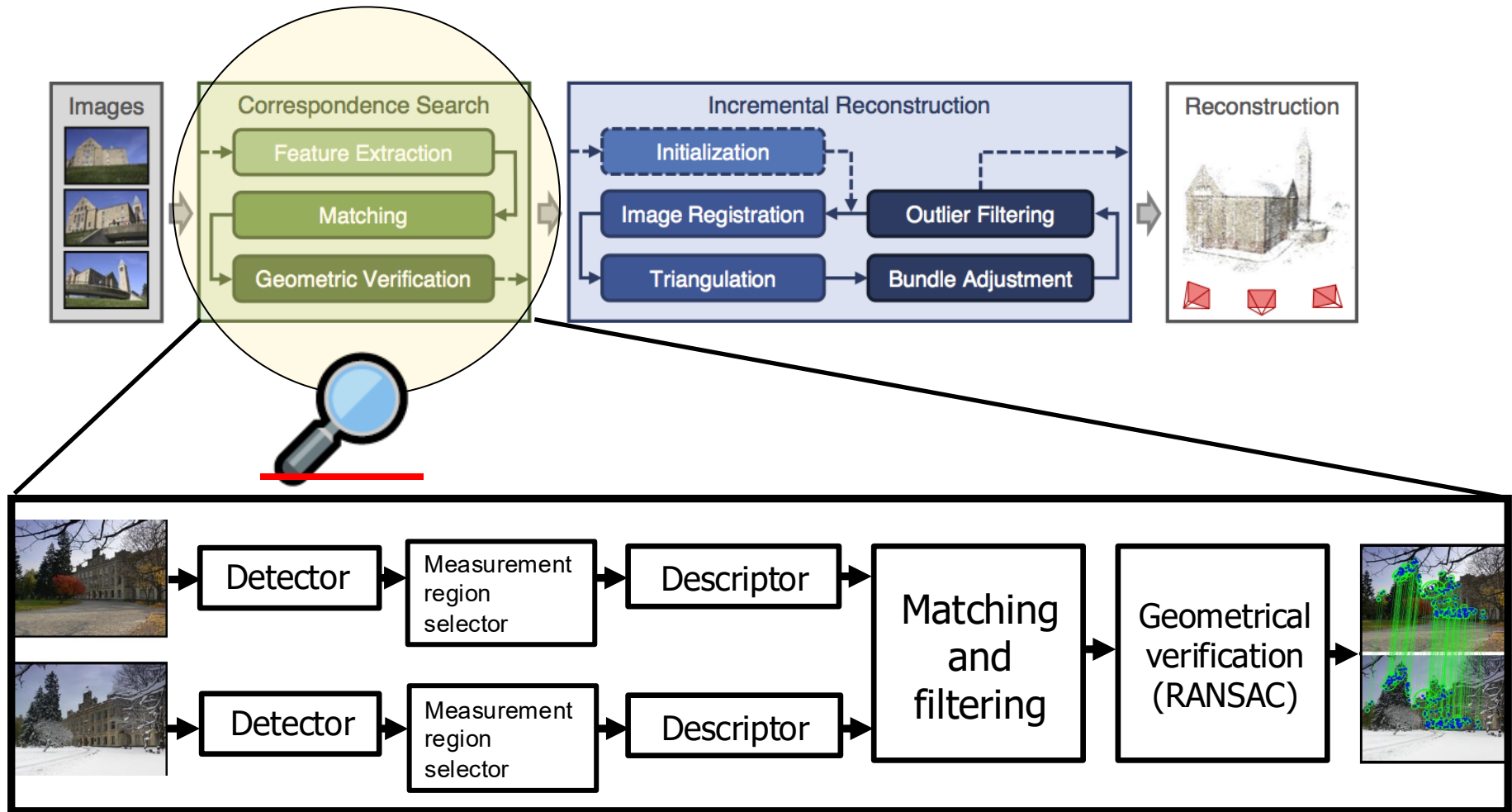
1. Detect features in both images
2. Find corresponding pairs
3. Estimate transformations (Geometry and Photometry)
4. Put all images into one frame, blend.



1. Detect features in both images
2. Find corresponding pairs
3. Estimate transformations (Geometry and Photometry)
4. Put all images into one frame, **blend**.



Image Matching Pipeline



Wide baseline stereo pipeline

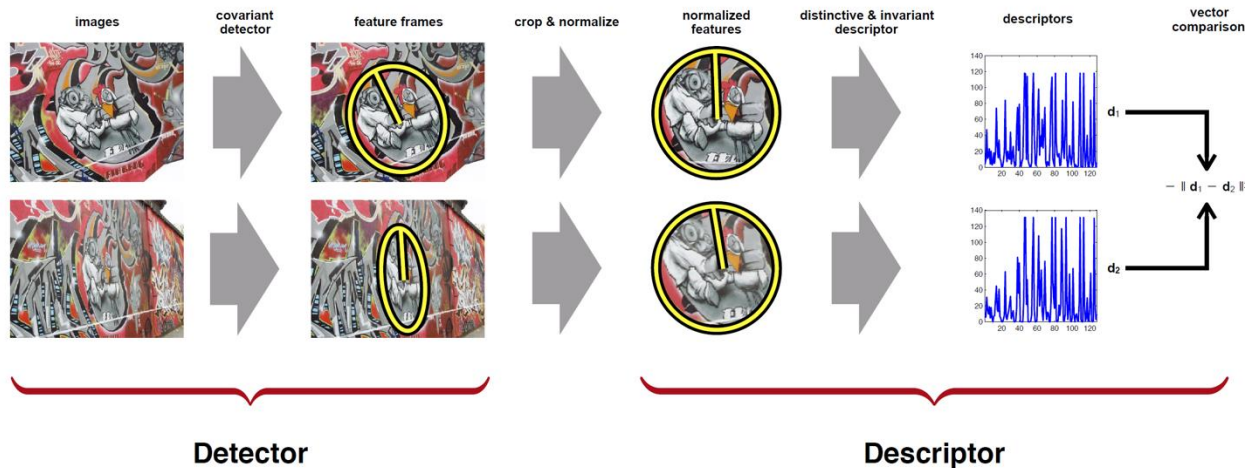
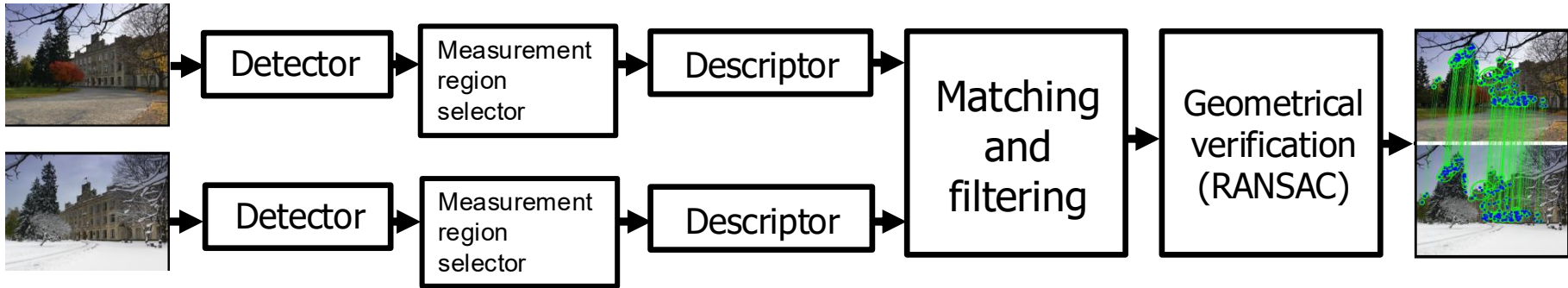


Image credit: Andrea Vedaldi, ICCW 2017

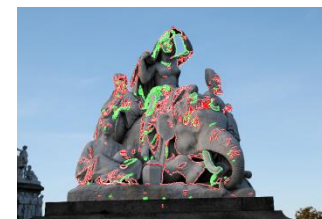
Correspondences are useful for 3D reconstruction

1. matching distinguished regions

⇒ tentative correspondences
(verification)

⇒ two view geometry

In this
lectures



2. camera calibration

⇒ camera positions

⇒ sparse reconstruction

3. dense stereoscopic matching

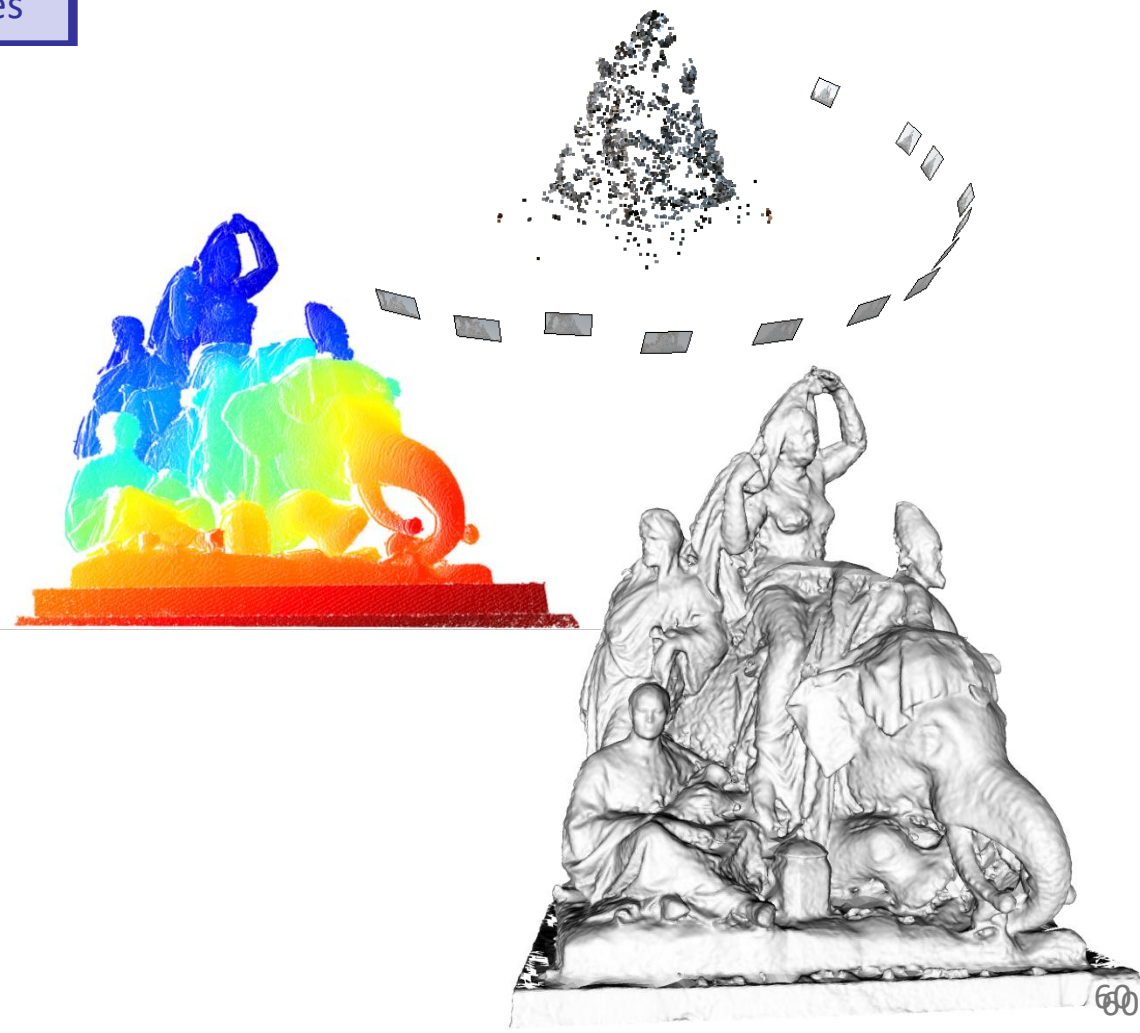
⇒ pixel/sub-pixel matching

⇒ depth maps, 3D point cloud

4. surface reconstruction

⇒ surface refinement

⇒ triangulated 3D model



Matas et al., IVC 2004.

Martinec, Pajdla., CVPR 2007.

Cech, Sara, CVPR 2007.

- Methods based on “Local Features” are the state-of-the-art for number of computer vision problems.
 - e.g. Wide-baseline stereo, 3D reconstruction
- Terminology (diverse, unfortunately) :
 - Local Feature = Interest “Point” = The “Patch” =
 - = Feature “Point”
 - = Distinguished Region
 - = (Transformation) Covariant Region

“Local Features” are **regions**, i.e. in principle arbitrary sets of pixels (not necessarily contiguous) with:

- High **repeatability**, (ideally invariance) under
 - Illumination changes
 - Changes of viewpoint => geometric transformations
i.e. are **distinguishable** in an image regardless of
viewpoint/illumination => are **distinguished regions**
- Are **robust to occlusion** => must be **local**
- Must have discriminative neighborhood => they are “**features**”

Methods based on local features/distinguished regions (DRs) formulate computer vision problems as matching of some representation derived from DR (as opposed to matching of entire images)

Two interpretations of the local features:

- “Detector”:
the task is to find all instances of the “object” == “good feature”, e.g. corner, blob, face.
- “Sorter”:
rank all the patches (distinguished regions) from the “best” to the worst, such that the ranking is consistent under nuisance factor. Then take top-K.



Keypoints here are the centers of the local features

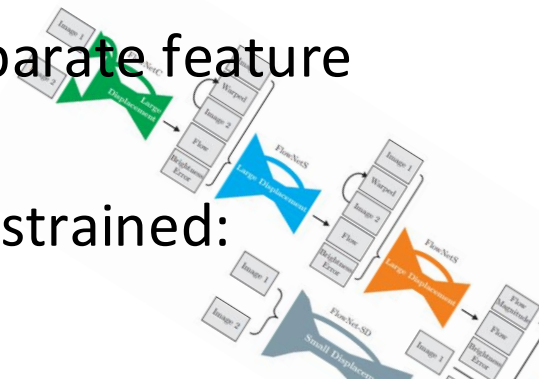
Drop the detector completely? Possible, but

The main reason we need the local feature detector is efficiency:

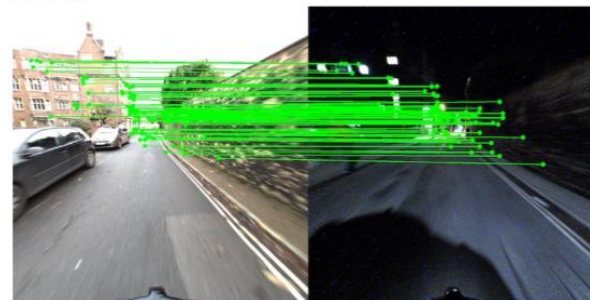
- Computation-wise: even small image contains A LOT of pixels, e.g. $640 \times 480 = 307\,200$. And even more regions (required for the descriptors), if we consider scale, orientation, etc.
- Memory-wise: storing $307k \times 128$ dim float descriptors = 157 Mb
- If any-to-any matching is possible, i.e. wide baseline setup, we have $307k \times 307k = 94$ Billion tentative correspondences

Drop the detector completely? Possible, but

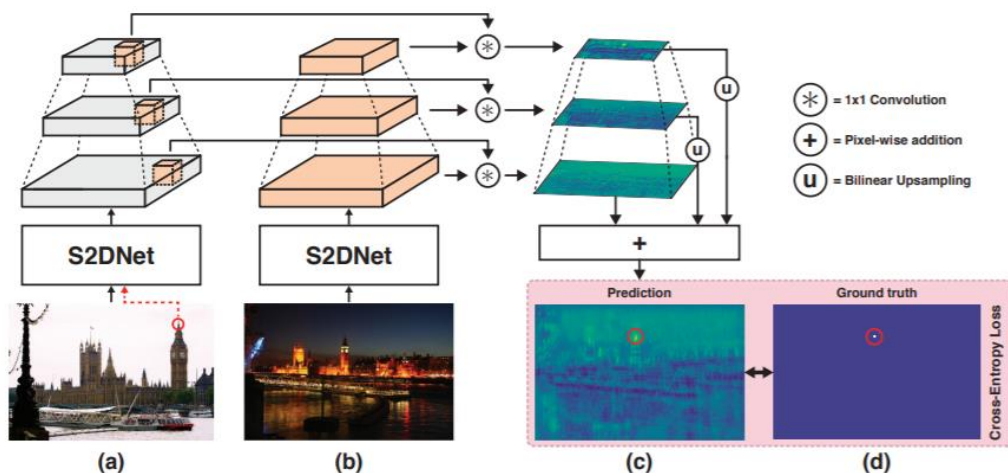
- Yet, if we can afford pairwise running (no separate feature extraction step)
- AND the correspondence search space is constrained:
 - Scene Flow (lecture in 2 month)
 - Optical Flow
 - Coarse-to-fine matching
 - Sparse-to-dense matching (asymmetric)



(a) Standard sparse-to-sparse matching using SuperPoint detections and two different descriptors : (Left) SuperPoint descriptors [4 inliers], (Right) HyperColumn descriptors [5 inliers]



(b) Our sparse-to-dense matching approach using Super-point detections in the left image only and HyperColumn descriptors [87 inliers]



Drop the detector completely? Possible, but

“Local Features” are **regions**, i.e. in principle arbitrary sets of pixels (not necessarily contiguous) with

High **repeatability**, (invariance in theory) under

Illumination changes

Changes of viewpoint => geometric transformations

Local feature detector is a form of attention

Methods based on local features/distinguished regions (DRs) formulate computer vision problems as matching of some representation derived from DR (as opposed to matching of entire images)

■ To summarize:

Classical local feature detectors have to be **designed** in a such way, to have the properties, we discussed before:

- Repeatability
- Distinguishability
- Locality
- Consistent ranking
- Robustness to the nuisance factors
- ...

Harris detector (1988) 22500 citations



undistinguished patches:



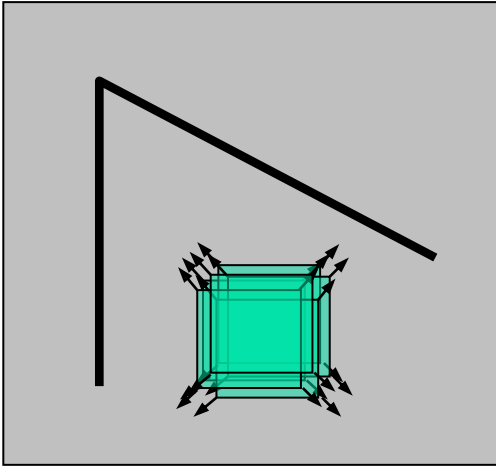
distinguished patches:



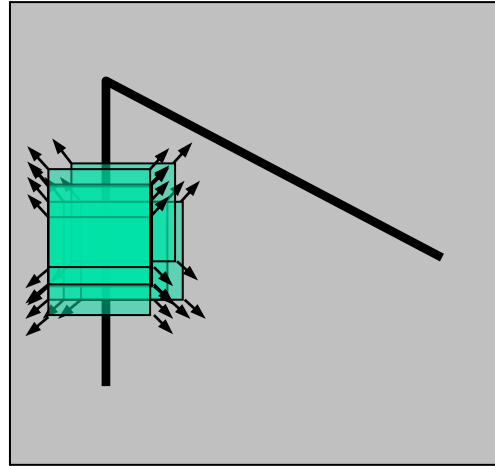
Two core ideas (in “modern terminology”):

1. To be a distinguished region, a region must be *at least* distinguishable from *all* its neighbours.
2. Approximation of Property 1. can be tested very efficiently, without explicitly testing.

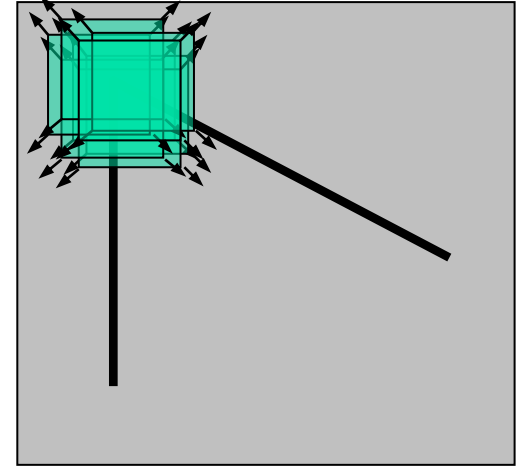
Note: both properties were proposed before Harris paper, (1) by Moravec, (1)+(2) by Foerstner.



“flat” region:
no change in
all directions



“edge”:
no change along
the edge
direction



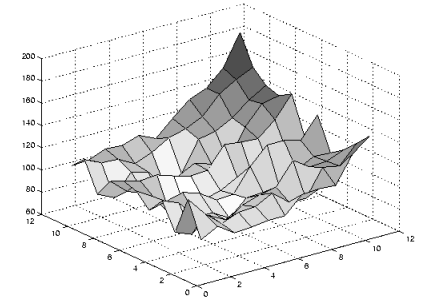
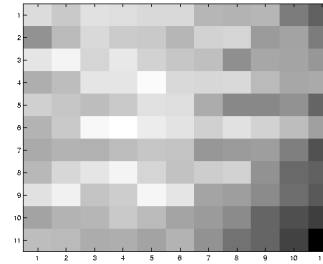
“corner”:
significant change
in all directions

- We should easily recognize the point by looking through a small window
- Shifting a window in *any direction* should give *a large change*

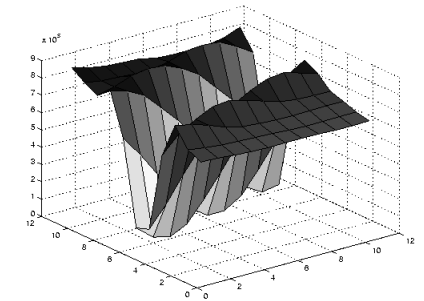
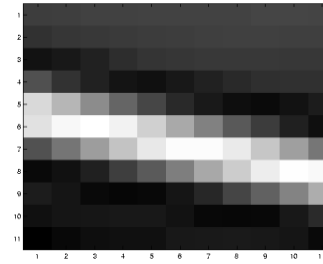
Harris Detector: Basic Idea



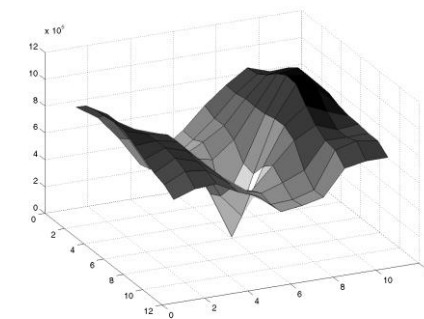
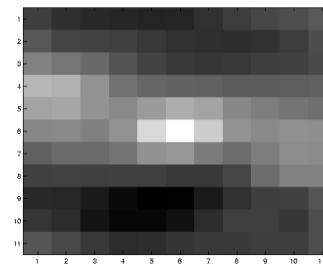
f1



f2



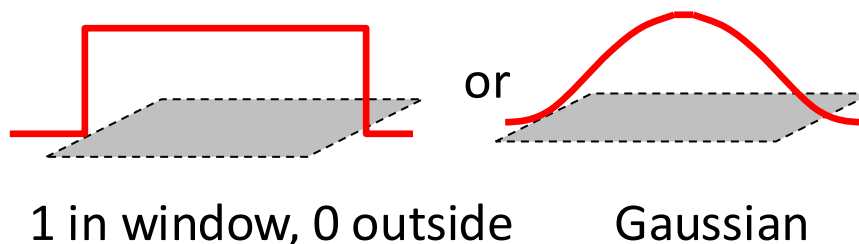
f3



Measure similarity of the image function $I(x_0, y_0)$ at point (x_0, y_0) of and its neighbors around $I(x_0 + u, y_0 + v)$ by weighted SSD:

$$E(x_0, y_0; u, v) = \sum_{(x, y) \in W(x_0, y_0)} w(x, y) (I(x, y) - I(x + u, y + v))^2 \quad (1)$$

- $W(x_0, y_0)$ is a window centered at point (x_0, y_0)
- $w(x, y)$ can be constant or (better) Gaussian



We are interested in “corner” points $I(x_0, y_0)$ where $E(x_0, y_0; u, v)$ is large for all (u, v) , i.e. $\min(E(x_0, y_0; u, v))$ is locally maximal.

$$E(x_0, y_0; u, v) = \sum_{(x,y) \in W(x_0, y_0)} w(x, y) (I(x, y) - I(x + u, y + v))^2$$

```
def ssd_min(img, ksize = 3, wsize=7):
    b, ch, h, w = img.size()
    out = torch.zeros(b, ch, ksize, ksize, h, w)
    pd = int(ksize/2)
    img_padded = F.pad(img, (pd, pd, pd, pd), mode='replicate')
    # u, v are possible shifts
    # wsize is the size of the window W
    for u in range(ksize):
        for v in range(ksize):
            diff_shift_sq = (img - img_padded[:, :, v:h+v, u:w+u]).pow(2)
            out[:, :, u, v, :, :] = K.gaussian_blur2d(diff_shift, (wsize, wsize), (1.0, 1.0))
    out[:, :, pd, pd, :, :] = 100000. # (pd, pd) is window center, so difference is 0
    out = out.min(dim=2)[0].min(dim=2)[0] # Minimum over x and y shifts
    return out
```

Code example
available [here](#)

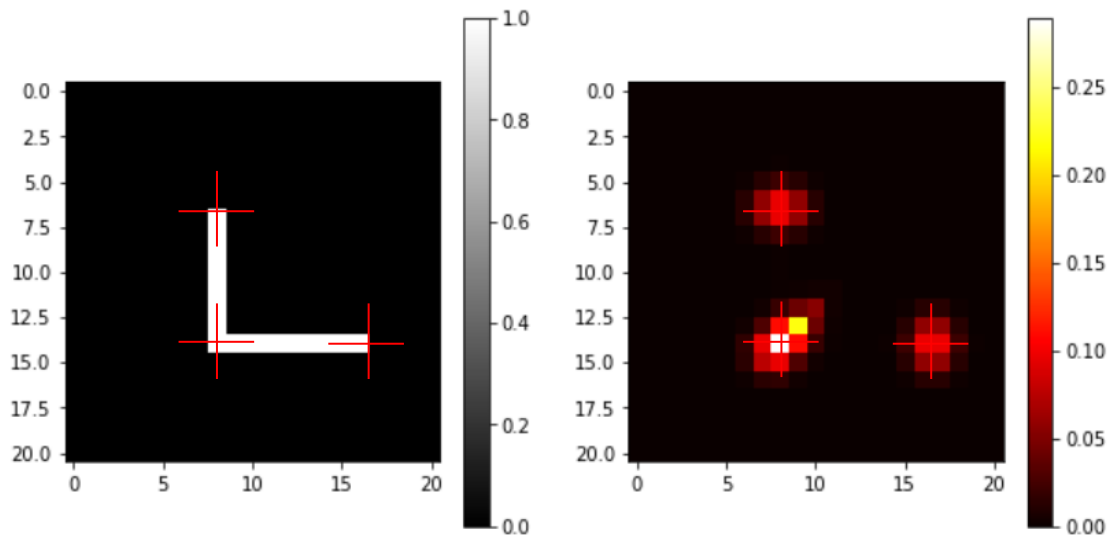
- $W(x_0, y_0)$ is a window centered at point (x_0, y_0)
- $w(x, y)$ can be constant or (better) Gaussian

$$E(x_0, y_0; u, v) = \sum_{(x,y) \in W(x_0, y_0)} w(x, y) (I(x, y) - I(x + u, y + v))^2$$

```
artificial_img = torch.zeros(1, 1, 21, 21)
artificial_img[:, :, 7:14, 8] = 1
artificial_img[:, :, 14, 8:17] = 1

in_img = artificial_img
out_img = ssd_min(in_img, 3, 7)
imshow_torch_channels(torch.cat([in_img, out_img], 0), 0, colorbar=True, cmap='hot')
```

Code example
available [here](#)



Input image

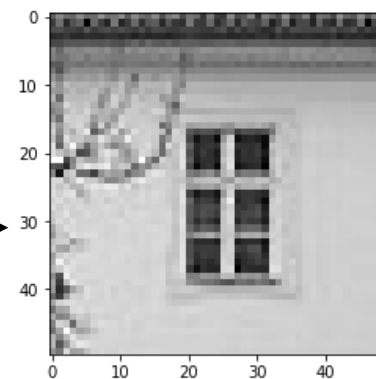
$\min_{(u,v)} E(x_0, y_0; u, v)$

Note, that response
is high not only at
the corners,
but also at the “L”
endpoints

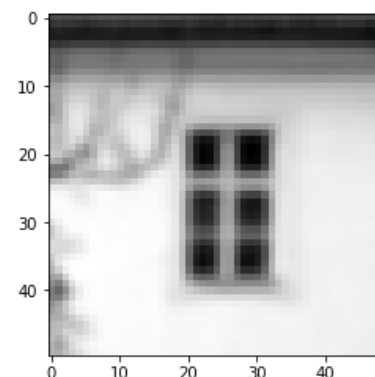
Harris Detector: Preliminaries (Moravec)



Code example
available [here](#)



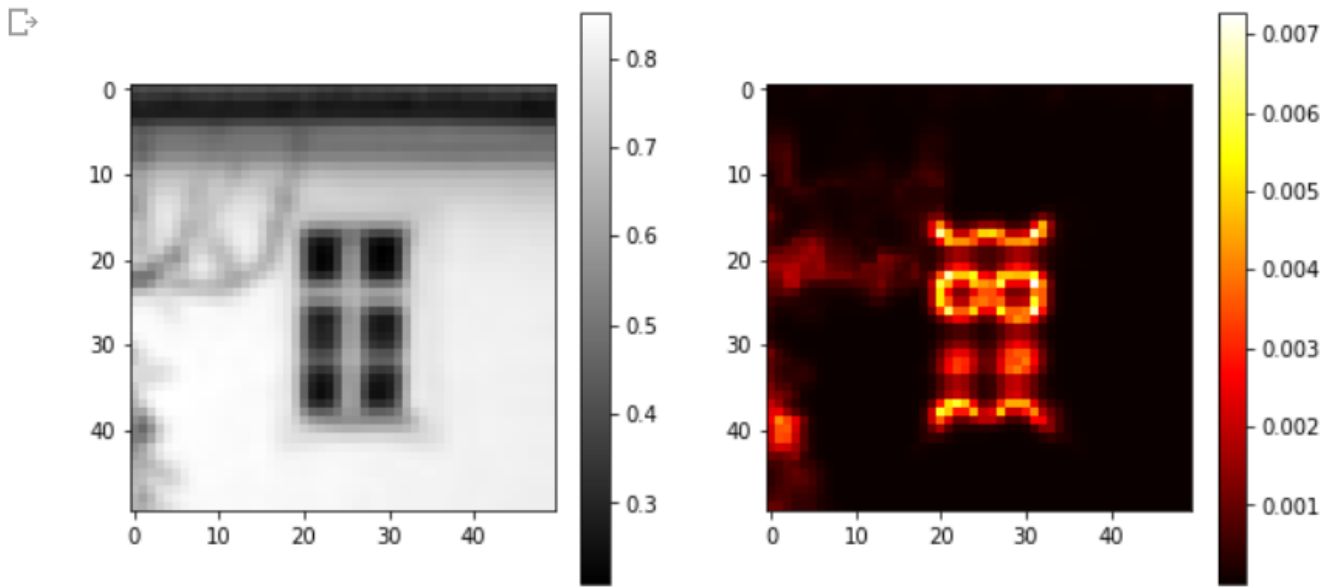
Gauss
filtering



$$E(x_0, y_0; u, v) = \sum_{(x,y) \in W(x_0, y_0)} w(x, y) (I(x, y) - I(x + u, y + v))^2$$

```
[18] in_img = tim_bluere.float()  
out_img = ssd_min(in_img, 3, 7)  
  
imshow_torch_channels(torch.cat([in_img, out_img], 0)[:,:,: , 80:130, 260:310], 0,  
                      colorbar=True, cmap='hot')
```

Code example
available [here](#)



Input image

$\min_{(u,v)} E(x_0, y_0; u, v)$

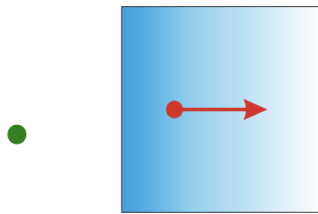
Approximate intensity function in shifted position by the first-order Taylor expansion:

$$I(x + u, y + v) \approx I(x, y) + [I_x(x, y), I_y(x, y)] \begin{bmatrix} u \\ v \end{bmatrix}$$

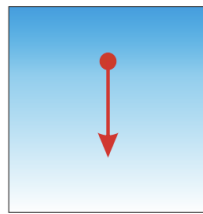
where I_x, I_y are partial derivatives of $I(x, y)$.

$$\begin{aligned} & \sum_{(x,y) \in W(x_0,y_0)} w(x, y) (I(x, y) - I(x + u, y + v))^2 \\ & \approx \sum_{(x,y) \in W(x_0,y_0)} w(x, y) (I(x, y) - (I(x, y) + [I_x(x, y), I_y(x, y)] \begin{bmatrix} u \\ v \end{bmatrix}))^2 \\ & = \sum_{(x,y) \in W(x_0,y_0)} w(x, y) ([I_x(x, y), I_y(x, y)] \begin{bmatrix} u \\ v \end{bmatrix})^2 \end{aligned}$$

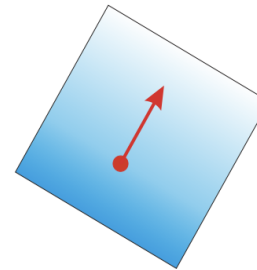
- ◆ For n variables in general, the gradient of a smooth function f is a vector of partial derivatives: $\nabla f(x_1, \dots, x_n) = \left(\frac{\partial f}{\partial x_1}, \dots, \frac{\partial f}{\partial x_n} \right)$
- ◆ For $n = 1$ (1D signal), the gradient simplifies to a (standard) derivative expressed in two notations: f' (Lagrange notation) or $\frac{df}{dx}$ (Leibnitz notation).
- ◆ For $n = 2$ (2D signal), $\nabla f(x, y) = \left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right)$



$$\nabla f = \left(\frac{\partial f}{\partial x}, 0 \right)$$



$$\nabla f = \left(0, \frac{\partial f}{\partial y} \right)$$

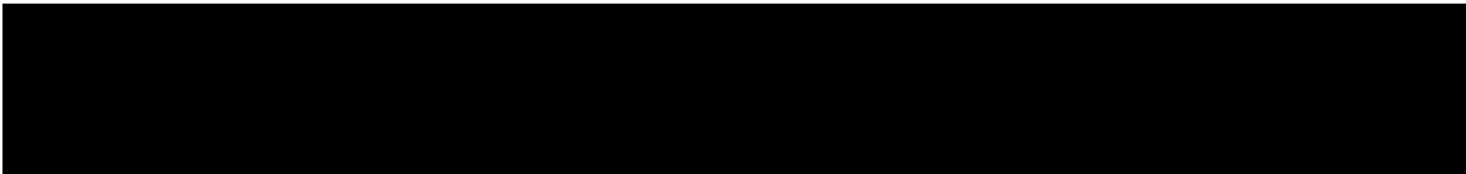


$$\nabla f = \left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right)$$

- $\nabla f(x, y)$ is often expressed in polar coordinates by its magnitude and angular direction ψ

$$\|\nabla f(x, y)\| = \sqrt{\left(\frac{\partial f}{\partial x} \right)^2 + \left(\frac{\partial f}{\partial y} \right)^2}, \quad \psi = \arctan \left(\frac{\partial f}{\partial y} / \frac{\partial f}{\partial x} \right).$$

$$E(x_0, y_0; u, v) \approx \sum_{(x,y) \in W(x_0, y_0)} w(x, y) [I_x(x, y), I_y(x, y)] \begin{bmatrix} u \\ v \end{bmatrix}^2$$

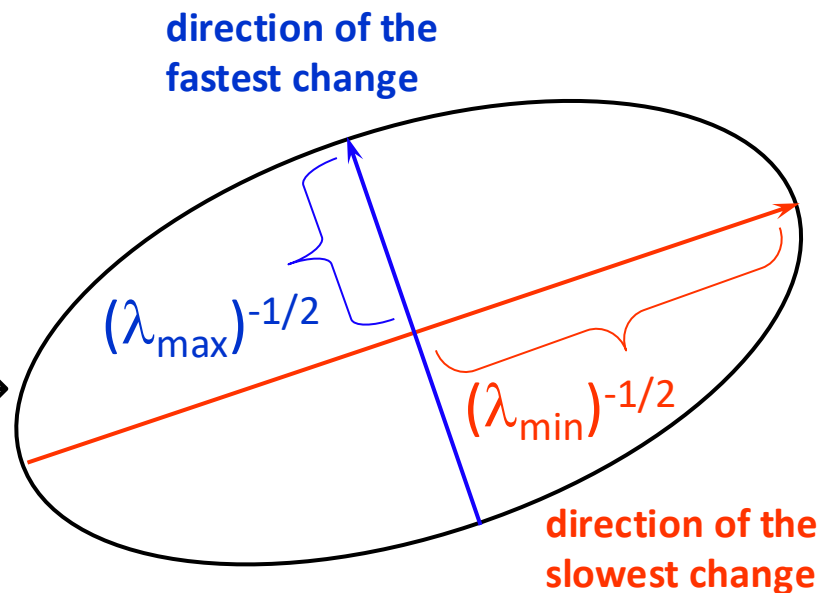


Note that M is symmetric and positive semi-definite,
Remember how E was designed, eq. (1), slide 52.

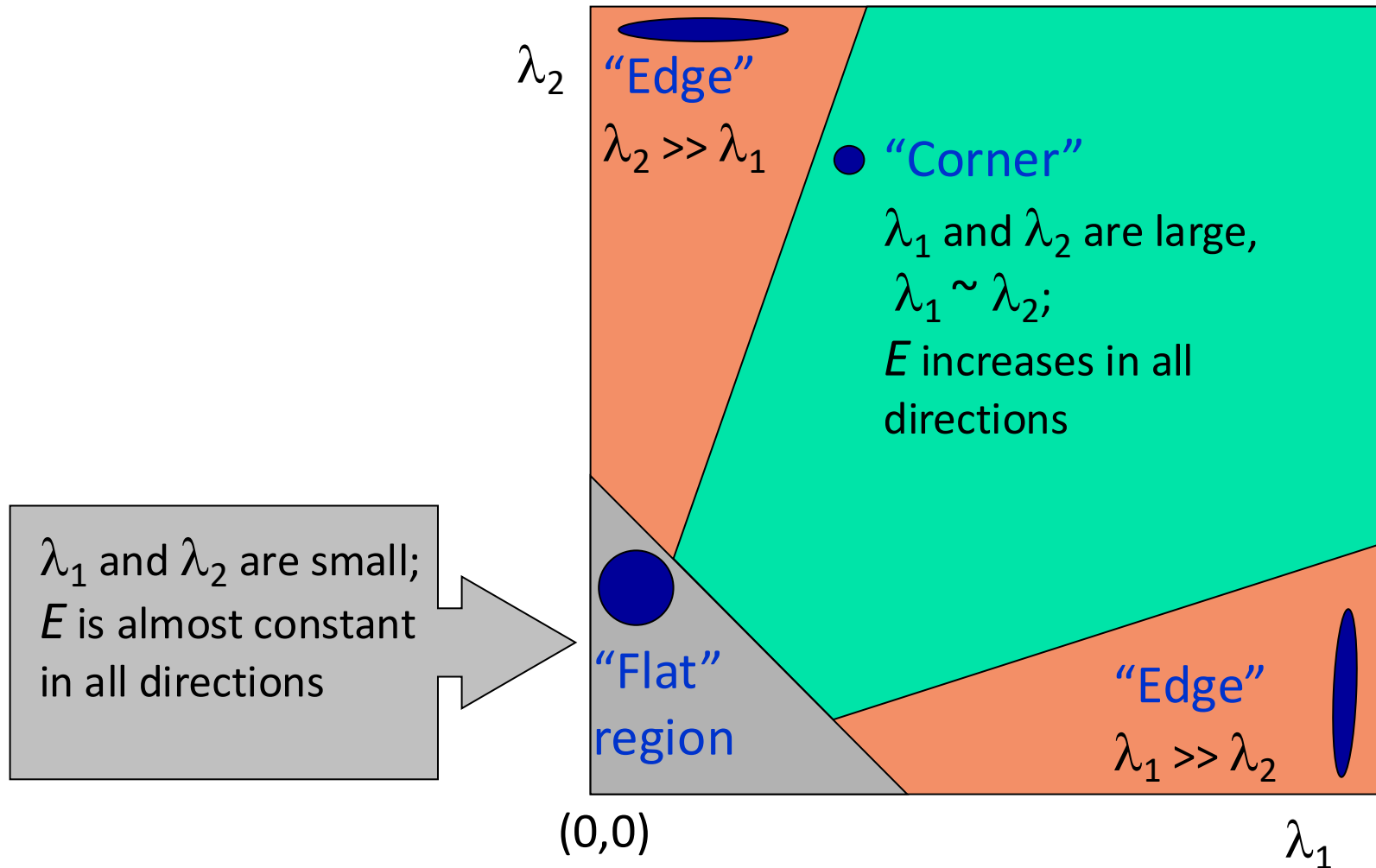
$$E(x_0, y_0; u, v) \approx [u, v]M(x_0, y_0) \begin{bmatrix} u \\ v \end{bmatrix}$$

Iso-contours of E , $E(x_0, y_0; u, v) = \text{const}$, form an ellipse with a center at $(0,0)$ and axes aligned with eigenvector of M

- $\lambda_{\min}, \lambda_{\max}$ – eigenvalues of M ,
 $\lambda_{\min} = \lambda_1, \lambda_{\max} = \lambda_2$
- M symmetric, positive definite $\rightarrow$ orthogonal eigenvectors, positive eigenvalues



Classification of image points using eigenvalues of M :



Measure of “corner” response (“corneriness”):

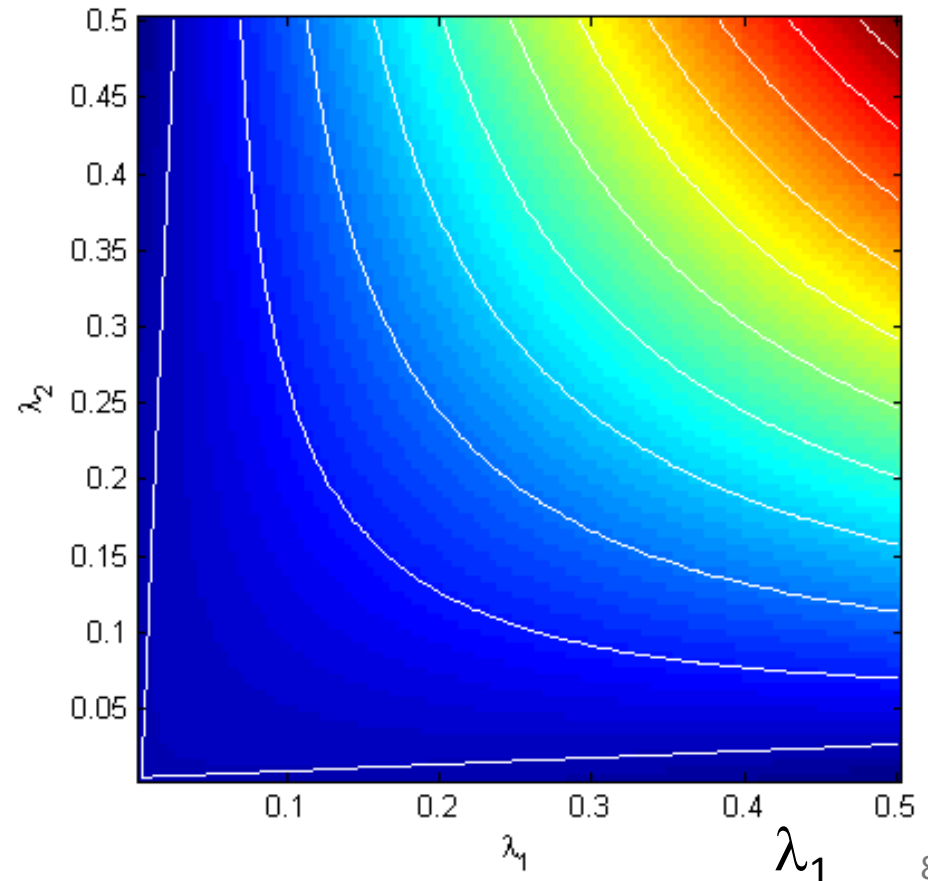
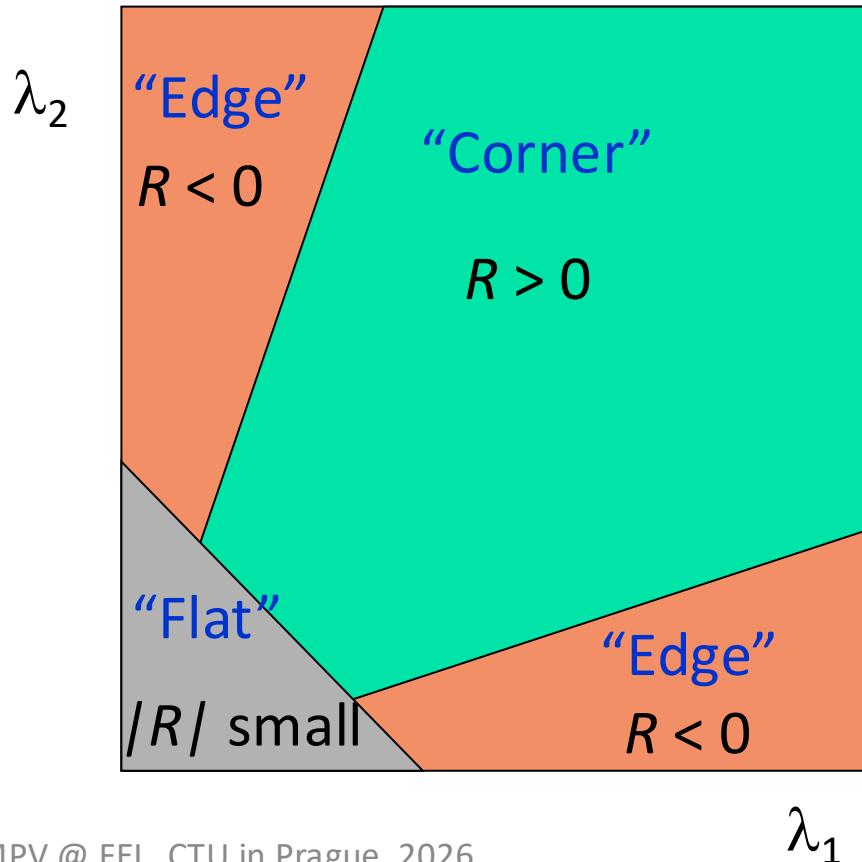
$$R = \det M - k(\text{trace } M)^2$$

- $M = \begin{bmatrix} A & B \\ B & C \end{bmatrix}$
- $\det M = \lambda_1 \lambda_2 = AC - B^2$
- $\text{trace } M = \lambda_1 + \lambda_2 = A + C$
- k ... empirical constant, $k \in (0.04, 0.06)$

Find “corner” points as **local maxima** of corner response R
in their neighborhood (3×3 , or 5×5)

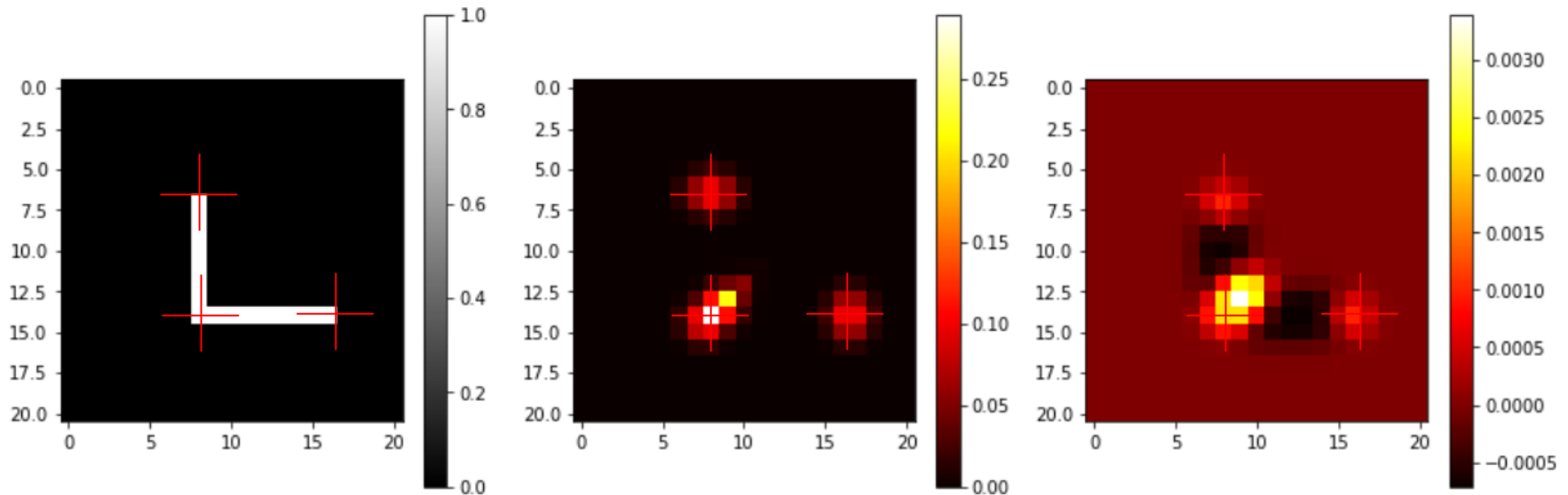
- R depends only on eigenvalues of M
- R is large for a “corner”

- R is negative with large magnitude for an edge
- $|R|$ is small for a flat region



```
[16] artificial_img = torch.zeros(1, 1, 21, 21)
      artificial_img[:, :, 7:14, 8] = 1
      artificial_img[:, :, 14, 8:17] = 1

      in_img = artificial_img
      out_img = ssd_min(in_img, 3, 7)
      out_img_2 = K.feature.harris_response(in_img, 0.05)
      imshow_torch_channels(torch.cat([in_img, out_img, out_img_2], 0), 0, colorbar=True, cmap='hot')
```



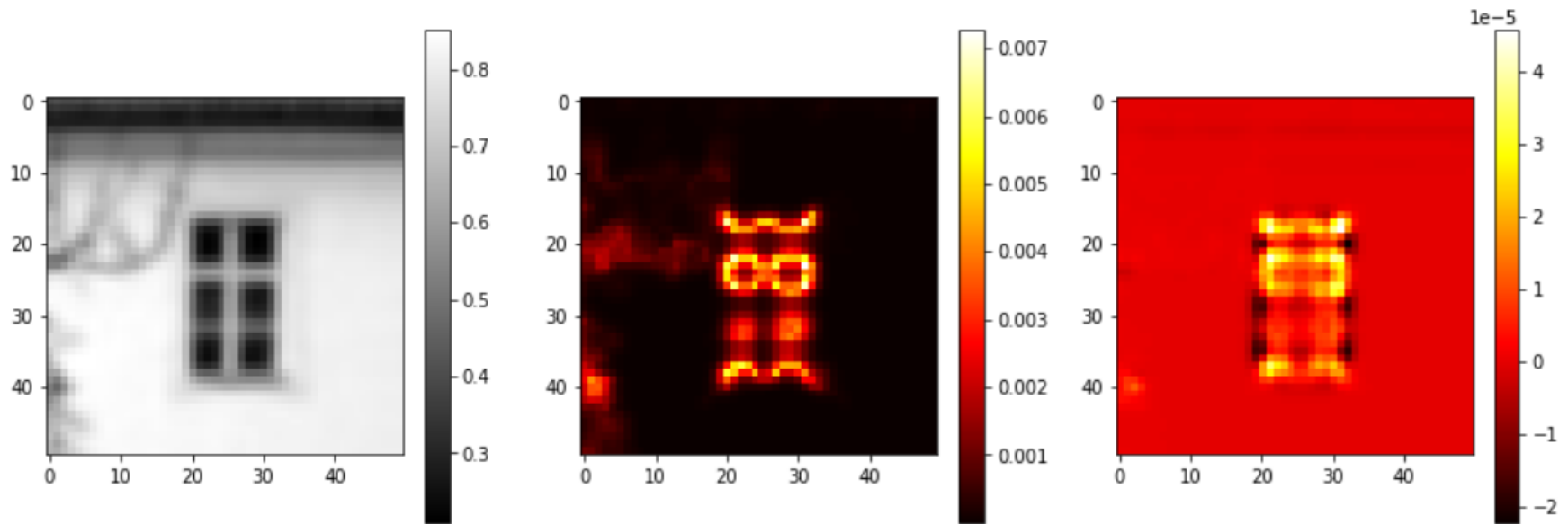
Input image

$\min_{(u,v)} E(x_0, y_0; u, v)$ Harris response

Harris Detector: Mathematics

```
[19] in_img = timimg_blured.float()
      out_img = ssd_min(in_img, 3, 7)
      out_img2 = K.feature.harris_response(in_img, 0.05)

      imshow_torch_channels(torch.cat([in_img, out_img, out_img2], 0)[:,:,: , 80:130,260:310], 0,
                           colorbar=True, cmap='hot')
```



Input image

$\min_{(u,v)} E(x_0, y_0; u, v)$

Harris response

■ The Algorithm:

1. Compute partial derivatives I_x, I_y
2. Compute: $A = \sum_W I_x^2, B = \sum_W I_x I_y, C = \sum_W I_y^2$
3. Compute “corner” response R
4. Find local maxima in R above user defined threshold T

■ Parameters:

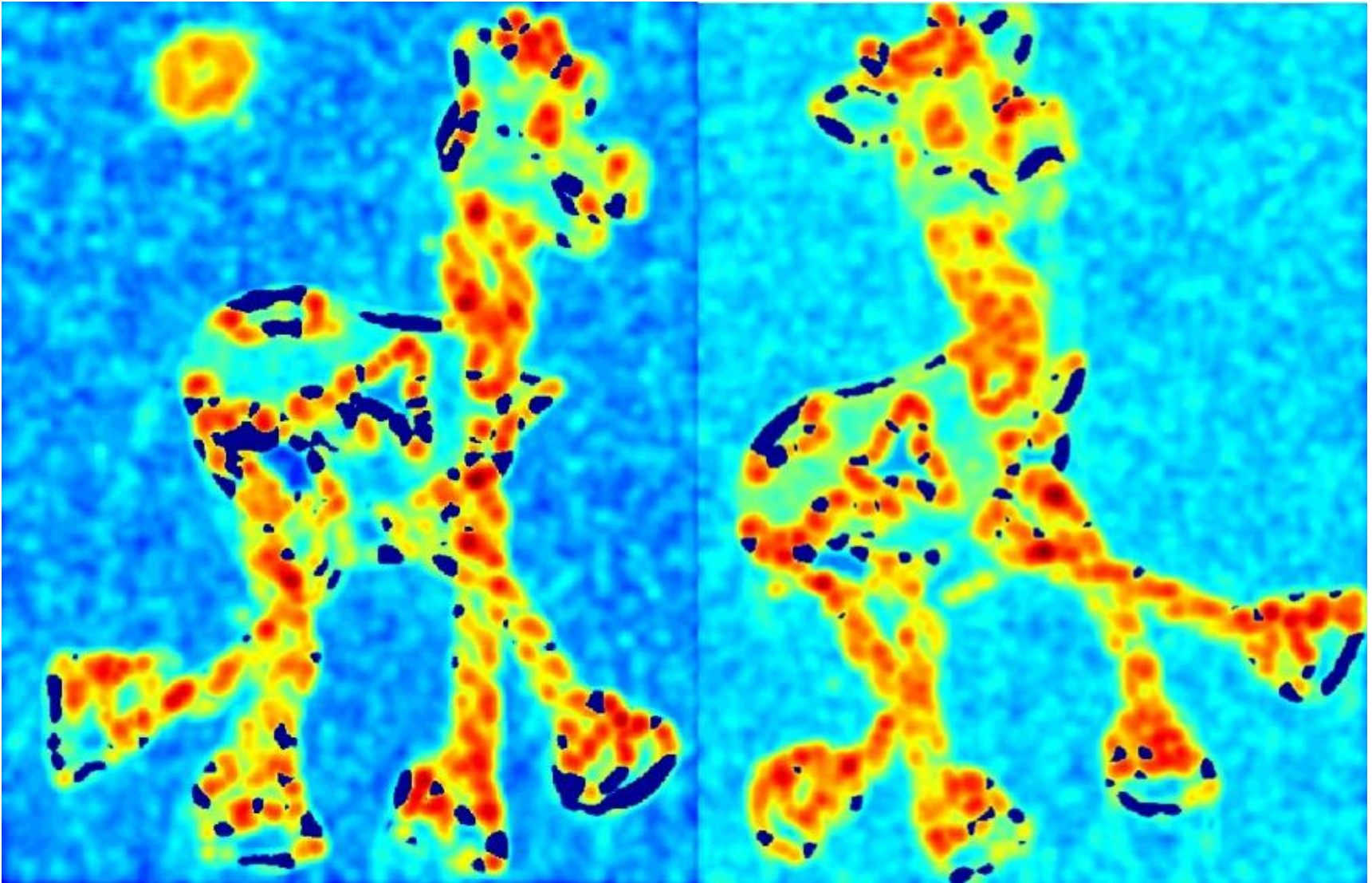
- Threshold T on R
- Scale of the derivative operator (standard setting: very small, just enough to filter anisotropy of the image grid)
- Size of window W (“integration scale”)
- Non-maximum suppression (NMS) algorithm setting, e.g. 8-neighbourhood

Harris Detector: Workflow



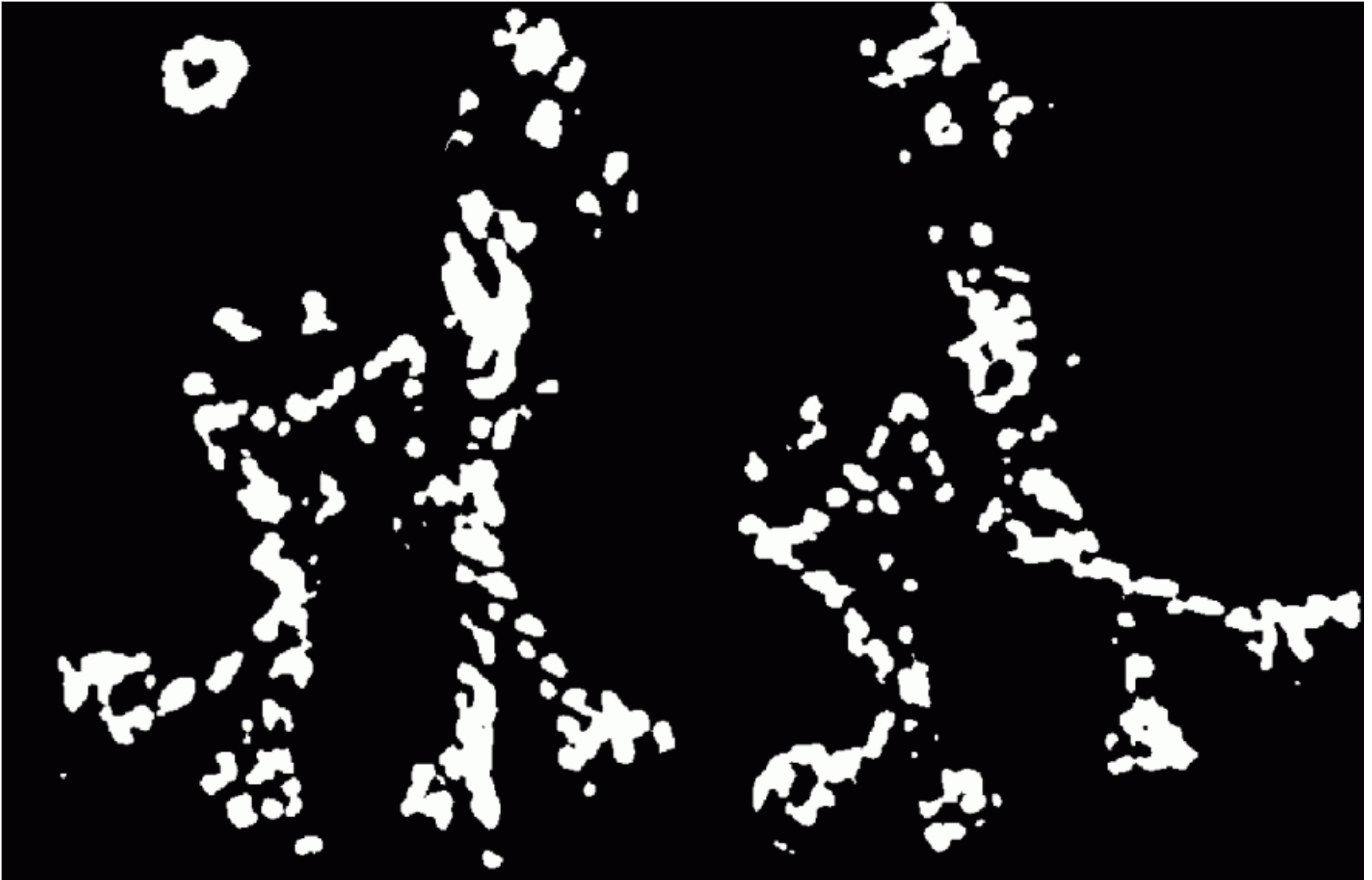
Harris Detector: Workflow

Compute corner response R



Harris Detector: Workflow

Find points with large corner response: $R > \text{threshold}$



Harris Detector: Workflow

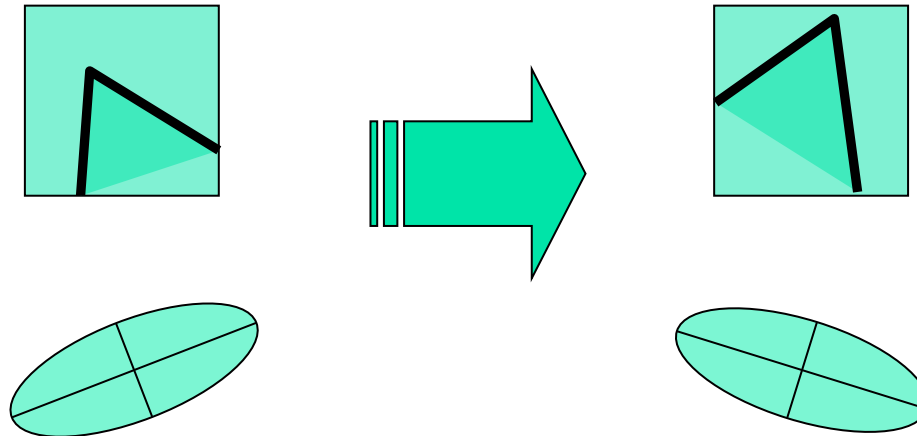
Take only the points of local maxima of R



Harris Detector: Workflow



■ Rotation invariance



Ellipse rotates but its shape (i.e. eigenvalues)
remains the same

Corner response R is invariant to image rotation

■ The Algorithm:

1. Compute partial derivatives I_x, I_y
2. Compute: $A = \sum_W I_x^2, B = \sum_W I_x I_y, C = \sum_W I_y^2$
3. Compute “corner” response R
4. Find local maxima in R above user defined threshold T

■ Parameters:

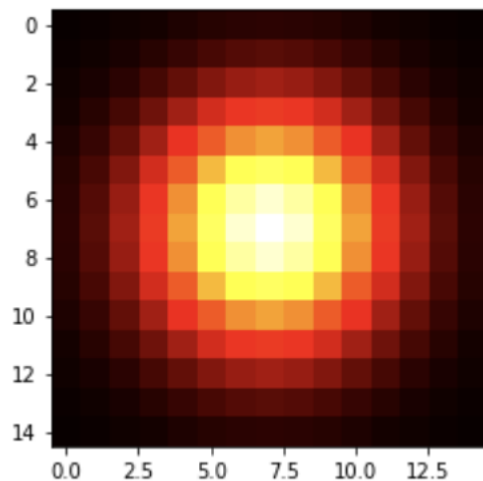
- Threshold T on R
- Scale of the derivative operator (standard setting: very small, just enough to filter anisotropy of the image grid)
- Size of window W (“integration scale”)
- Non-maximum suppression (NMS) algorithm setting, e.g. 8-neighbourhood

2D Gaussian filtering (blur)

```
def gaussian(window_size: int, sigma: float) -> torch.Tensor:
    x = torch.arange(window_size, device=device, dtype=dtype) - window_size // 2
    if window_size % 2 == 0:
        x = x + 0.5
    gauss = torch.exp(-x.pow(2.0) / (2 * sigma ** 2))
    return gauss / gauss.sum()
```

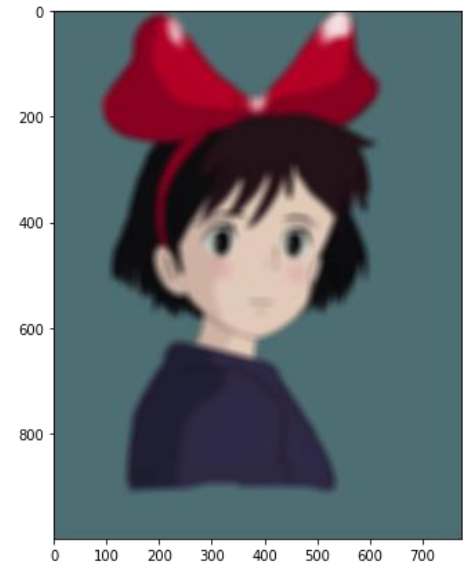
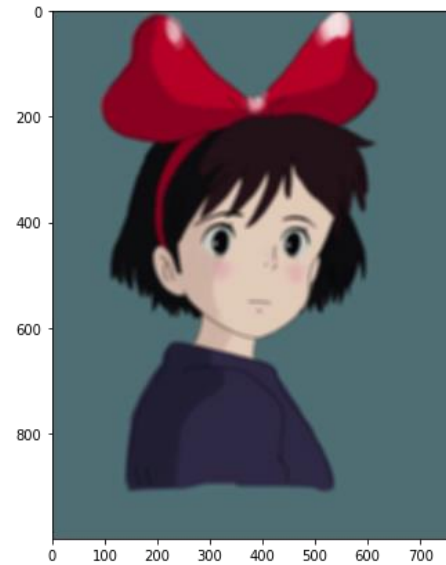
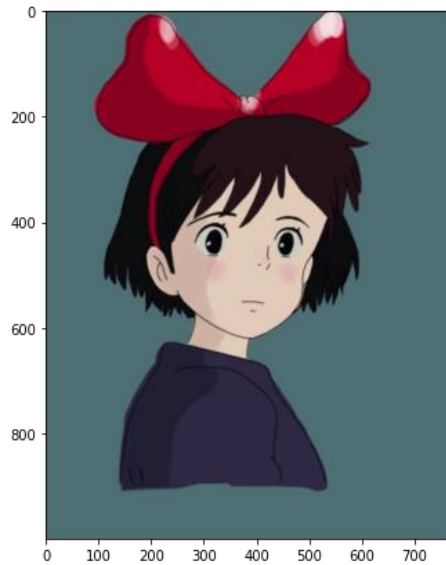
```
▶ gauss_kernel = K.get_gaussian_kernel2d((15,15), (3.0, 3.0))
plt.imshow(K.tensor_to_image(gauss_kernel), cmap='hot')
```

☞ <matplotlib.image.AxesImage at 0x7f384716f1d0>

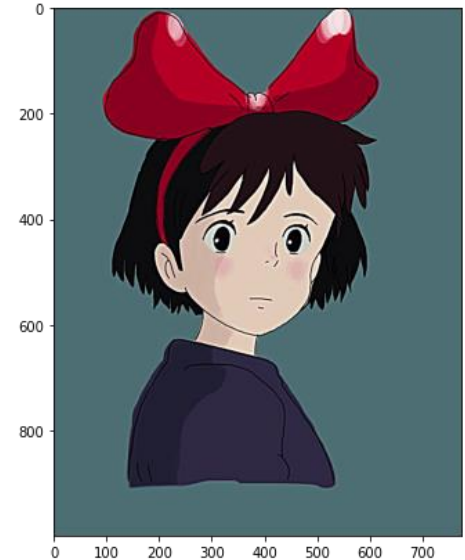
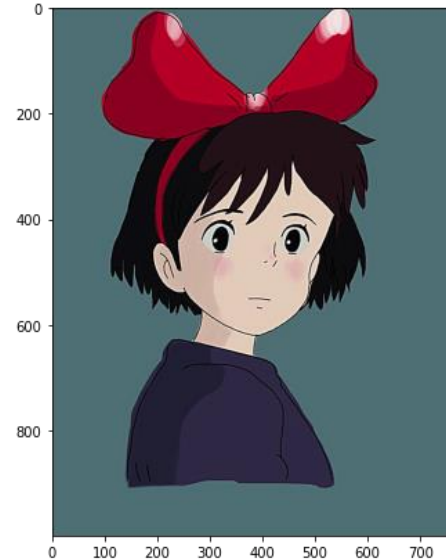
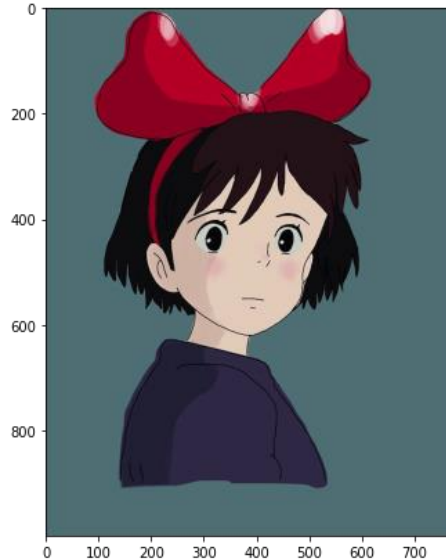


2D image filtering

smoothing



sharpening



2D Gaussian filtering (blur)

```
▶ img_blurred = K.filter2d(timg, gauss_kernel)  
plt.figure(figsize=(16,12))  
plt.imshow(K.tensor_to_image(img_blurred))
```



Image Gradient Calculation Revisited

- Can be done by either of these two ways (with similar results).
 - Reconstruct the continuous function from the discrete one and compute its derivative.
 - Approximate the derivative by finite differences.

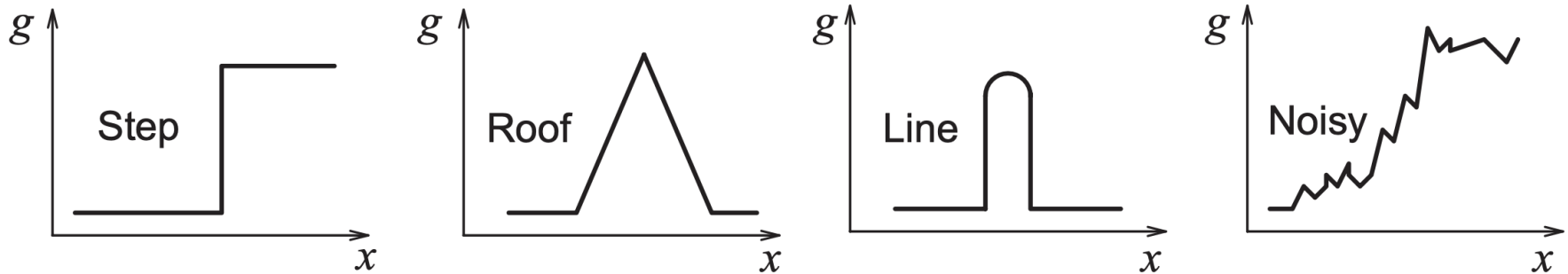
Finite differences:

- non-symmetric: $f'(i) \approx f(i) - f(i - 1)$.
(left difference, does not use pixel value $f(i + 1)$)
- symmetric: $f'(i) \approx \frac{f(i + 1) - f(i - 1)}{2}$

This can be done by convolution:

$$f' \approx [-1, +1] * f, f \approx [-0.5, 0, +0.5] * f.$$

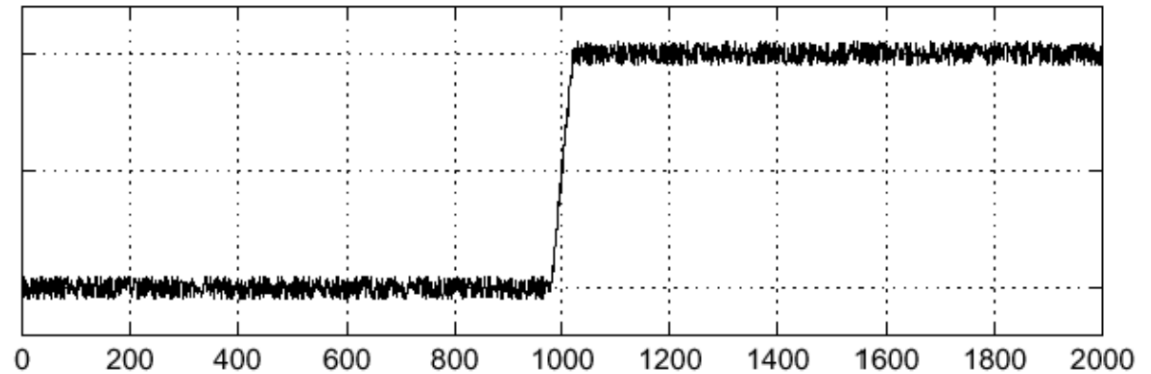
Intensity profiles in the pixel neighborhood



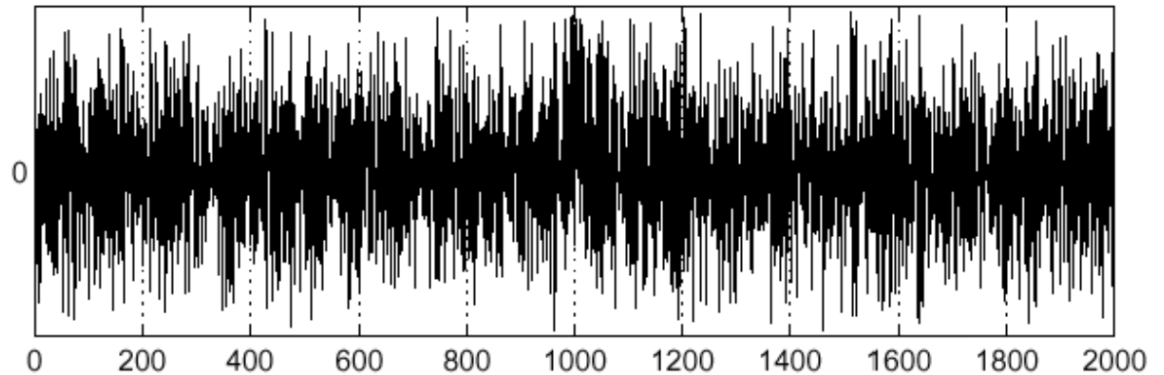
- The first profiles from the left are idealized, i.e. a step edge, a roof edge and a thin line.
- The profile on the right side corresponds to a noise edge. Such edges appear in real images.

Derivatives are susceptible to noise

image profile with noise

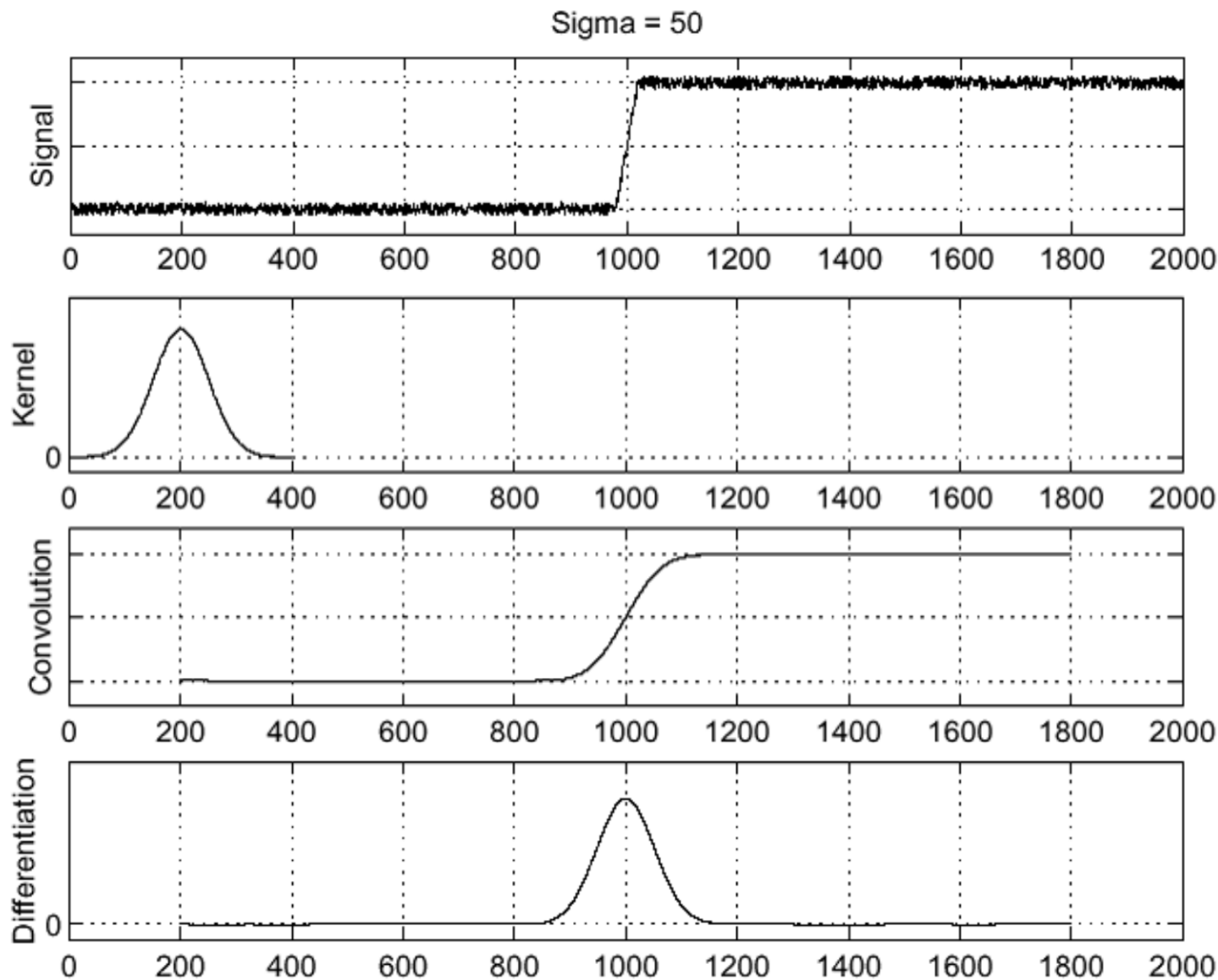


its derivative

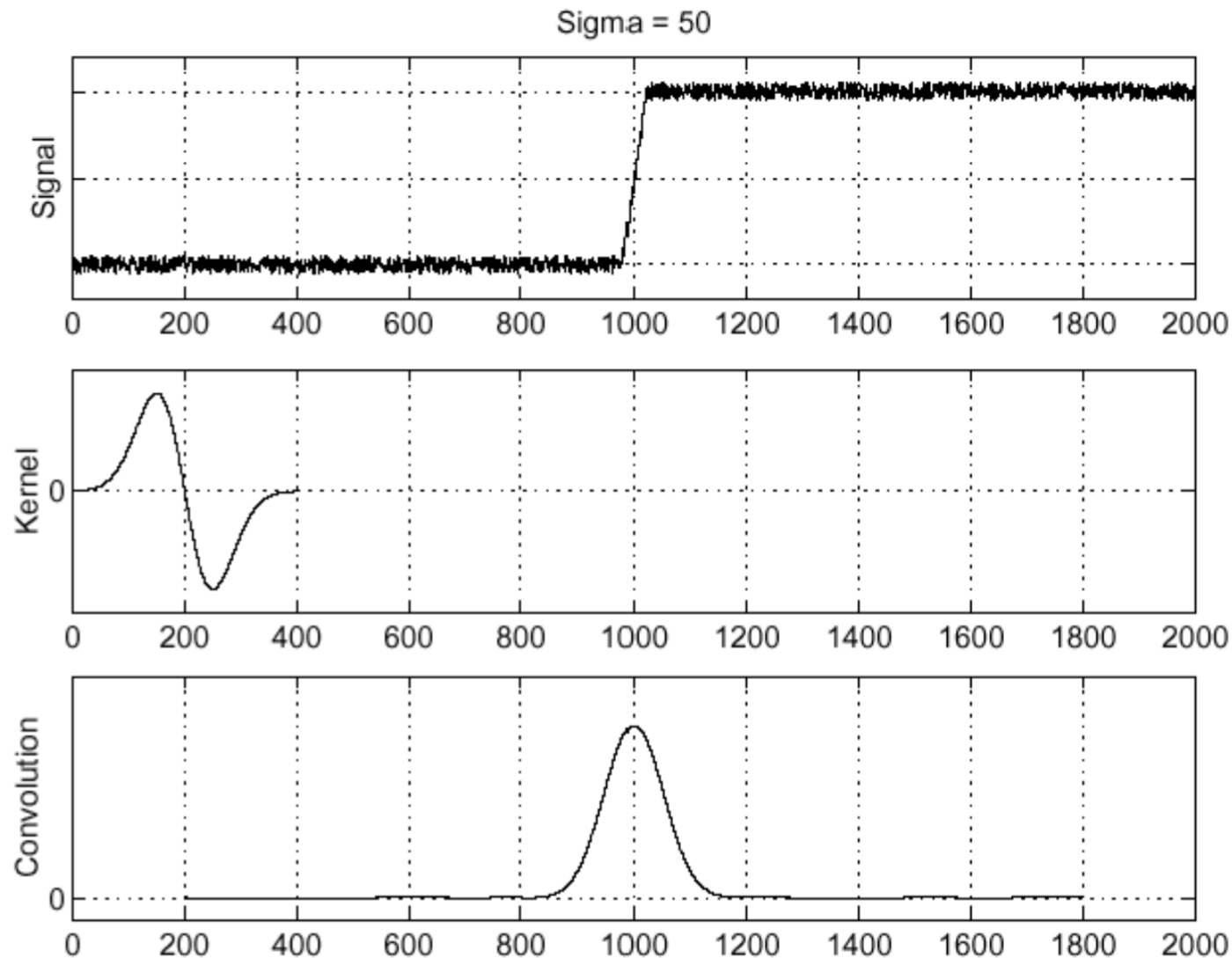


Where is the edge located in the noisy image?

Image should be smoothed before grad



Derivation and smoothing combined



Sobel operator in 3x3 neighborhood

$$h_1 = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}, \quad h_2 = \begin{bmatrix} 0 & 1 & 2 \\ -1 & 0 & 1 \\ -2 & -1 & 0 \end{bmatrix}$$

$$h_3 = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \dots$$

Sobel example

```
kiki_grad = K.spatial_gradient(timg, order=1, mode='sobel')  
print (kiki_grad.shape)
```

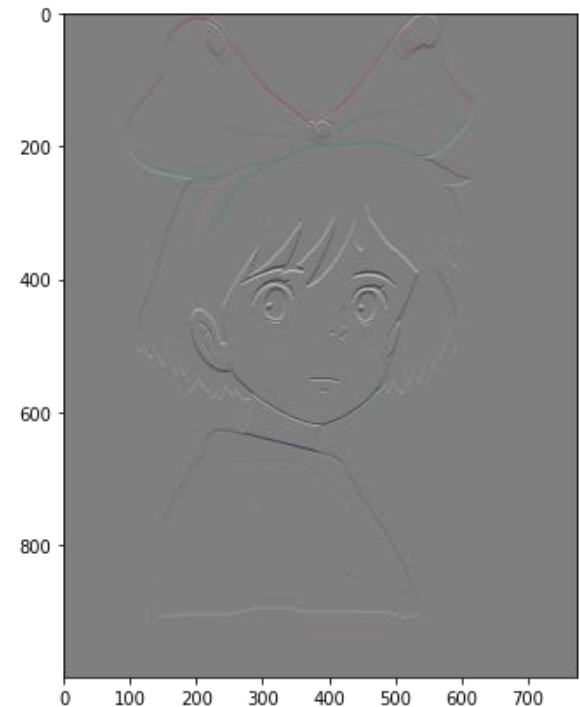
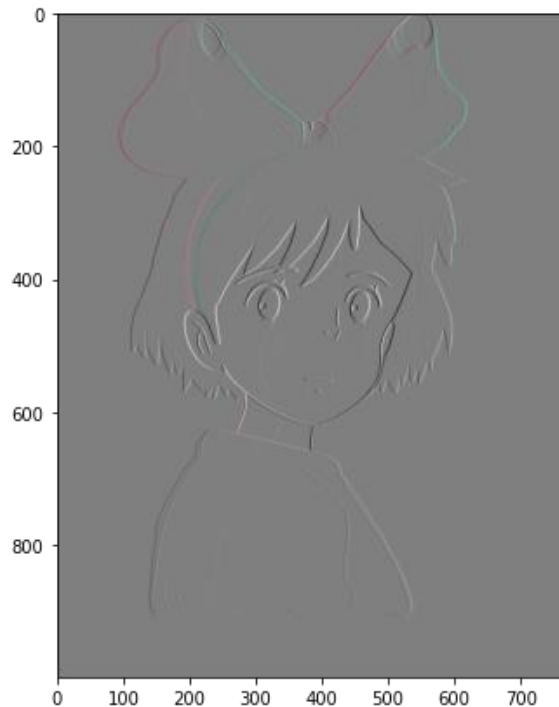
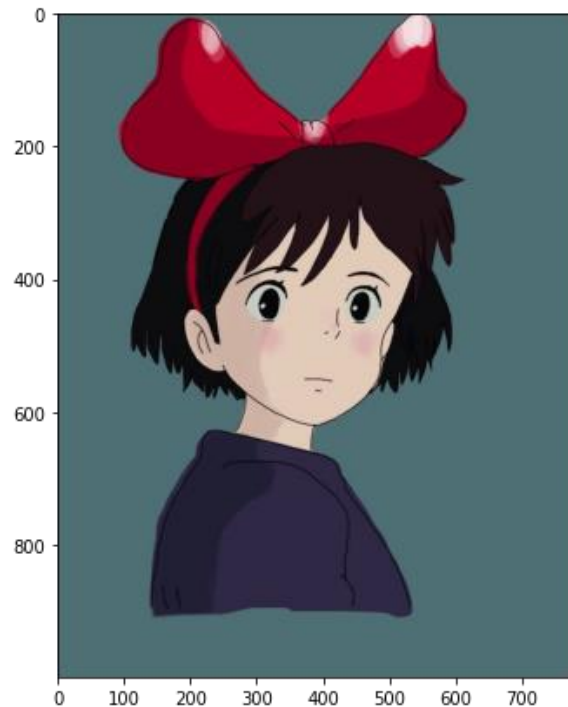
```
torch.Size([1, 3, 2, 1000, 776])
```

```
fig, axes = plt.subplots(1, 3, figsize=(18,10))  
axes[0].imshow(K.tensor_to_image(timg))  
axes[1].imshow(K.tensor_to_image(kiki_grad[:, :, 0])+0.5)  
axes[2].imshow(K.tensor_to_image(kiki_grad[:, :, 1])+0.5)
```

Original

gx

gy

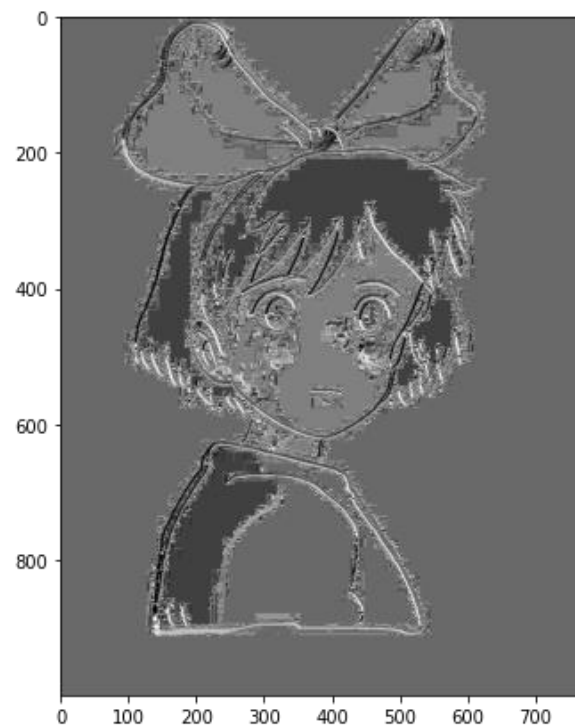
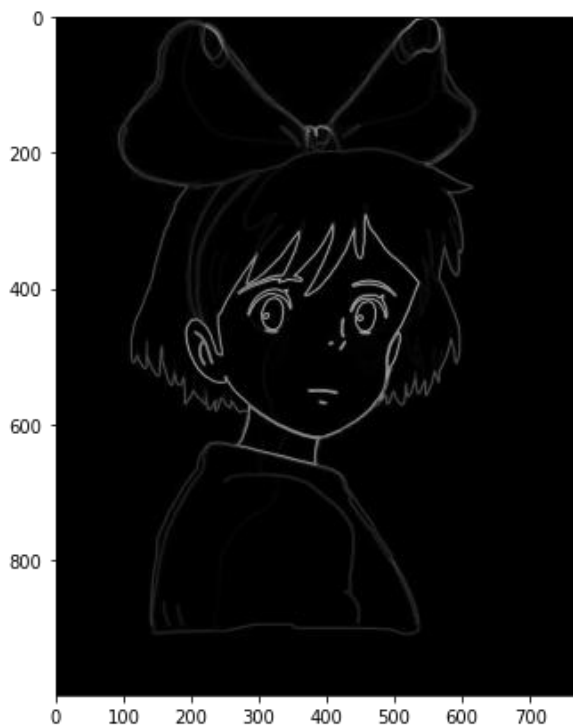
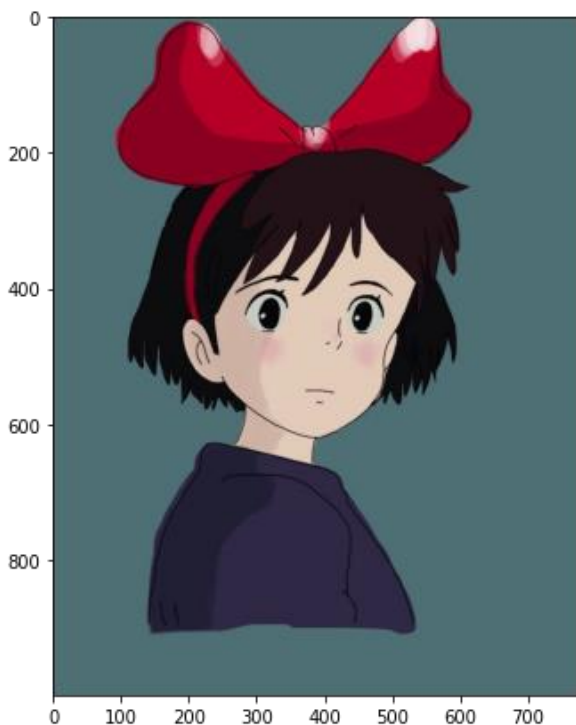


Gradient in polar representation

```
▶ mag, ori = K.cart2pol(kiki_grad[:, :, 0], kiki_grad[:, :, 1])  
fig, axes = plt.subplots(1, 3, figsize=(18, 10))  
axes[0].imshow(K.tensor_to_image(timg))  
axes[1].imshow(K.tensor_to_image(mag.mean(dim=1, keepdim=True)), cmap='gray')  
axes[2].imshow(K.tensor_to_image(ori.mean(dim=1, keepdim=True)), cmap='gray')
```

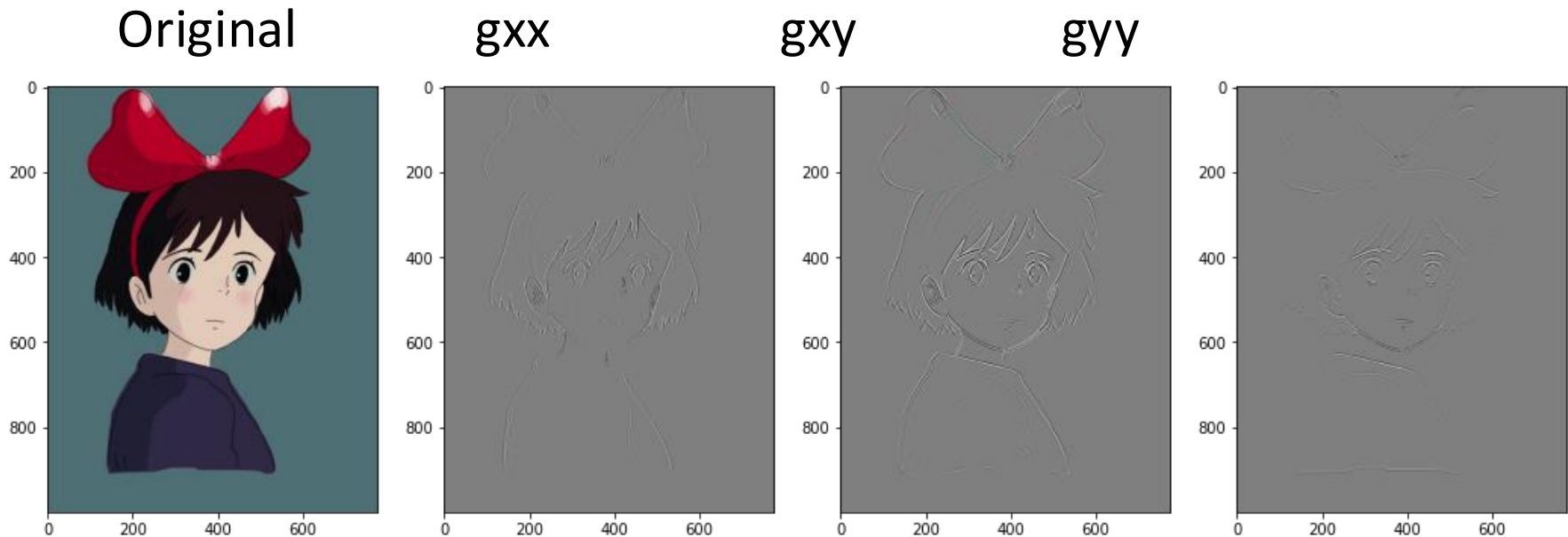
magnitude

orientation



2nd order gradient example

```
▶ kiki_grad = K.spatial_gradient(timg, order=2, mode='diff')  
print (kiki_grad.shape)  
fig, axes = plt.subplots(1, 4, figsize=(18,10))  
axes[0].imshow(K.tensor_to_image(timg))  
axes[1].imshow(K.tensor_to_image(10*kiki_grad[:, :, 0])+0.5)  
axes[2].imshow(K.tensor_to_image(10*kiki_grad[:, :, 1])+0.5)  
axes[3].imshow(K.tensor_to_image(10*kiki_grad[:, :, 2])+0.5)
```



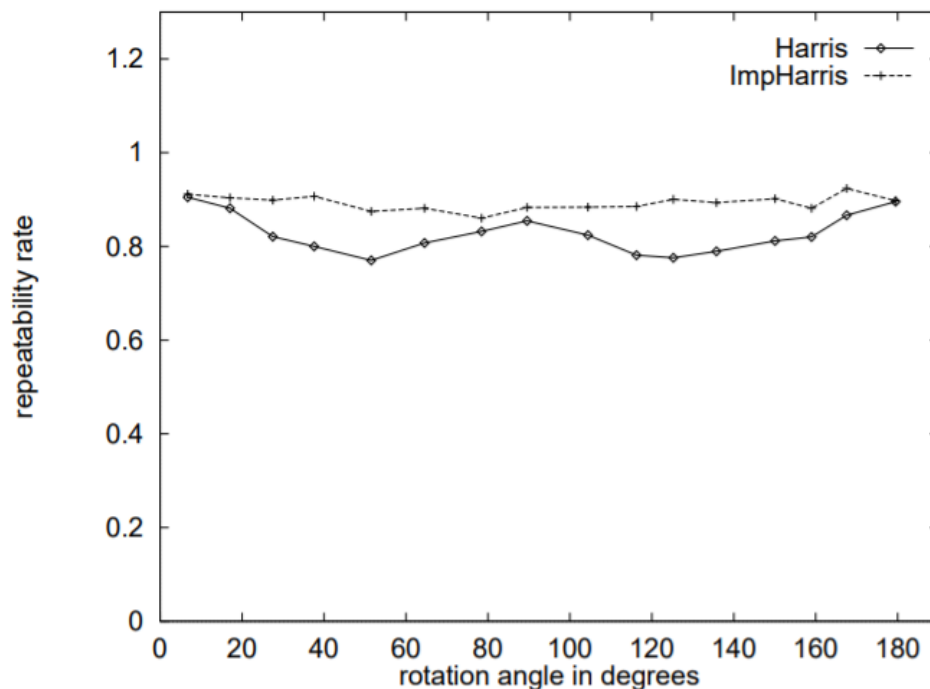
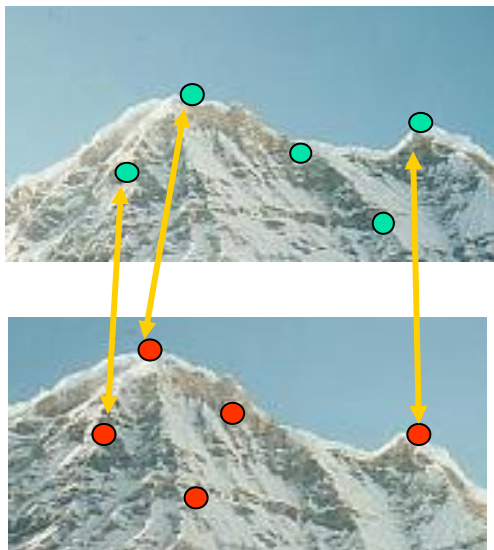
Rotation Invariance of Harris Detector

ImpHarris: using Gaussian deriv (as we do)

Harris: finite differences without Gauss filtering

Repeatability rate:

$\frac{\# \text{ correspondences}}{\# \text{ possible correspondences}}$



C.Schmid et.al. "Evaluation of Interest Point Detectors". IJCV 2000

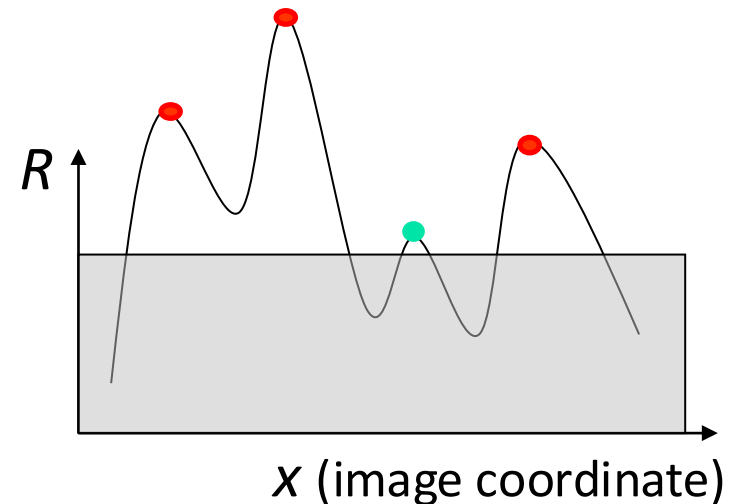
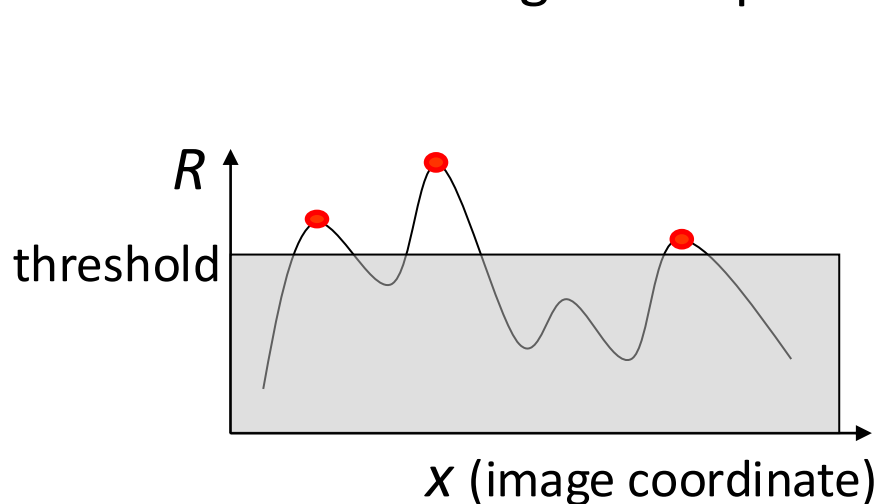
■ Partial invariance to additive and multiplicative intensity changes

✓ Only derivatives are used =>

invariance to intensity shift $I \rightarrow I + b$

? Intensity scale: $I \rightarrow a I$.

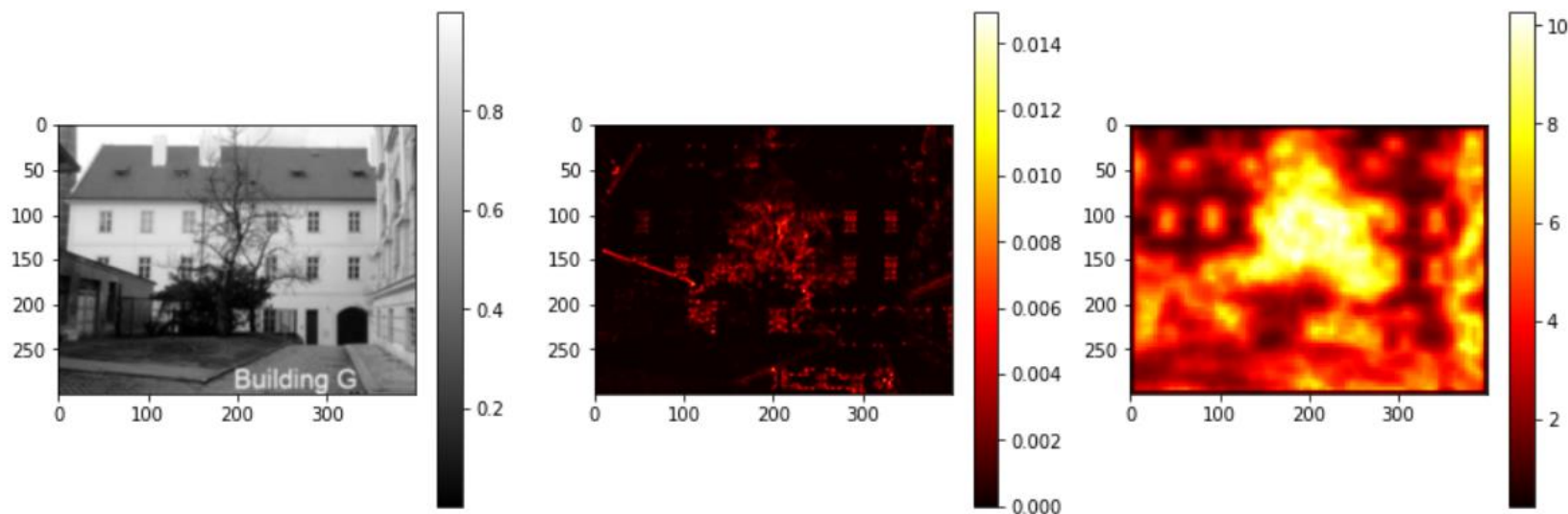
But ordering of R is preserved



$$E(x_0, y_0; u, v) = \sum_{(x,y) \in W(x_0, y_0)} w(x, y) (I(x, y) - I(x + u, y + v))^2$$

Idea. Replace pixel intensity I with descriptor $F \in R^d$

$$RS = \sum_{(x,y) \in W(x_0, y_0)} w(x, y) \|F(x, y) - F(x + u, y + v)\|_2$$



Input image

$\min_{(u,v)} E(x_0, y_0; u, v)$ $\min_{(u,v)} RS(x_0, y_0; u, v)$

Realtime: working at >24 fps

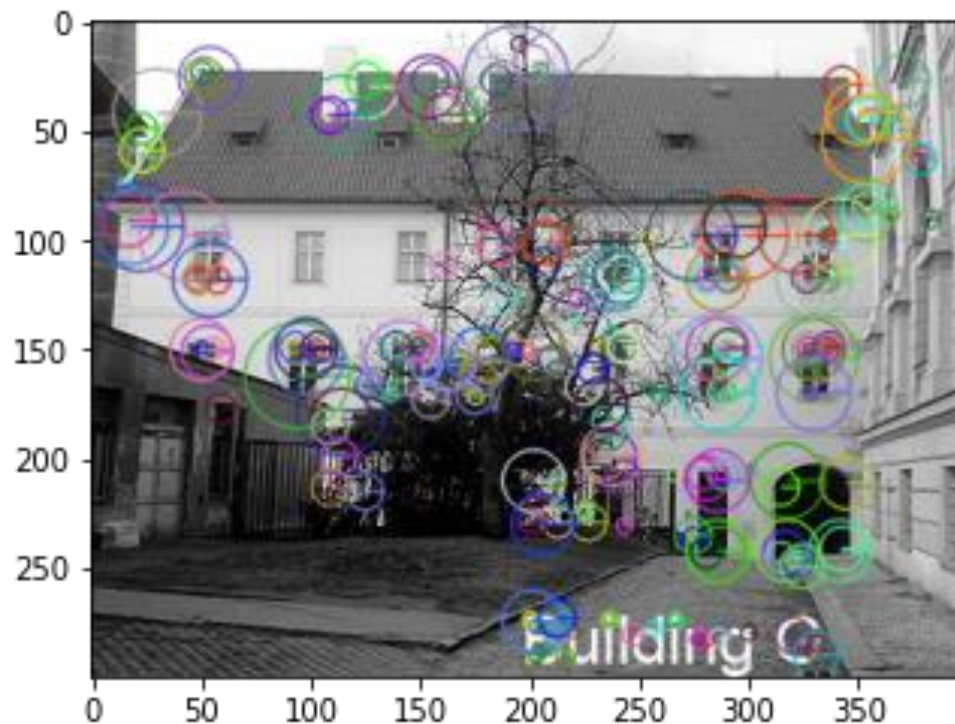
```
import cv2
det = cv2.xfeatures2d.HarrisLaplaceFeatureDetector_create(3, 0.05)
print (img.shape)
%timeit kps = det.detect(img, None)

(300, 400)
10 loops, best of 5: 37.5 ms per loop
```

*Laplace – scale selection method,
see around slide 118

Harris:

- 37.5 ms per 400x300 image
= 26.6 fps
 - without descriptor,
matching, etc.
- Too slow for SLAM



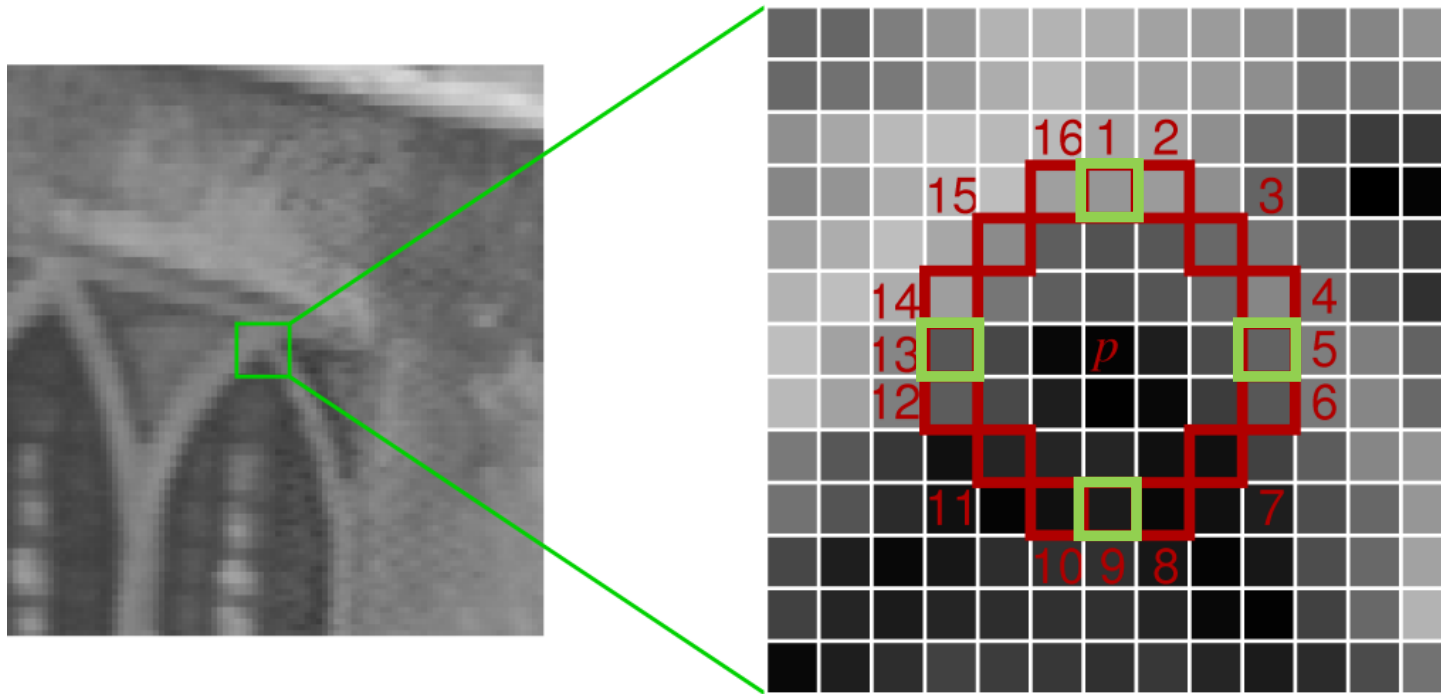
```
[17] import cv2
      det = cv2.ORB_create(1000)
      print (img.shape)
      %timeit kps = det.detect(img, None)
```

→ (300, 400)
100 loops, best of 5: 4.41 ms per loop

FAST (detector part of ORB):

- 4.41 ms per 400x300 image
= 226.7 fps
- Good enough for SLAM





- Considers a circle of 16 pixels around the corner candidate p
- ≥ 12 contiguous pixels brighter/darker than $I_p \pm t, t \dots$ threshold
- Rapid rejection by testing 1, 9, 5 then 13
- Only if at least 3 of those are brighter/darker than $I_p \pm t$ the full segment test is applied

- Corners are clustered:
 - Use non-maximal suppression:

$$V = \max \left(\sum_{q \in S_b} |I_q - I_p| - t, \sum_{q \in S_d} |I_p - I_q| - t \right)$$

where $S_b = \{q | I_q \geq I_p + t\}$, $S_d = \{q | I_q \leq I_p - t\}$

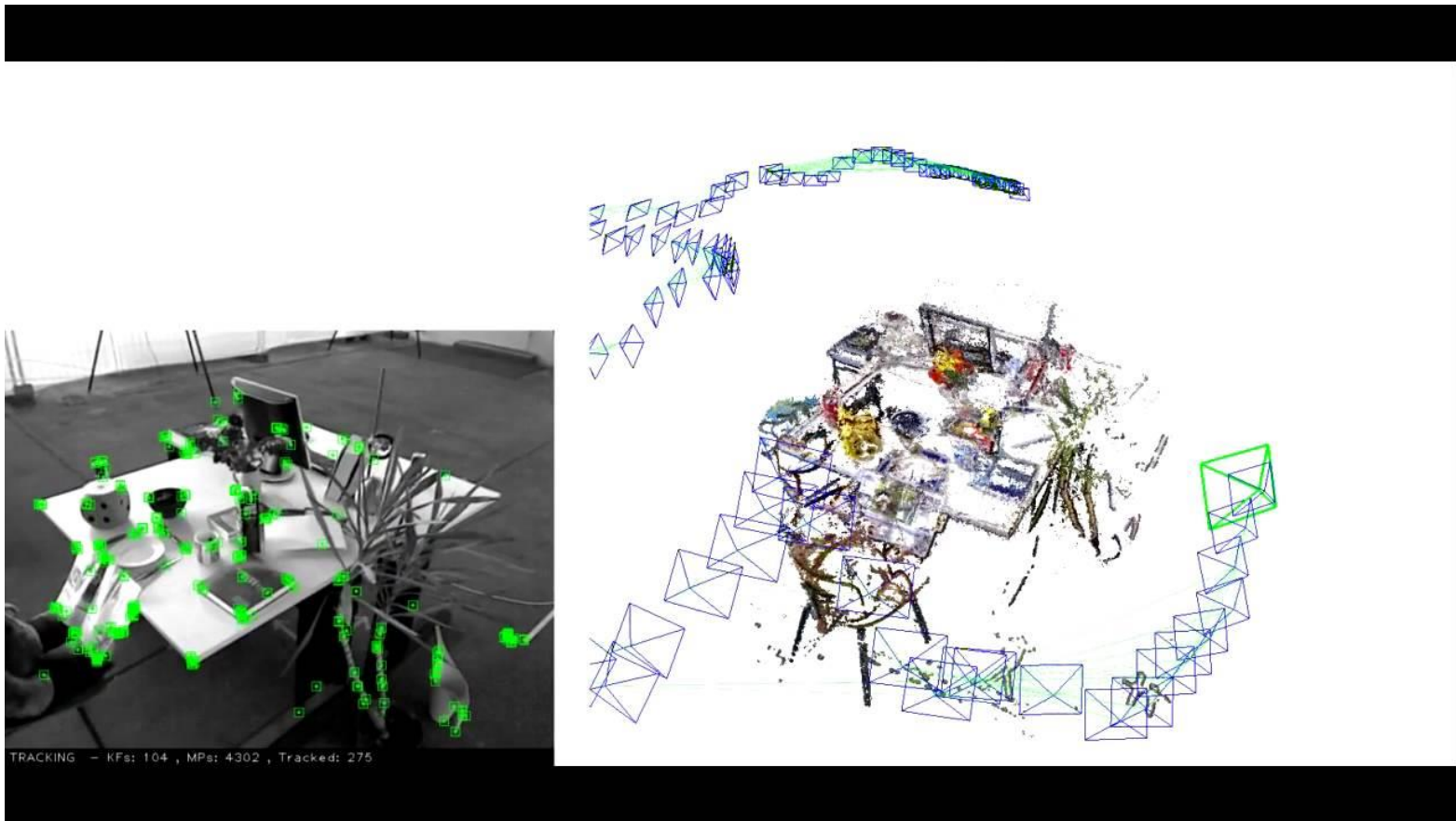
- High speed test does not generalize well for $n < 12$
- Knowledge from the first 4 tests is discarded
- Multiple features are detected adjacent to one another

Detector	Opteron 2.6GHz		Pentium III 850MHz	
	ms	%	ms	%
Fast $n = 9$ (non-max suppression)	1.33	6.65	5.29	26.5
Fast $n = 9$ (raw)	1.08	5.40	4.34	21.7
Fast $n = 12$ (non-max suppression)	1.34	6.70	4.60	23.0
Fast $n = 12$ (raw)	1.17	5.85	4.31	21.5
Original FAST $n = 12$ (non-max suppression)	1.59	7.95	9.60	48.0
Original FAST $n = 12$ (raw)	1.49	7.45	9.25	48.5
Harris	24.0	120	166	830
DoG	60.1	301	345	1280
SUSAN	7.58	37.9	27.5	137.5

Table 1. Timing results for a selection of feature detectors run on fields (768×288) of a PAL video sequence in milliseconds, and as a percentage of the processing budget per frame. Note that since PAL and NTSC, DV and 30Hz VGA (common for web-cams) have approximately the same pixel rate, the percentages are widely applicable. Approximately 500 features per field are detected.

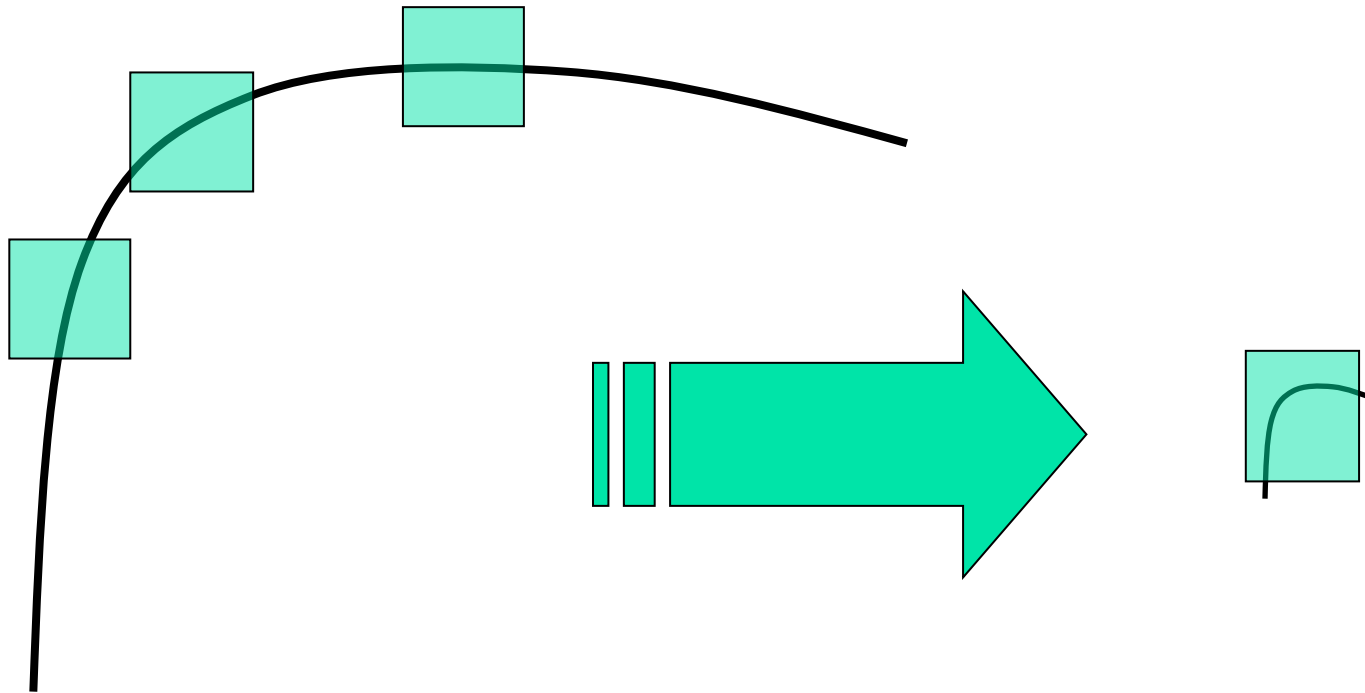
FAST is a part of the ORB feature

- FAST is a backbone of the ORB-SLAM2
 - Uses Harris cornerness function for NMS
- Detects corner orientation (covered later in slides)



Back to Harris corners

- Not invariant to *image scale*!



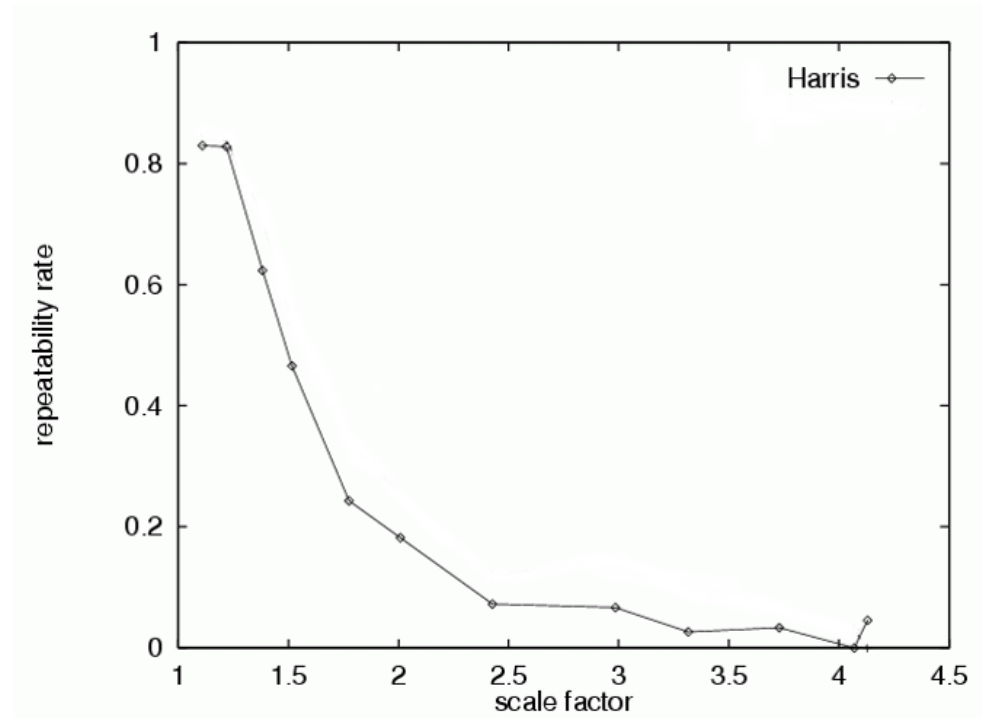
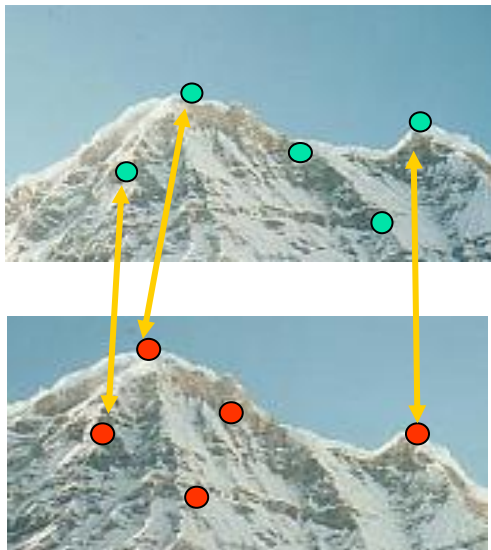
All points will be classified as **edges**

Corner !

■ Quality of Harris detector for different scale changes

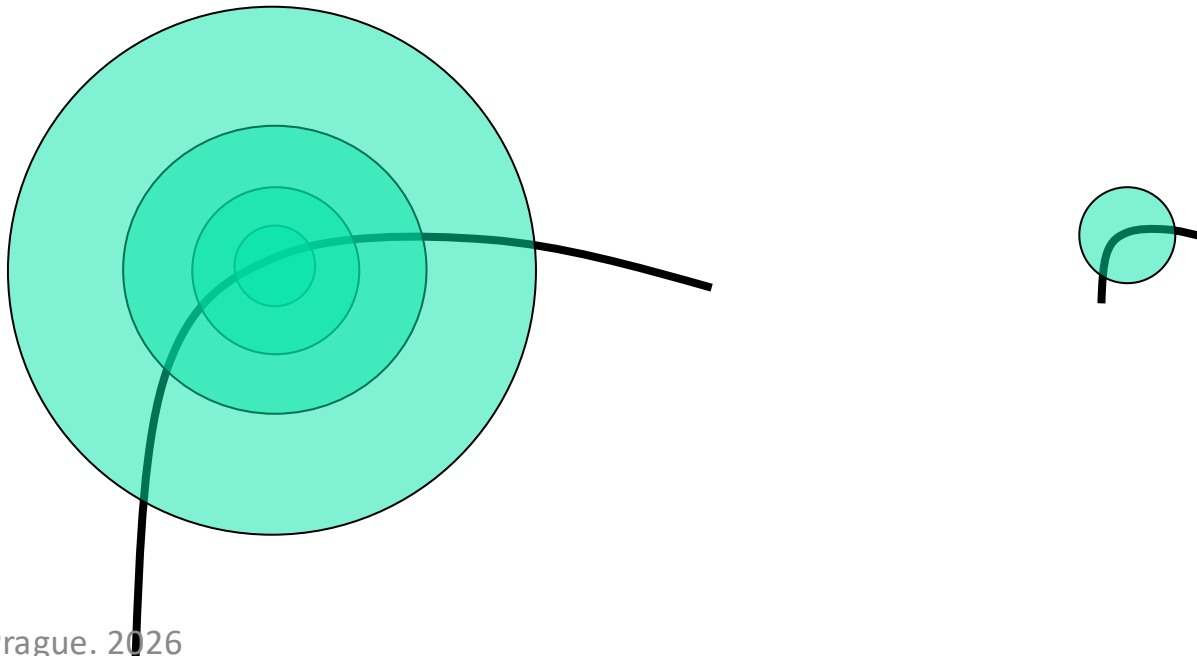
Repeatability rate:

$\frac{\text{\# correspondences}}{\text{\# possible correspondences}}$

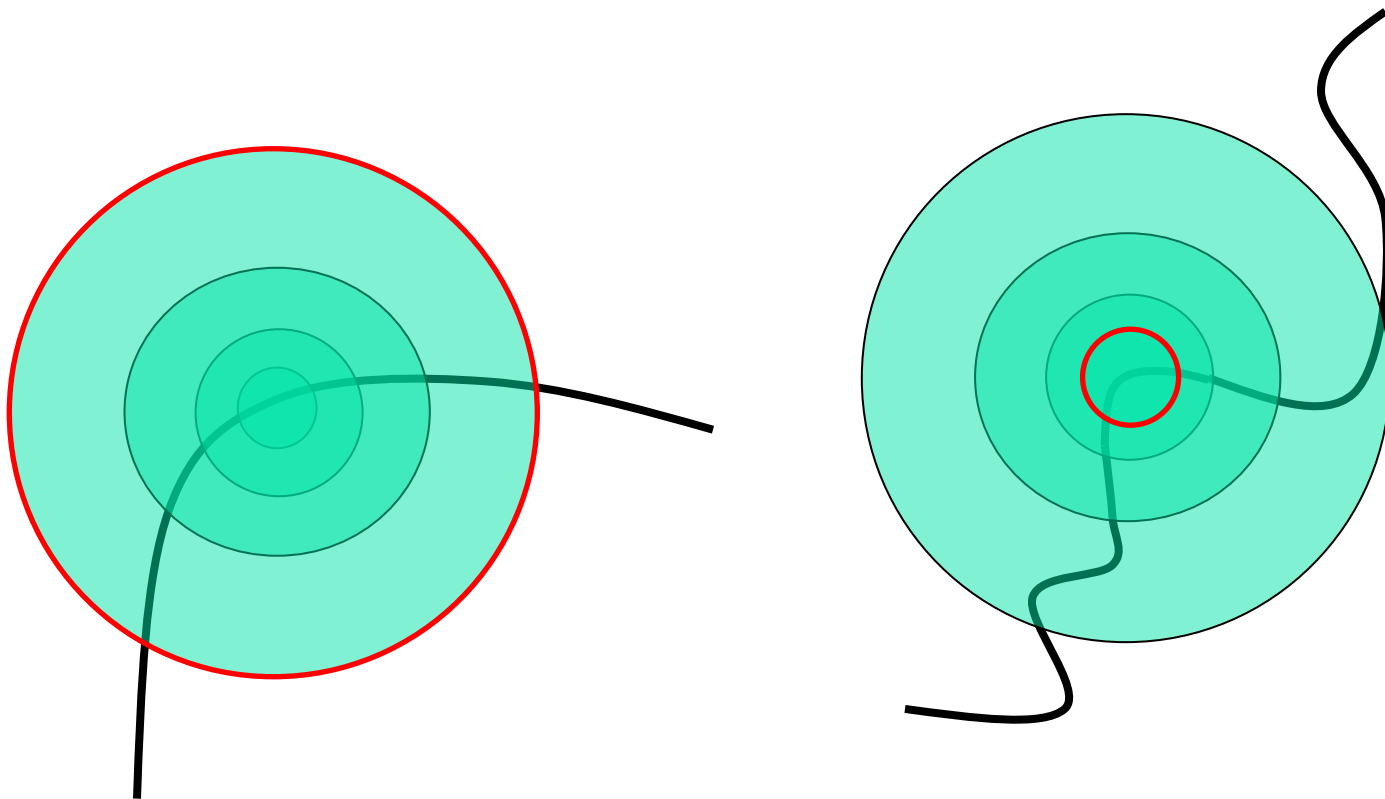


C. Schmid et.al. "Evaluation of Interest Point Detectors". IJCV 2000

- Consider regions (e.g. circles) of different sizes around a point
- Regions of corresponding sizes will look the same in both images

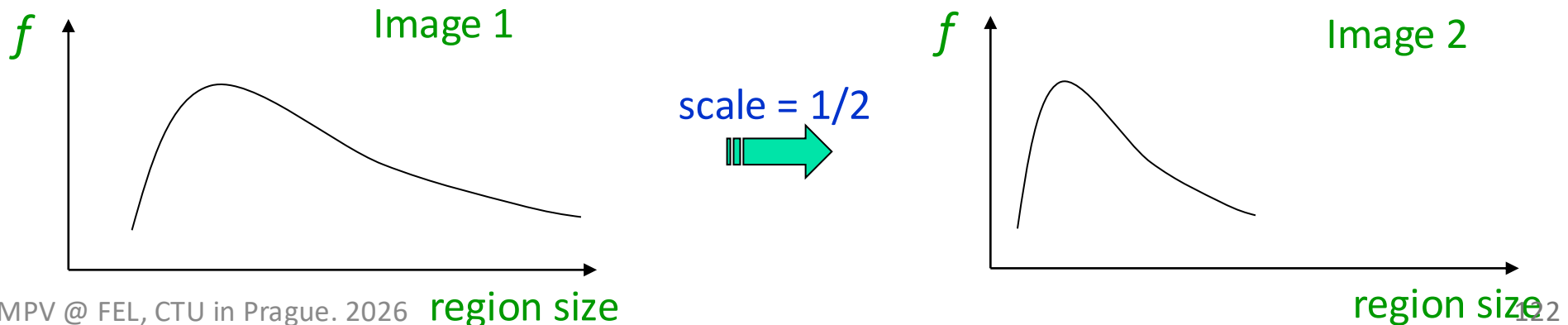


- The problem: how do we choose corresponding circles *independently* in each image?



■ Solution:

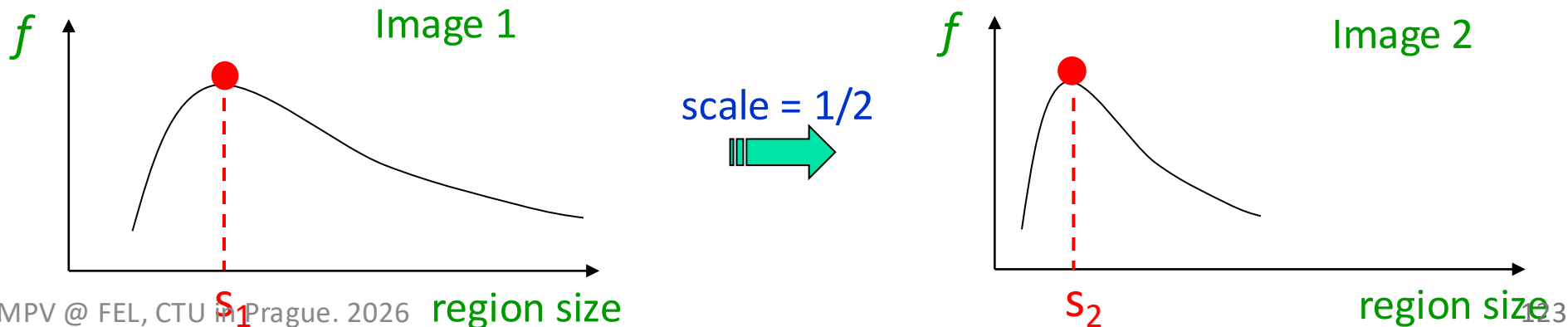
- Design a function on the region (circle), which is “scale covariant” (the same for corresponding regions, even if they are at different scales)
 - For a point in one image, we can consider it as a function of region size (circle radius)



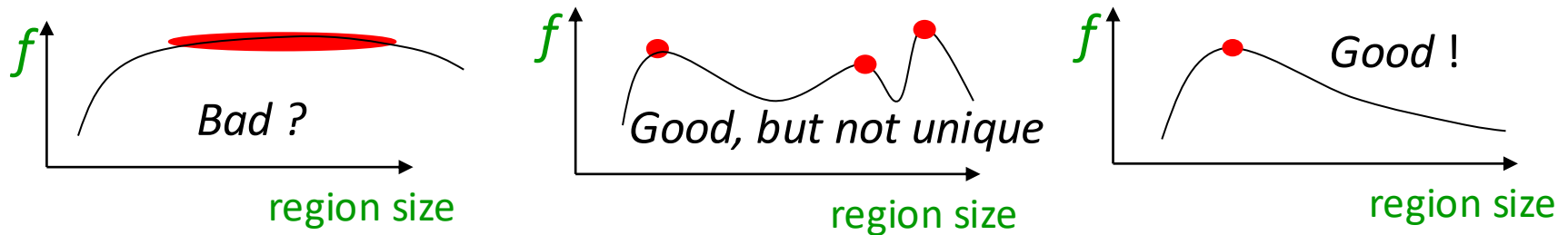
■ Common approach:

- Take a local maximum of some function
- *Observation*: region size, for which the maximum is achieved, should be *invariant* to image scale.

Important: this scale invariant region size is found in each image **independently**!



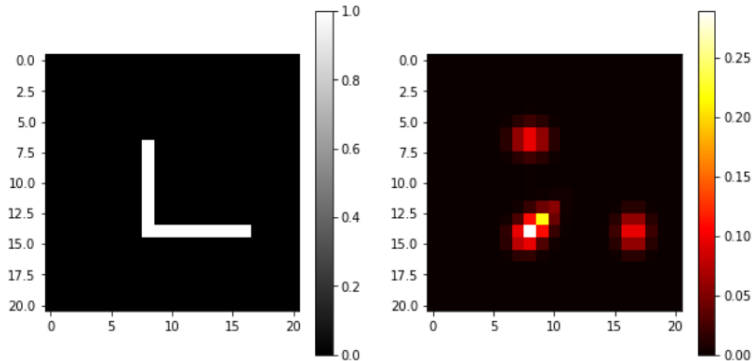
- A “good” function for scale detection:
has one stable sharp peak



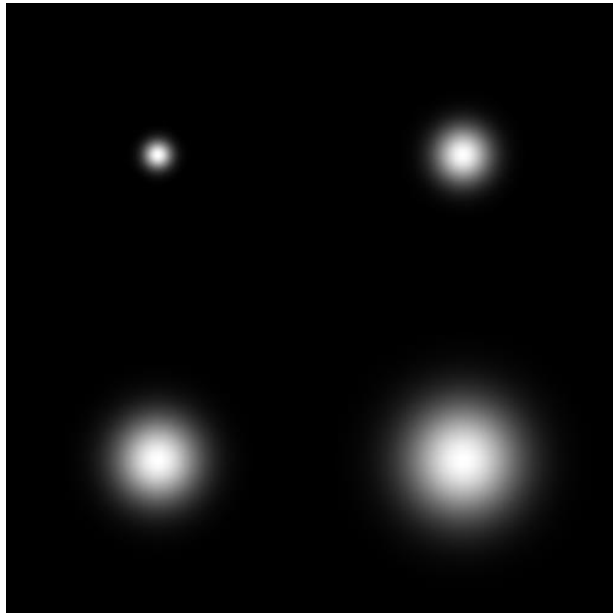
- For usual images: a good function would be a one which responds to contrast (sharp local intensity change)

- Advantage of the “corner” detectors: they have “reference” point, i.e. corner itself

```
artificial_img = torch.zeros(1, 1, 21, 21)
artificial_img[:, :, 7:14, 8] = 1
artificial_img[:, :, 14, 8:17] = 1
|
in_img = artificial_img
out_img = ssd_min(in_img, 3, 7)
imshow_torch_channels(torch.cat([in_img, out_img], 0), 0, colorbar=True, cmap='hot')
```



- Drawback: the ideal corner is literally a point not a region, i.e. has no natural scale. *Note: local maxima of Harris response R are not always “semantically corners”.*
- What simple structure has both a well-defined center and size?
 - A circle!



- The simplest way of detecting a blob is to create a “blob” template, or “kernel”:
 - Laplacian-of-Gaussian – LoG
 - Difference of Gaussians – DoG
- Hessian:
 - Determinant of Hessian – DoH

■ Functions for determining scale

$$f = \text{Kernel} * \text{Image}$$

Kernels:

$$L = \sigma^2 \left(G_{xx}(x, y, \sigma) + G_{yy}(x, y, \sigma) \right)$$

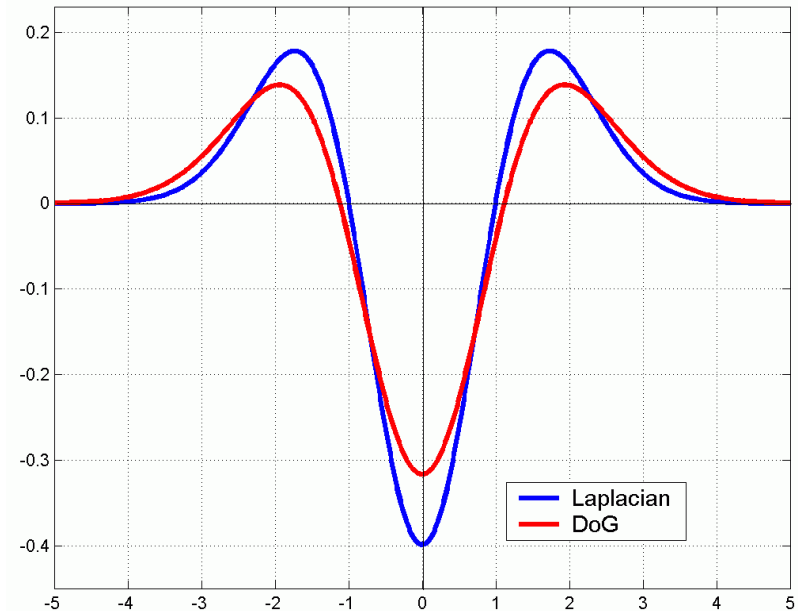
(Laplacian)

$$DoG = G(x, y, k\sigma) - G(x, y, \sigma)$$

(Difference of Gaussians)

where Gaussian

$$G(x, y, \sigma) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

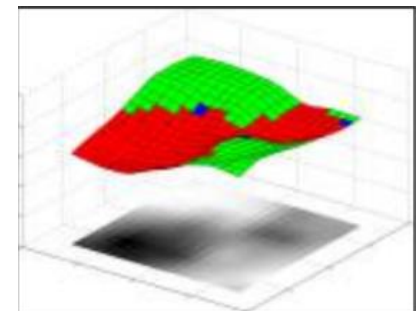


$$\text{Hessian}(x_0, y_0) = H(x_0, y_0) = \begin{bmatrix} I_{xx}(x_0, y_0) & I_{xy}(x_0, y_0) \\ I_{yx}(x_0, y_0) & I_{yy}(x_0, y_0) \end{bmatrix}$$

Note, that $I_{xy}(x_0, y_0) == I_{yx}(x_0, y_0)$

- H is symmetric
- not positive definite matrix, unlike M from Harris
- $\det H > 0$ white blob or dark blob
- $\det H < 0$ saddle point
- $\det H$ is rotation invariant

Saddle point



$$\text{Hessian}(x_0, y_0, \sigma) = H(x_0, y_0, \sigma) = \begin{bmatrix} I_{xx}(x_0, y_0, \sigma) & I_{xy}(x_0, y_0, \sigma) \\ I_{yx}(x_0, y_0, \sigma) & I_{yy}(x_0, y_0, \sigma) \end{bmatrix}$$

We can do

Gaussian blur + finite differences,
or Gaussian derivatives directly

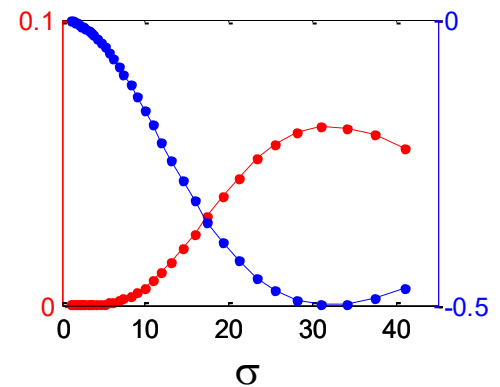
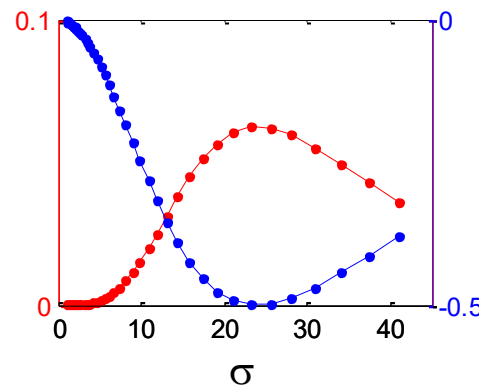
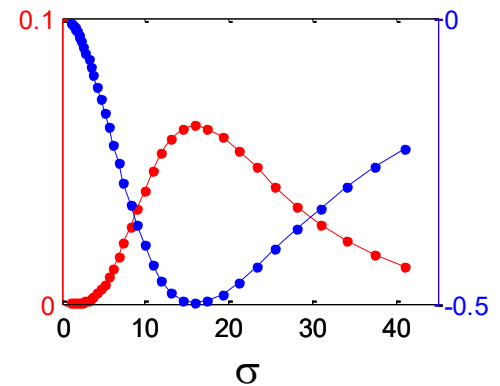
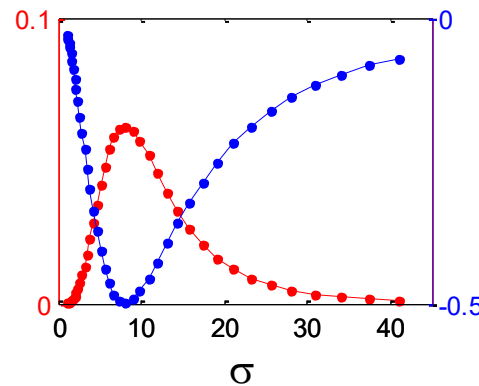
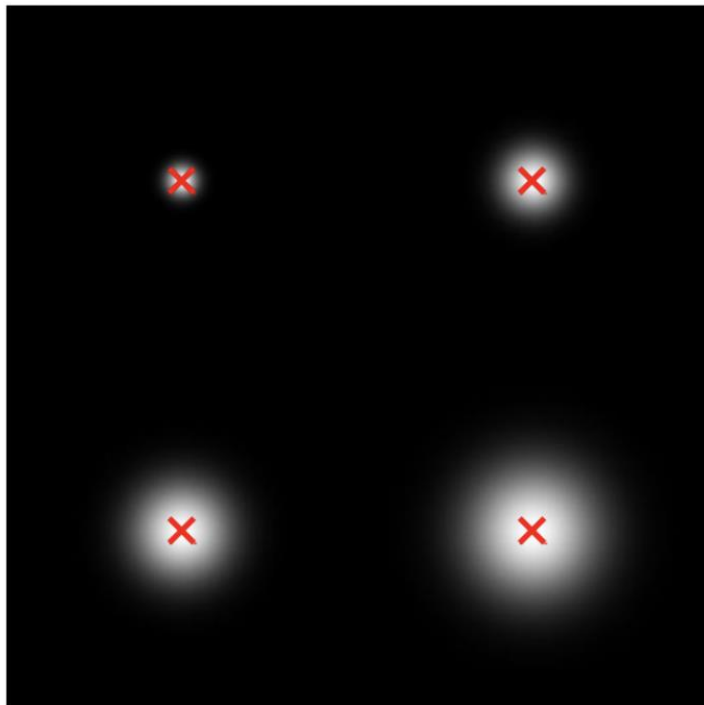
$$\text{Hessian}(x_0, y_0) = H(x_0, y_0) = \begin{bmatrix} I_{xx}(x_0, y_0) & I_{xy}(x_0, y_0) \\ I_{yx}(x_0, y_0) & I_{yy}(x_0, y_0) \end{bmatrix}$$

Note, that $I_{xy}(x_0, y_0) == I_{yx}(x_0, y_0)$

We are looking for the local maxima and minima of the **det H**
Because the signal value is degrading with blur (sigma),
we have to calculate normalized Hessian determinant:

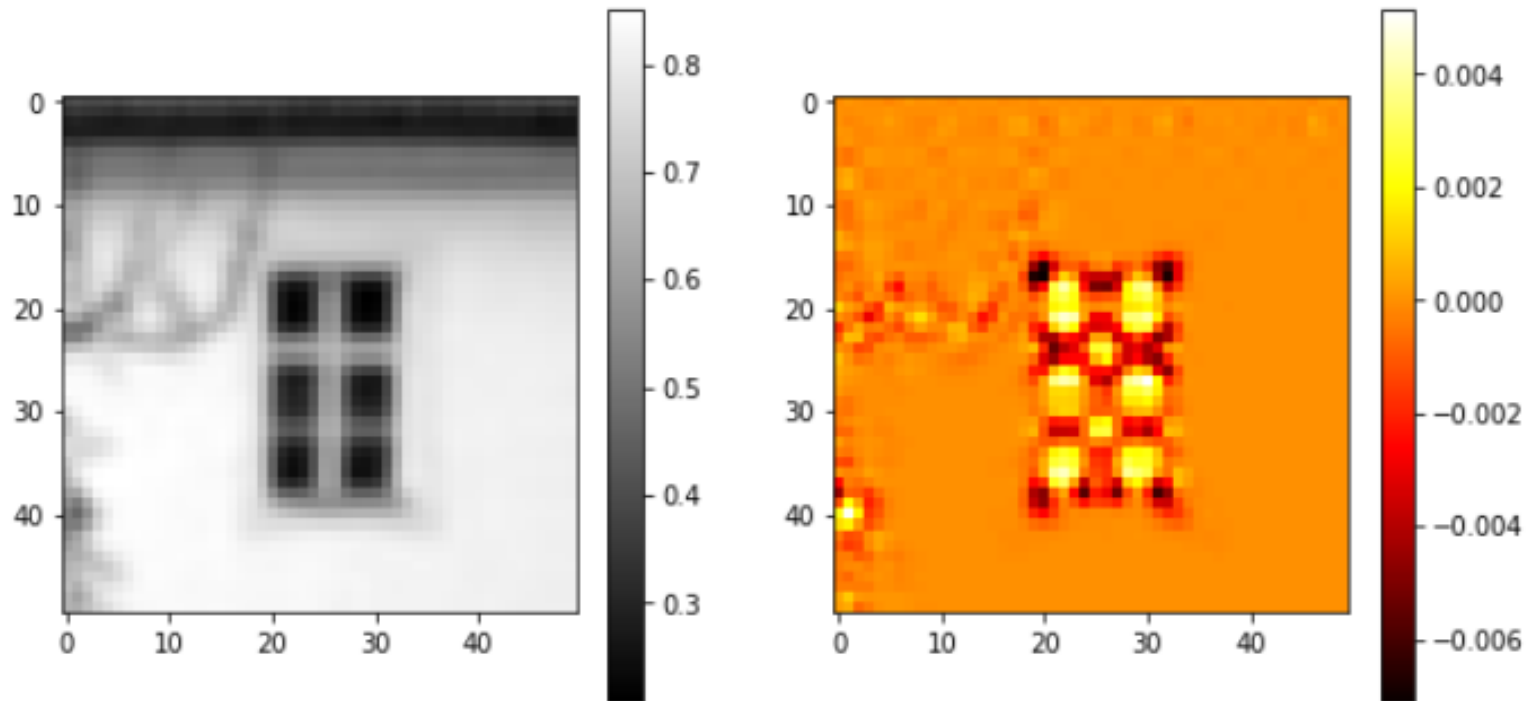
$$H_{\text{norm}}(x_0, y_0) = \sigma^2 \begin{bmatrix} I_{xx}(x_0, y_0) & I_{xy}(x_0, y_0) \\ I_{yx}(x_0, y_0) & I_{yy}(x_0, y_0) \end{bmatrix}$$

- Gaussian scalespace, “stack of gradually smoothed versions” of original image
- Response of **Laplacian** and **the determinant of the Hessian** on Gaussian blobs with standard deviations 8,16,24 and 32 in red **x** points of Gaussian scalespace

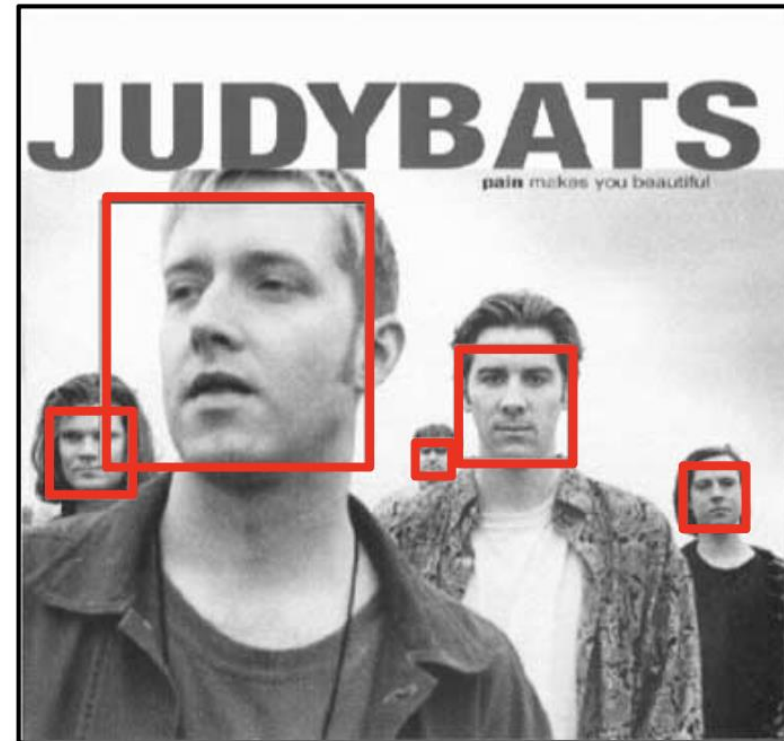


Hessian detector

```
▶ in_img = timg_blured.float()  
out_img_3 = K.feature.hessian_response(in_img, 'sobel')  
  
imshow_torch_channels(torch.cat([in_img, out_img_3], 0)[:,:,: , 80:130,260:310], 0,  
                      colorbar=True, cmap='hot')
```

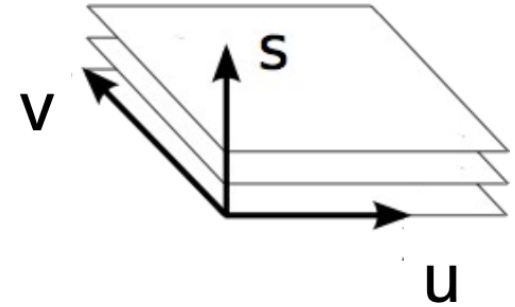


- Objects at different distances have different sizes in the image plane
- In object detection:
 - We want to detect them all
- How?



Example: face
detection

- Basic idea
 - Stack images in a 3D space
 - The third axis, s , is called *scale*
 - $s = 0$ corresponds to the original image
 - As s grows, the image becomes more blurred
- Intuitively: s can be thought of as a *defocus* or *blurring parameter*



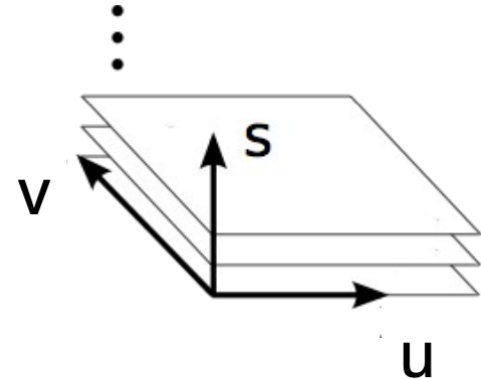
[Iijima, 1959] specifies properties of T_s :

1. Linear
2. Shift-invariant
3. Semi-group property
4. Scale- and rotation-invariant
5. Maintain positivity

6. Separability (by later authors)

- A1+A2: T_s is a convolution
 - original image $I_o(u,v)$
 - blur kernel $g_s(u,v)$
 - The scale space of I_o is given as the convolution:

$$I_s(u, v) = (g_s * I_o)(u, v)$$



In the Fourier domain: $F\{I_s\} = G_s \cdot F\{I_o\}$

- The remaining axioms lead to a unique formulation of G_s as a Gaussian function:

$$G_s(\omega_u, \omega_v) = e^{-s \frac{\omega_u^2 + \omega_v^2}{2}} \quad g_s(u, v) = \frac{1}{2\pi s} e^{-\frac{u^2 + v^2}{2s}}$$

- Separability:

$$g_s(u, v) = \frac{1}{\sqrt{2\pi s}} e^{-\frac{u^2}{2s}} \frac{1}{\sqrt{2\pi s}} e^{-\frac{v^2}{2s}}$$

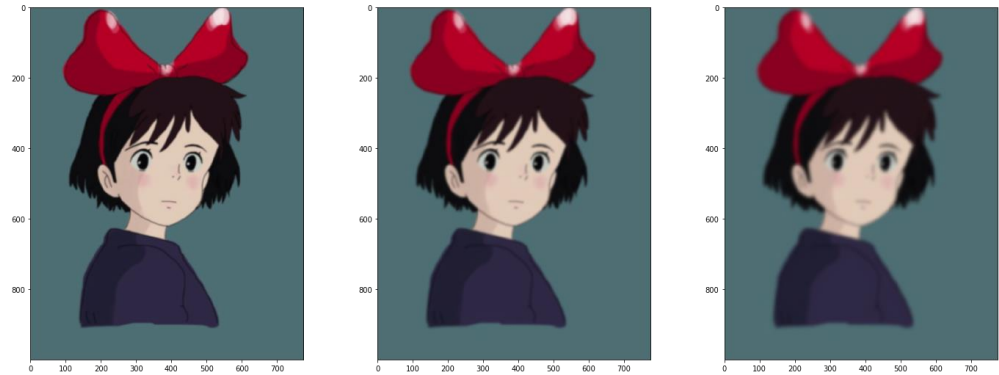
- Blurring (LP-filtering) reduces high frequencies
- At some scale s_o : frequencies over $\pi/2$ are sufficiently attenuated to allow down-sampling with a factor 2 without much aliasing
- At scale $2s_o$ we can down-sample the image with a factor 4, etc.

```
▶ scalePyr = K.ScalePyramid()  
pyr_img, sigmas, downscale_factors = scalePyr(timg)  
for level in pyr_img:  
    print (level.shape)  
print ([sigmas[0]])  
  
↳ torch.Size([1, 3, 6, 1000, 776])  
torch.Size([1, 3, 6, 500, 388])  
torch.Size([1, 3, 6, 250, 194])  
torch.Size([1, 3, 6, 125, 97])  
torch.Size([1, 3, 6, 62, 48])  
torch.Size([1, 3, 6, 31, 24])  
tensor([[1.6000, 2.0159, 2.5398, 3.2000, 4.0317, 5.0797]])
```

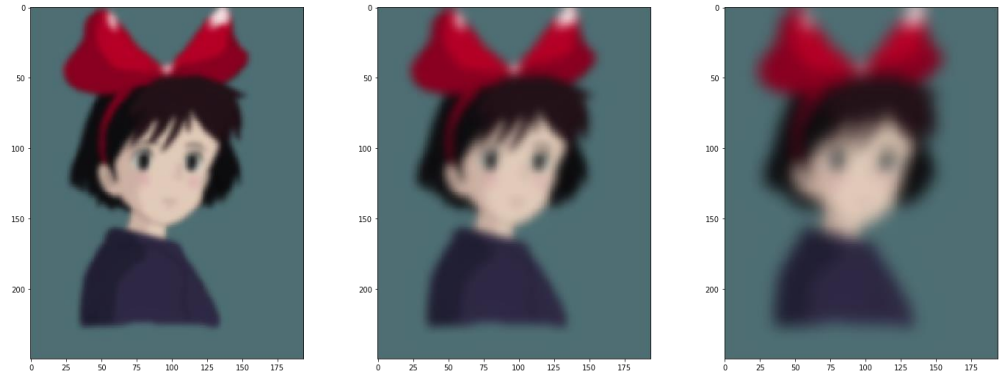
6 levels, each is contains 6 blur layers itself

Kiki Scale Space

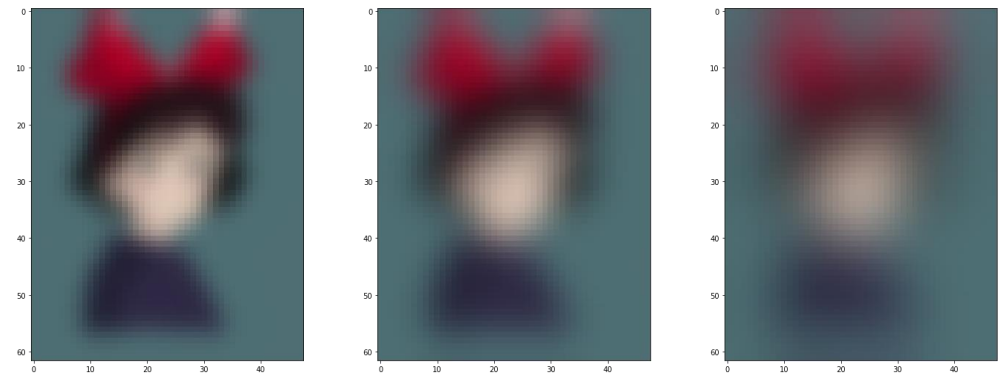
Level 0, 776 x 1000 px



Level 2, 192 x 250 px



Level 4, 48 x 61 px



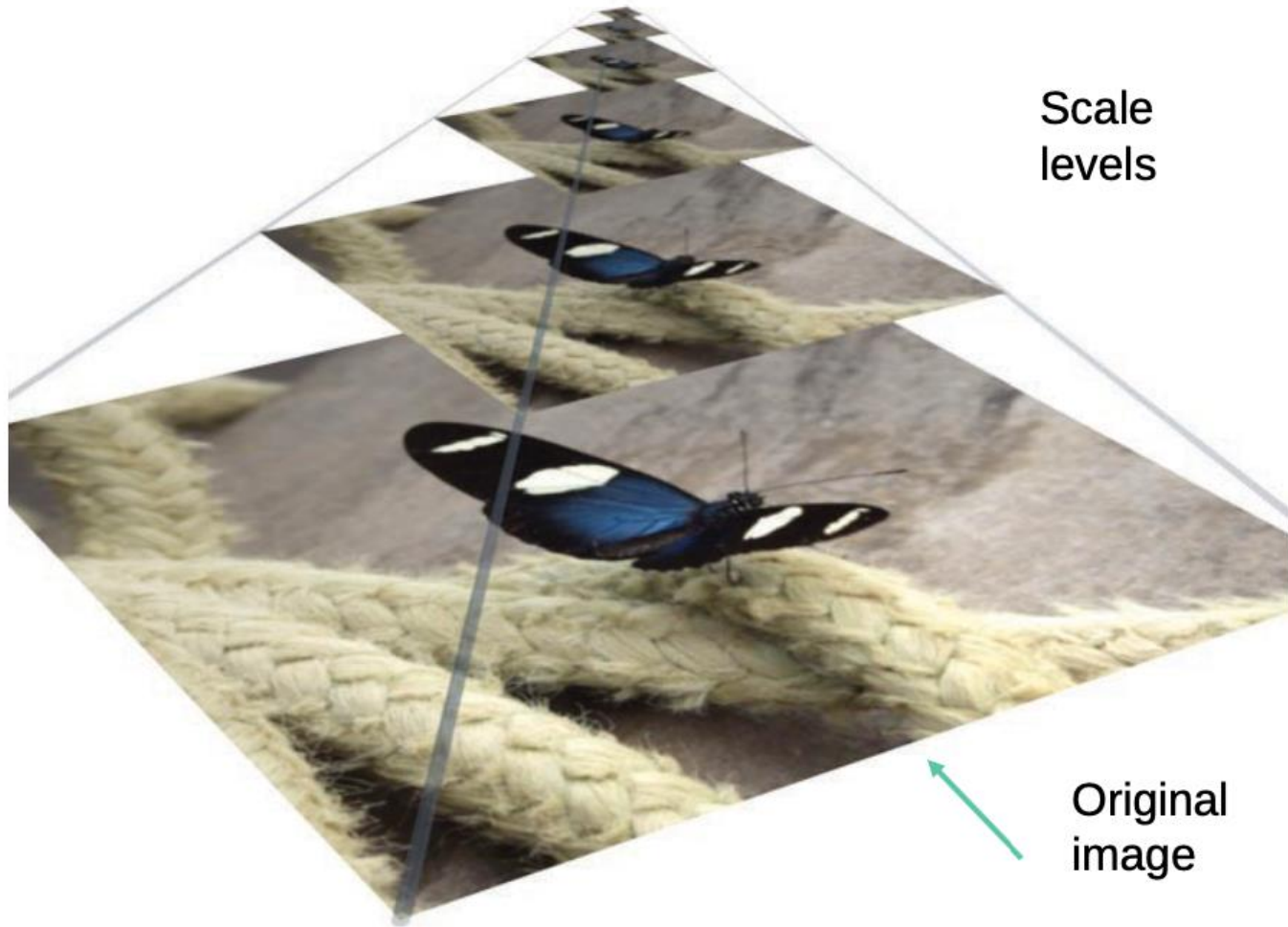
Scale space example

Coarser
scale

Scale
levels

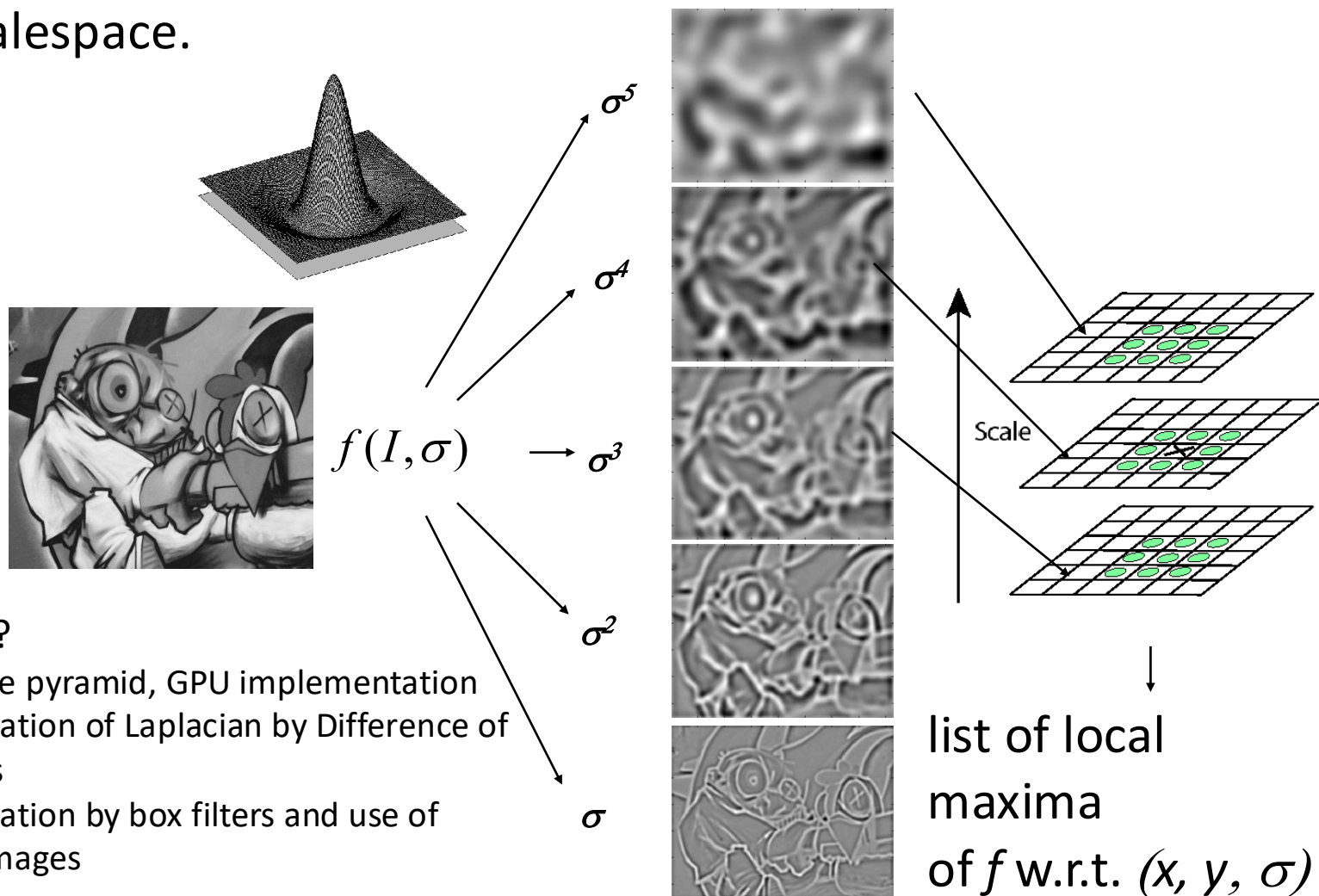
Finer
scale

Original
image



Scale invariant detectors

Scale invariant detectors, find local extrema (both in space and scale) of **Laplacian** (f in example) or **determinant of Hessian** response in gaussian scalespace.



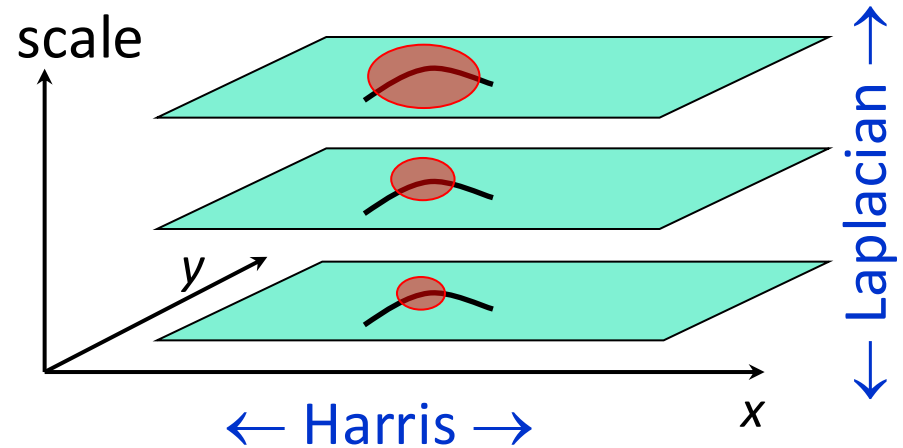
Faster detection?

- Scalespace pyramid, GPU implementation
- Approximation of Laplacian by Difference of Gaussians
- Approximation by box filters and use of integral images

Harris-Laplacian¹

Find local maximum of:

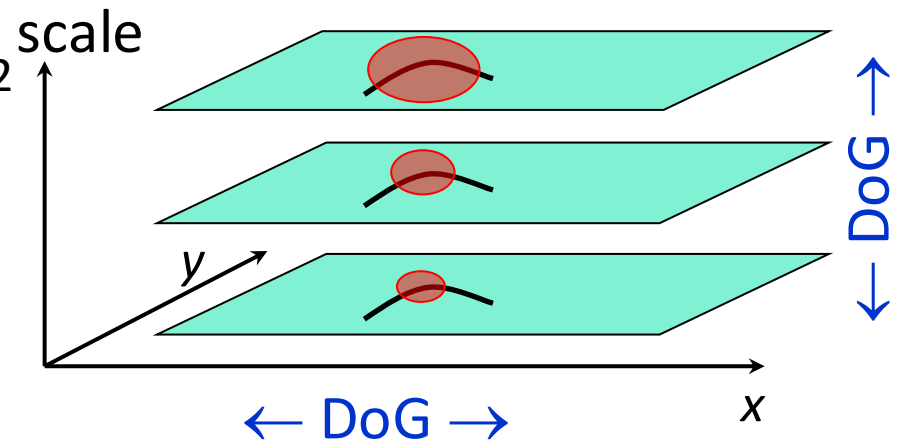
- Harris corner detector in space (image coordinates)
- Laplacian in scale



Laplacian-Laplacian = "SIFT"(Lowe)²

Find local maximum of:

- Difference of Gaussians in space and scale



Other options: Hessian-Laplacian, **Hessian-Hessian**, etc ...

Harris does not work that well for scale selection

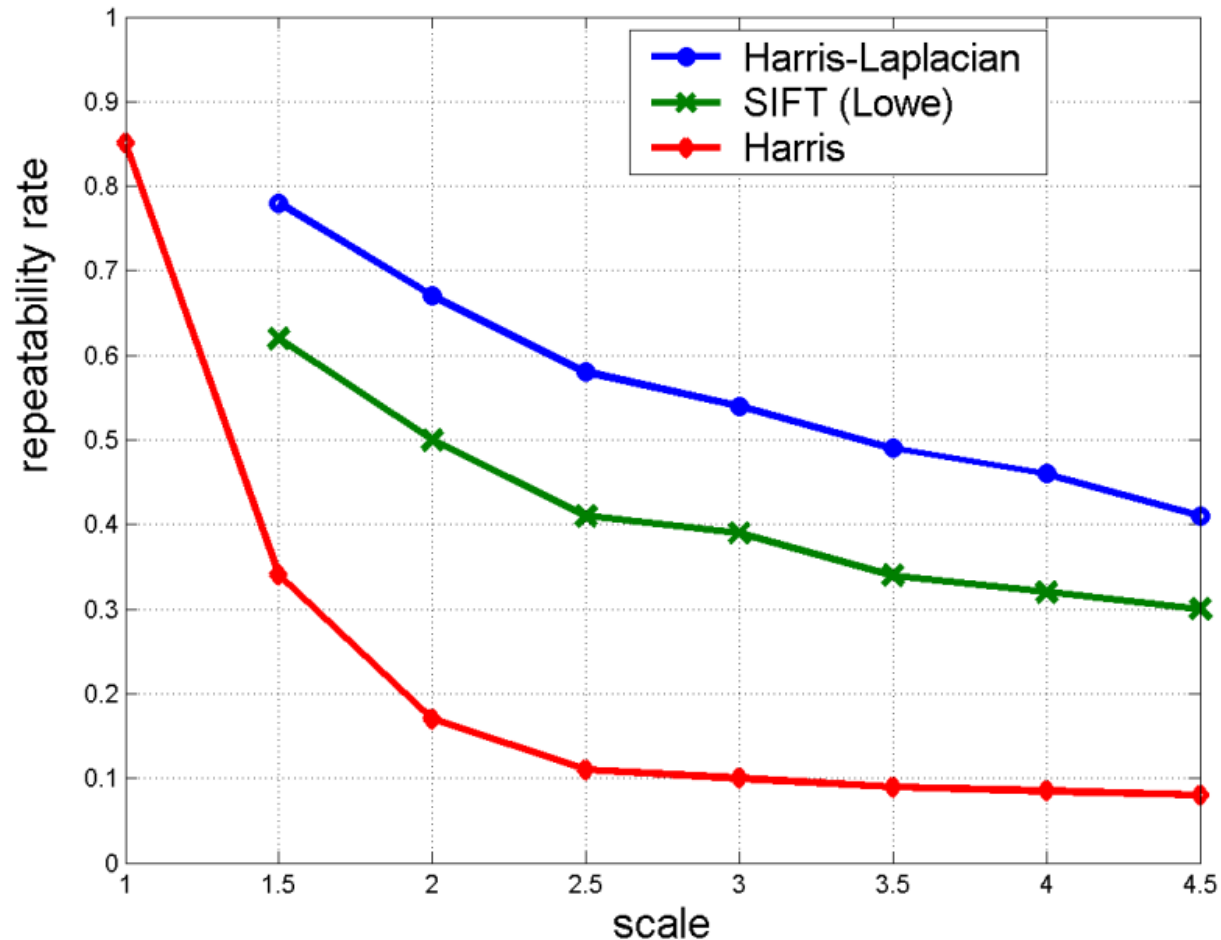
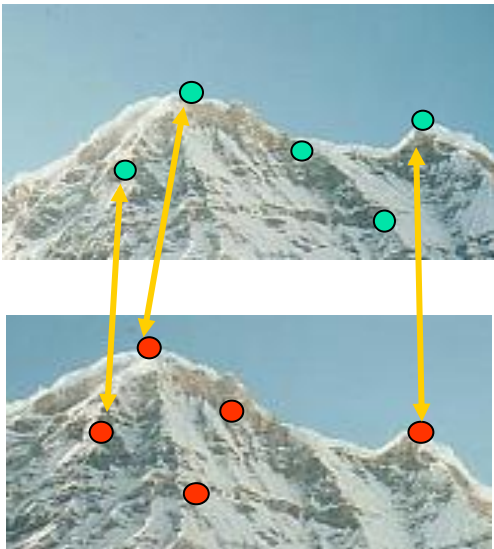
¹ K.Mikolajczyk, C.Schmid. "Indexing Based on Scale Invariant Interest Points". ICCV 2001

² D.Lowe. "Distinctive Image Features from Scale-Invariant Keypoints". IJCV 2004

Experimental evaluation of detectors w.r.t. scale change

Repeatability rate:

$$\frac{\# \text{ correspondences}}{\# \text{ possible correspondences}}$$



Detector:

- Scale-space peaks of Difference-of-Gaussians filter response (Lindeberg 1995)
- Sub-pixel quadratic interpolation
- **Similarity frame** from modes of gradient histogram

SIFT Descriptor:

- Local histograms of gradient orientation
- Allows for small misalignments
=> robust to non-similarity transforms

Indexing :

- kD-tree structure

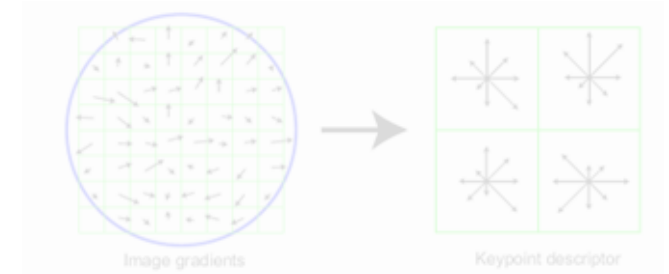
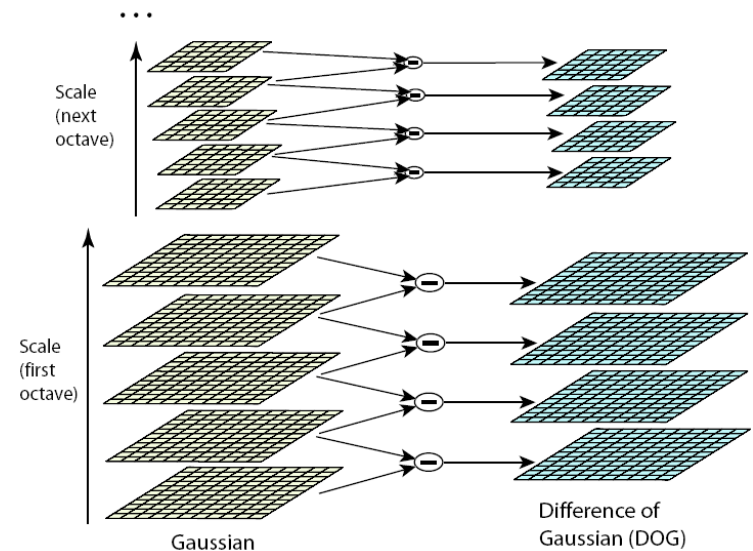
Matching:

- test on euclidean distance of 1st and 2nd match

Verification:

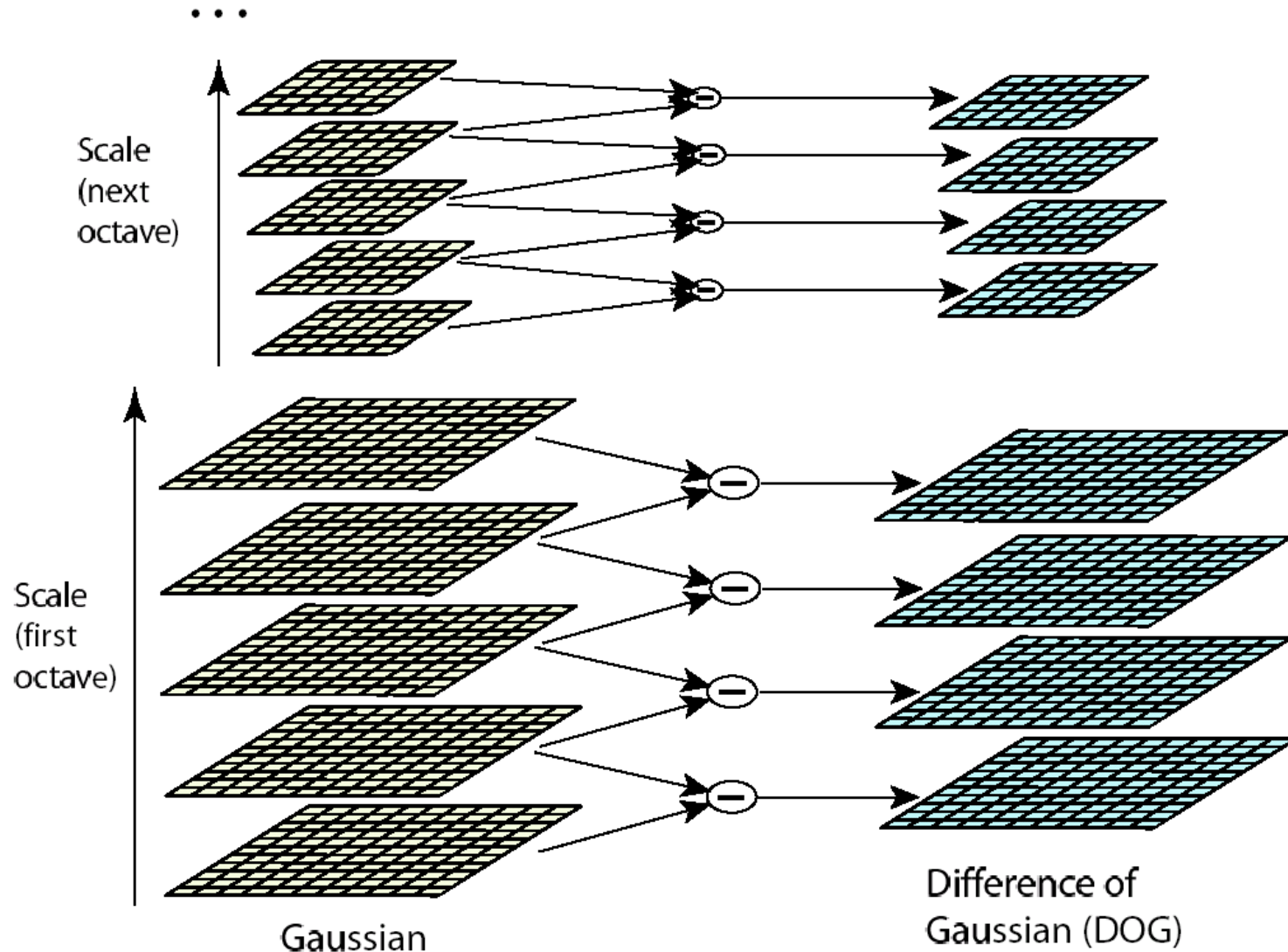
- Hough transform based clustering of correspondences with similar transformations

Fast, efficient implementation, **real-time recognition**



D. G. Lowe: “Distinctive image features from scale-invariant keypoints”. IJCV, 2004.

Scale space processed one octave at a time

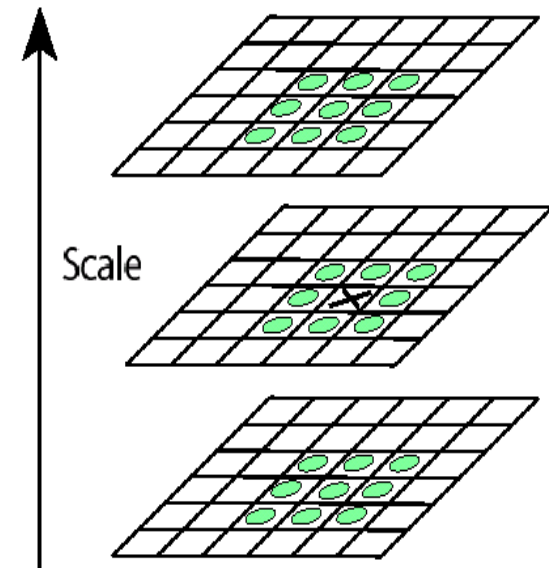


- Detect maxima and minima of difference-of-Gaussian in scale space
- Fit a quadratic to surrounding values for sub-pixel and sub-scale interpolation (Brown & Lowe, 2002)
- Taylor expansion around point:

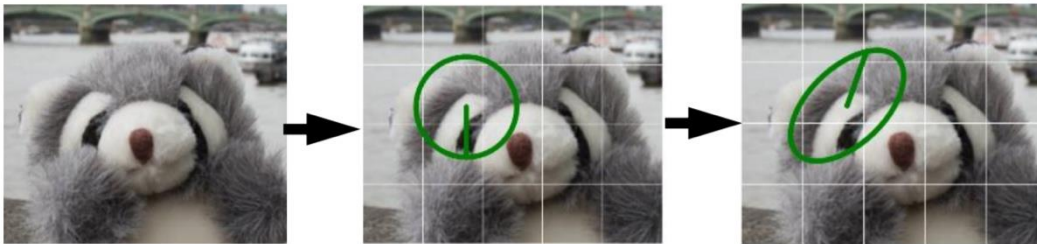
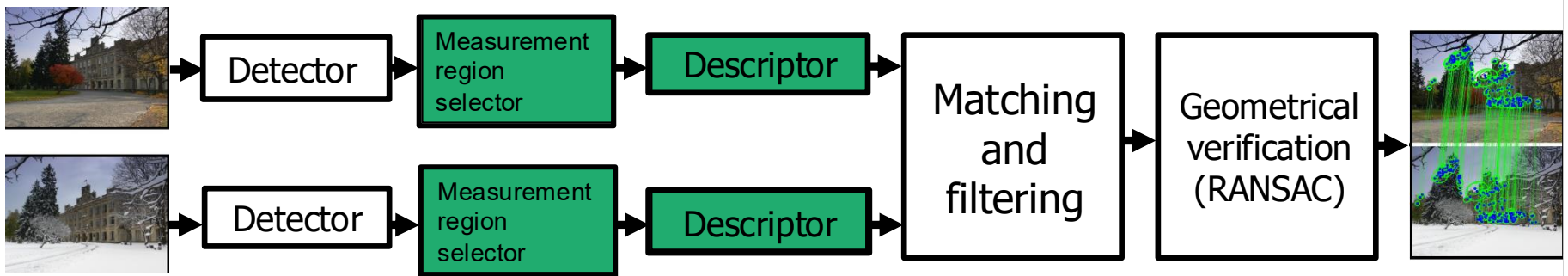
$$D(\mathbf{x}) = D + \frac{\partial D^T}{\partial \mathbf{x}} \mathbf{x} + \frac{1}{2} \mathbf{x}^T \frac{\partial^2 D}{\partial \mathbf{x}^2} \mathbf{x}$$

- Offset of extremum
(use finite differences for derivatives):

$$\hat{\mathbf{x}} = -\frac{\partial^2 D^{-1}}{\partial \mathbf{x}^2} \frac{\partial D}{\partial \mathbf{x}}$$



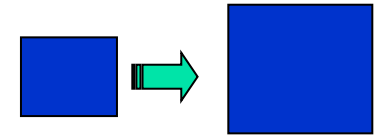
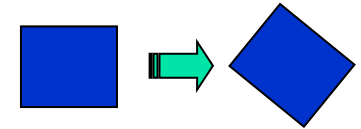
Invariant local patch description



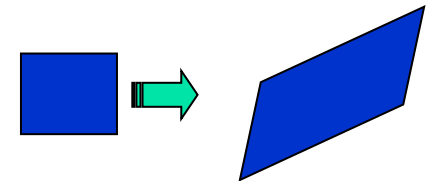
- We have detected the keypoints/regions up to scale, i.e. we have (x, y, and circle radius).
 - How could we describe it in viewpoint-invariant way?
 - Two approaches:
 - a) Design an invariant descriptor, which does not depend on orientation or viewpoint.
 - b) Design an invariant procedure to estimate a geometric normalization parameters
- (a) is older, performs worse, almost abandoned.

■ Geometry

- Rotation
- Similarity (rotation + uniform scale)

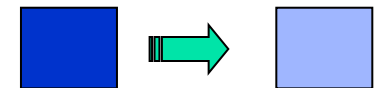


- Affine (scale dependent on direction)
valid for: orthographic camera, locally planar object



■ Photometry

- Affine intensity change ($I \rightarrow a I + b$)



■ Patch normalization under different transformations

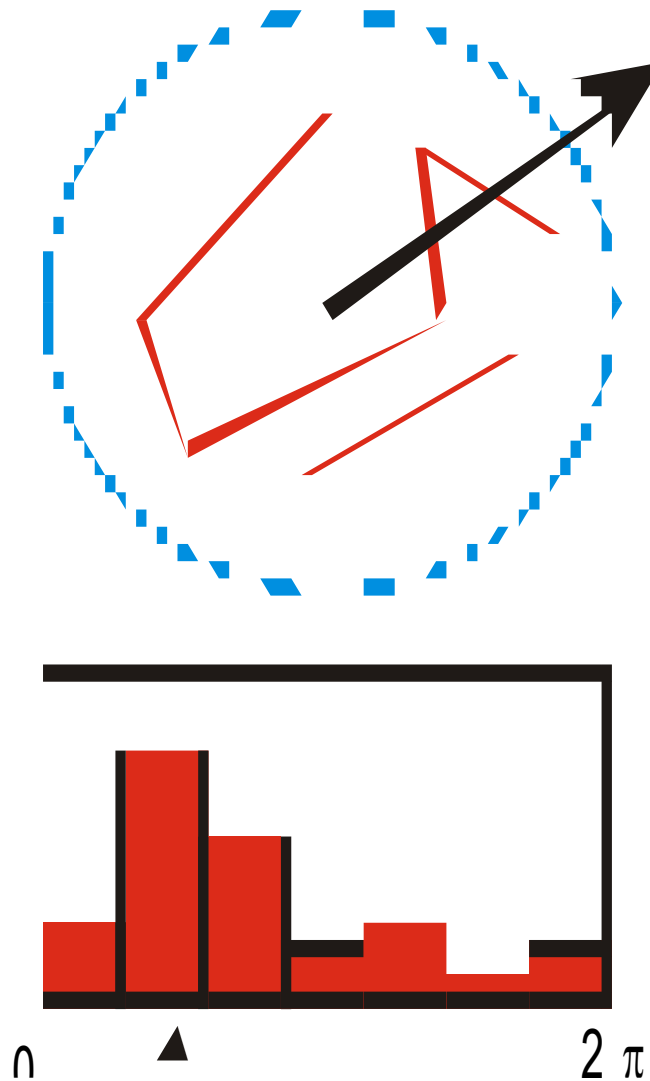
Original image	Translation	Translation and rotation	Similarity	Affine transformation
				
				

Example of geometric normalization, with different classes of transformation (author of images: Štěpán Obdržálek).

Select canonical orientation(s)

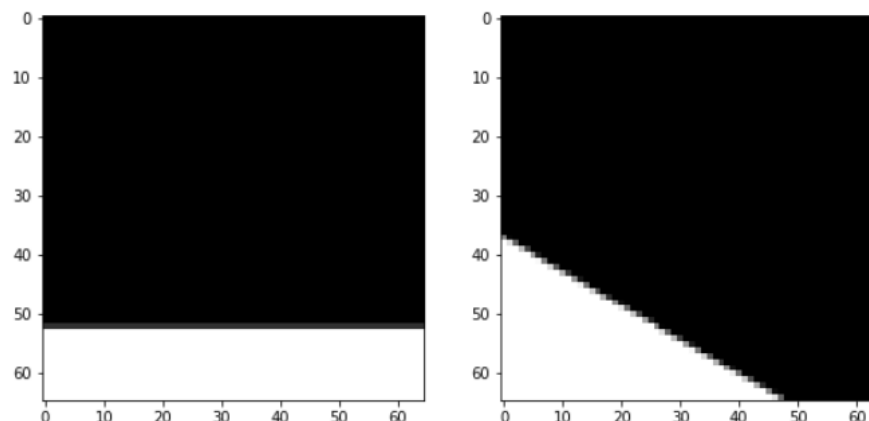
- Compute a histogram of local gradient directions computed at the selected scale
- Assign canonical orientation(s) at peak(s) of smoothed histogram
- (x, y, scale) + orientation defines a local *similarity frame*; equivalent to detecting 2 distinguished points

Note: if orientation of the object (image) is known, it may replace this construction



Building a Similarity Frame(s)

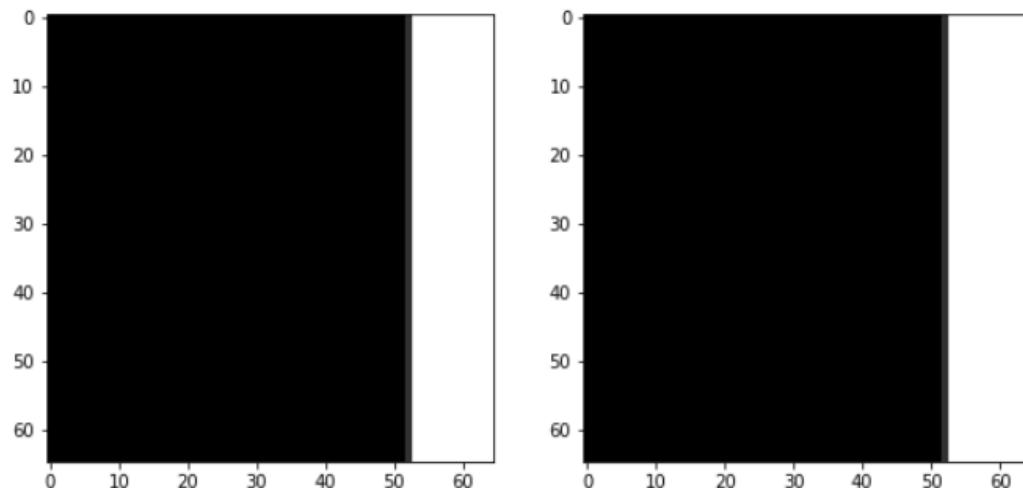
Input patches



```
Ori1 = KF.LAFOrienter(32,36)
lafs_ori = Ori1(lafs, artifical_image)
patches_rot = KF.extract_patches_from_pyramid(artifical_image,lafs_ori,65).squeeze(1)

imshow_torch_channels(patches_rot[0], dim=0, cmap='gray')
```

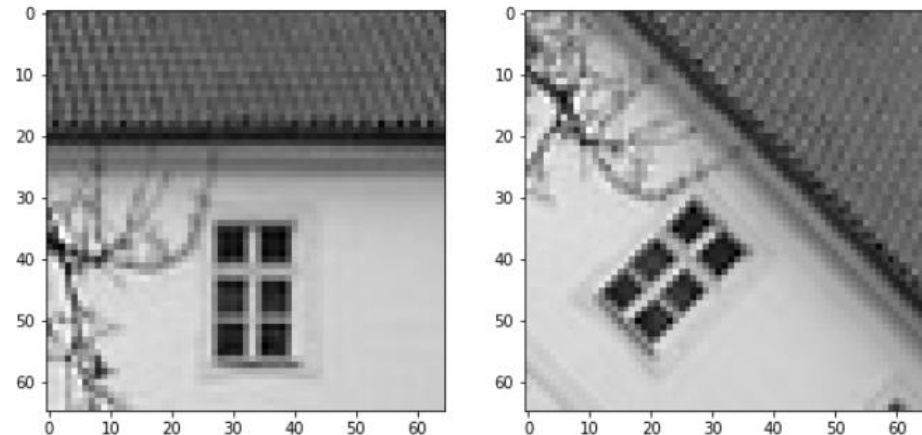
Output patches



Code available [here](#)

Building a Similarity Frame(s)

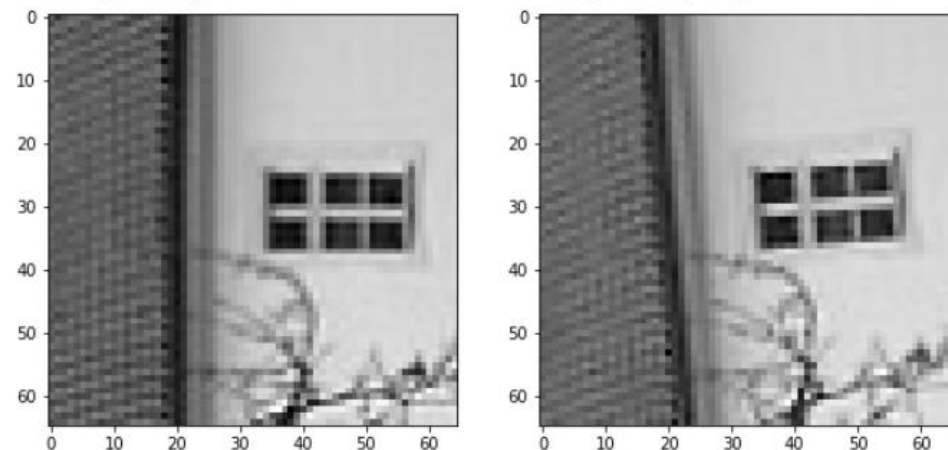
Input patches



```
Ori1 = KF.LAFOrienter(32,36)
lafs_ori = Ori1(lafs, timg)
patches_rot = KF.extract_patches_from_pyramid(timg,lafs_ori,65).squeeze(1)

imshow_torch_channels(patches_rot[0], dim=0, cmap='gray')
```

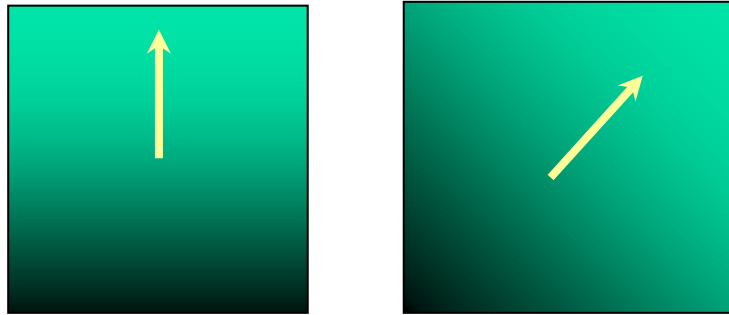
Output patches



Code available [here](#)

- Find local orientation

Dominant direction of gradient (prev. slide)



- Compute image derivatives relative to this orientation
or
- Extract the normalized patch

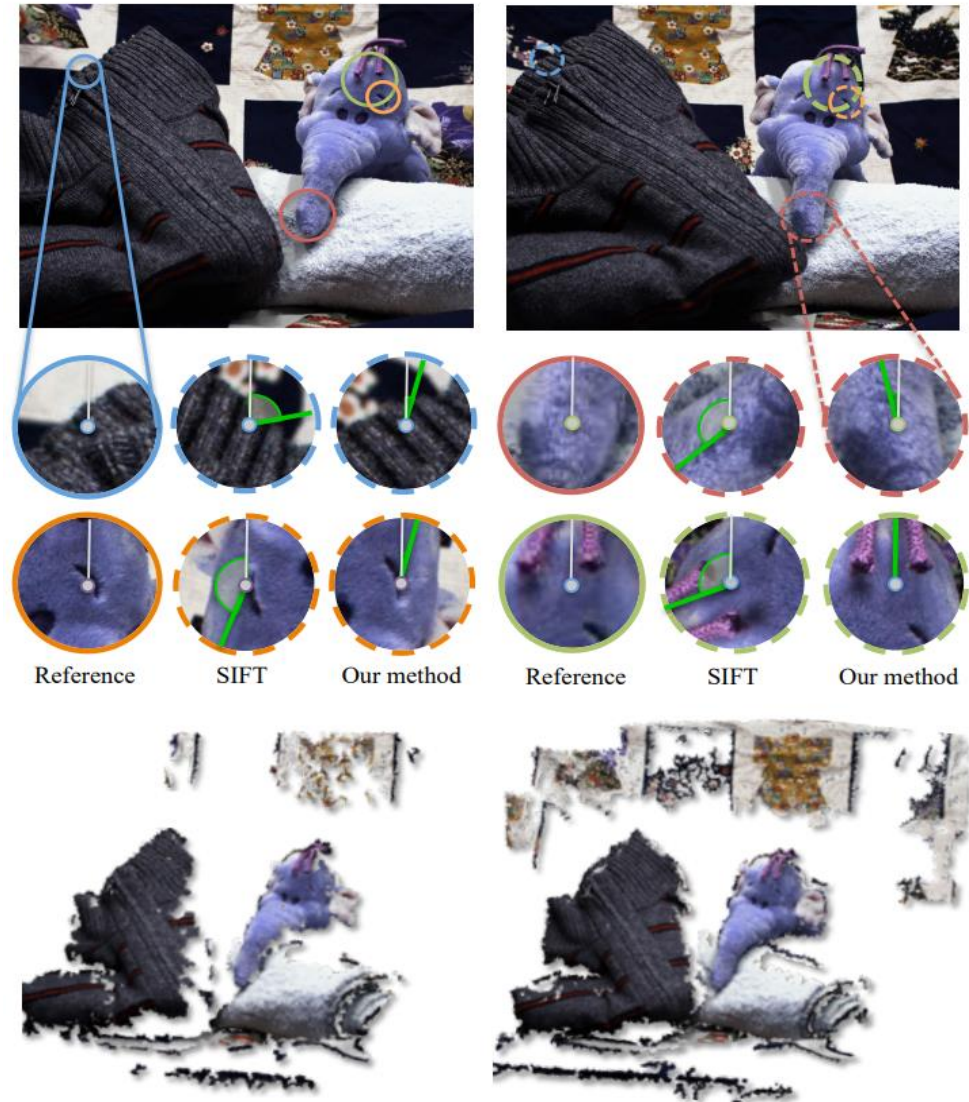
¹ K.Mikolajczyk, C.Schmid. "Indexing Based on Scale Invariant Interest Points". ICCV 2001

² D.Lowe. "Distinctive Image Features from Scale-Invariant Keypoints". IJCV 2004

Patch orientation estimation

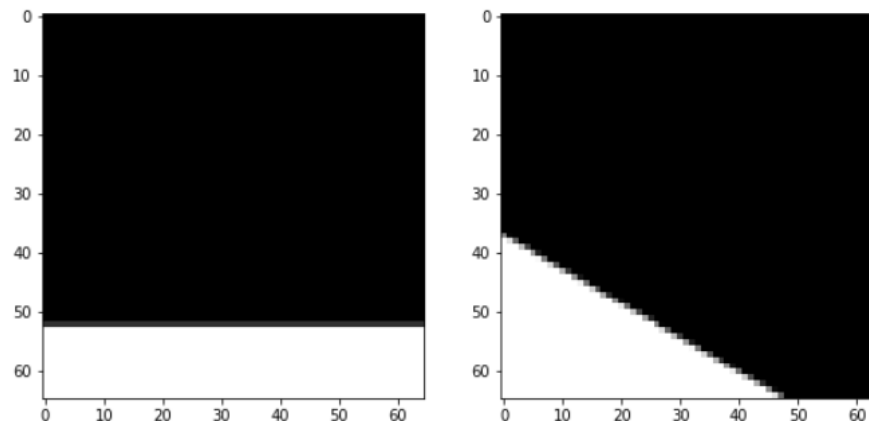
Learned orientation regression:
Patch $\rightarrow$ CNN $\rightarrow$ angle

For details – see the next lecture



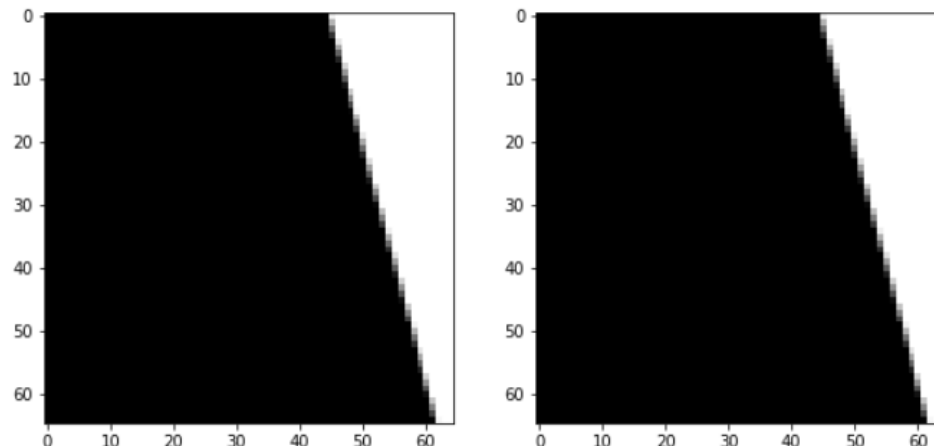
Building a Similarity Frame(s): OriNet

Input patches



```
Ori2 = KF.LAFOrienter(32, angle_detector=KF.OriNet(True))  
lafs_ori2 = Ori2(lafs, artifical_image)  
patches_rot2 = KF.extract_patches_from_pyramid(artifical_image,lafs_ori2, 65).squeeze(1)  
  
imshow_torch_channels(patches_rot2[0], dim=0, cmap='gray')
```

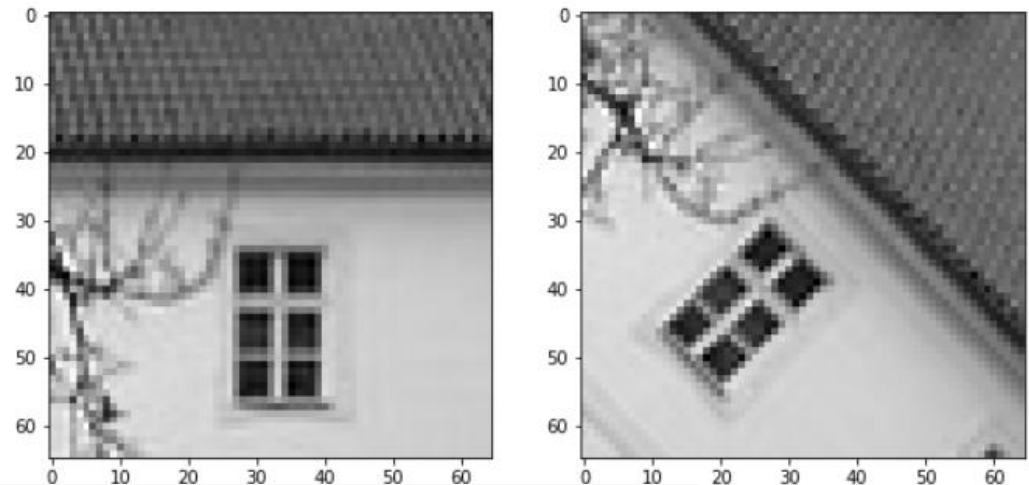
Output patches



Code available [here](#)

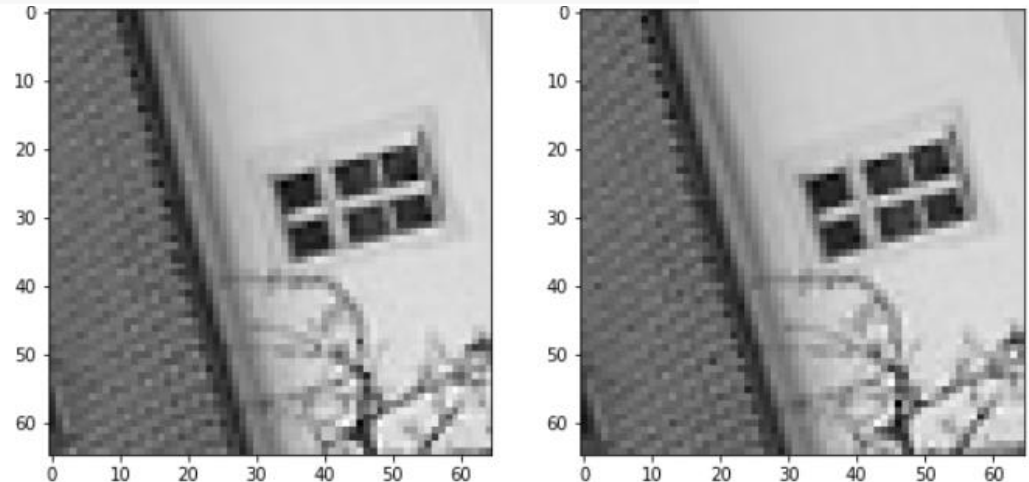
Building a Similarity Frame(s)

Input patches



```
Ori2 = KF.LAFOrienter(32, angle_detector=KF.OriNet(True))  
lafs_ori2 = Ori2(lafs, timg)  
patches_rot2 = KF.extract_patches_from_pyramid(timg, lafs_ori2, 65).squeeze(1)  
  
imshow_torch_channels(patches_rot2[0], dim=0, cmap='gray')
```

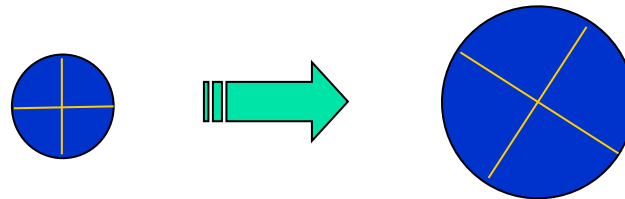
Output patches



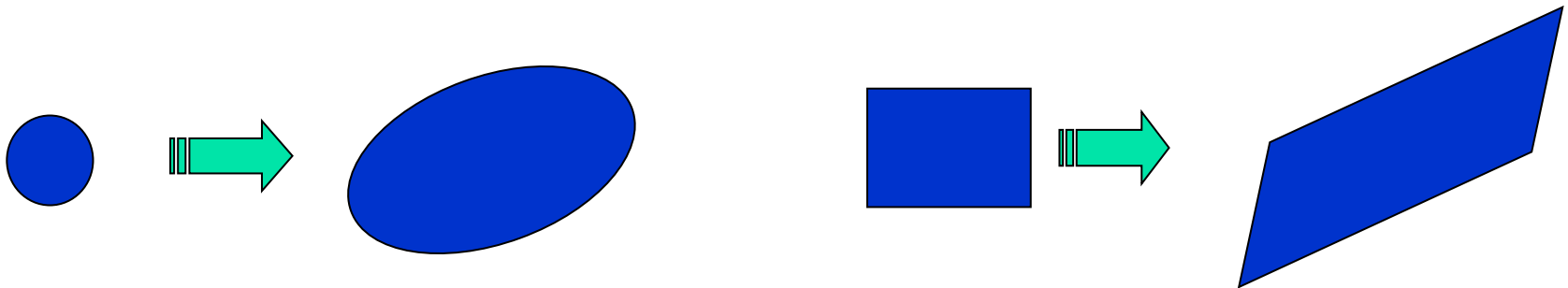
Code available [here](#)

Affine Invariant Detection

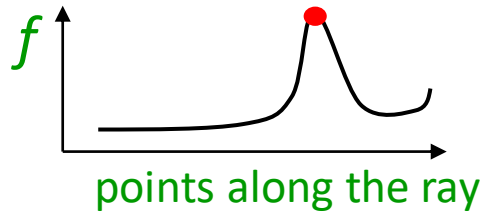
- We have considered:
Similarity transform (rotation + uniform scale)



- Now we go on to:
Affine transform (rotation + non-uniform scale)



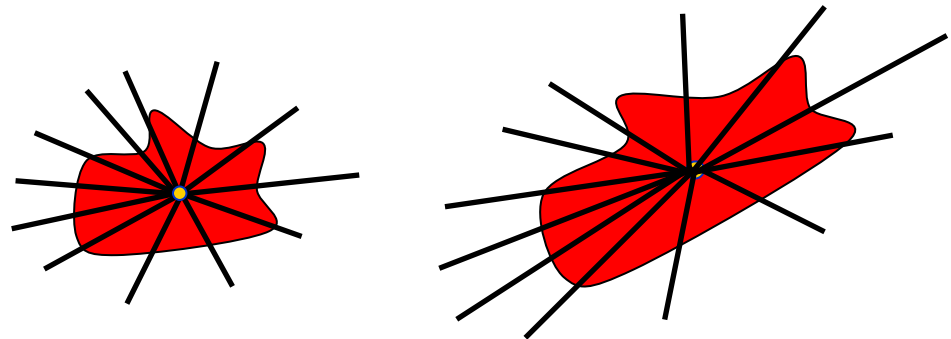
- Take a local intensity extremum as initial point
- Go along every ray starting from this point and stop when extremum of function f is reached



$$f(t) = \frac{|I(t) - I_0|}{\frac{1}{t} \int_0^t |I(t) - I_0| dt}$$

- We will obtain approximately corresponding regions

Remark: we search for scale in every direction



- The regions found may not exactly correspond, so we approximate them with **ellipses**

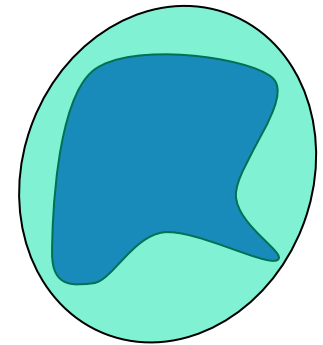
- Geometric Moments:

$$m_{pq} = \int_{\mathbb{R}^2} x^p y^q f(x, y) dx dy$$

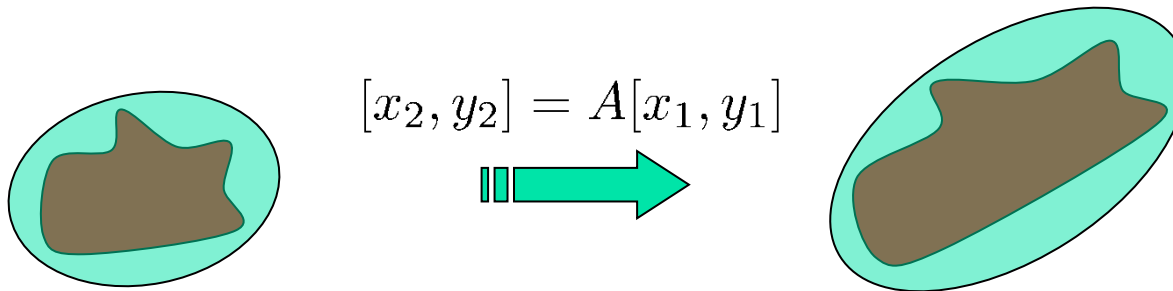
Fact: moments m_{pq} uniquely determine the function f

Taking f to be the characteristic function of a region (1 inside, 0 outside), moments of orders up to 2 allow to approximate the region by an ellipse

This ellipse will have the same moments of orders up to 2 as the original region



- Covariance matrix of region points defines an ellipse:



$$[x_1, y_1]^T \Sigma_1^{-1} [x_1, y_1] = 1$$

$$\Sigma_1 = \langle [x_1, y_1][x_1, y_1]^T \rangle_{\text{region}_1}$$

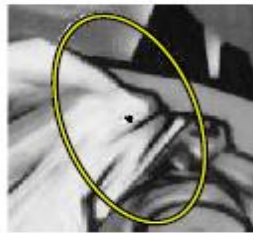
$$[x_2, y_2]^T \Sigma_2^{-1} [x_2, y_2] = 1$$

$$\Sigma_2 = \langle [x_2, y_2][x_2, y_2]^T \rangle_{\text{region}_2}$$

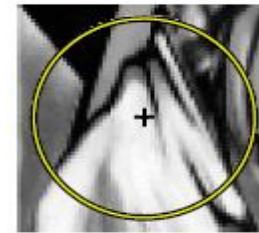
$$\Sigma_2 = A \Sigma_1 A^T$$

Ellipses, computed for corresponding regions, also correspond!

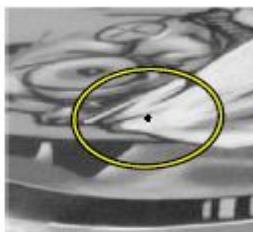
1. Detect initial region with Harris or Hessian detector and select the scale
2. Estimate the shape with the second moment matrix
3. Normalize the affine region to the circular one
4. Go to step 2 if the eigenvalues of the second moment matrix for the new point are not equal



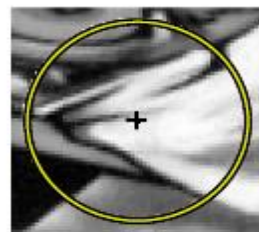
$$[x_1, y_1] \rightarrow M_1^{-1/2}[x'_1, y'_1]$$



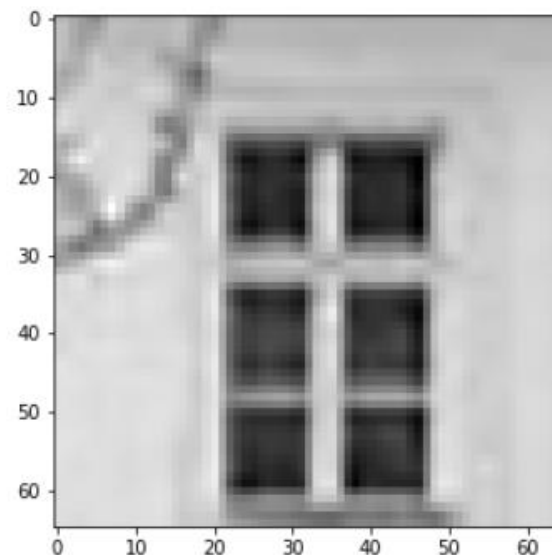
$$[x'_1, y'_1] \xrightarrow{\downarrow} R[x'_2, y'_2]$$



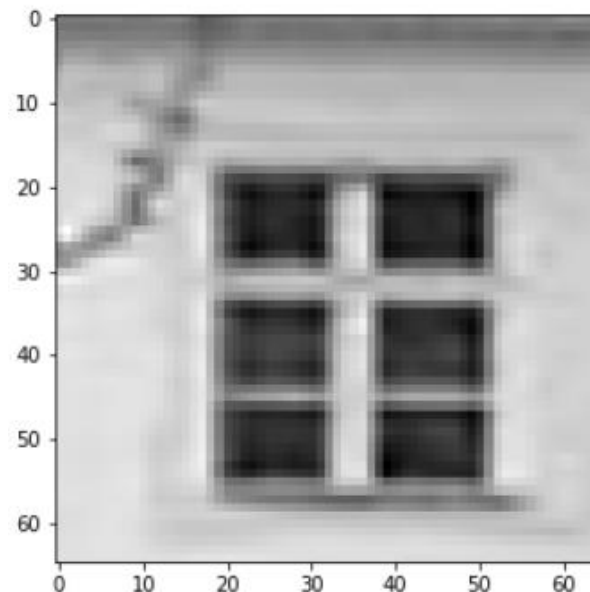
$$[x_2, y_2] \rightarrow M_2^{-1/2}[x'_2, y'_2]$$



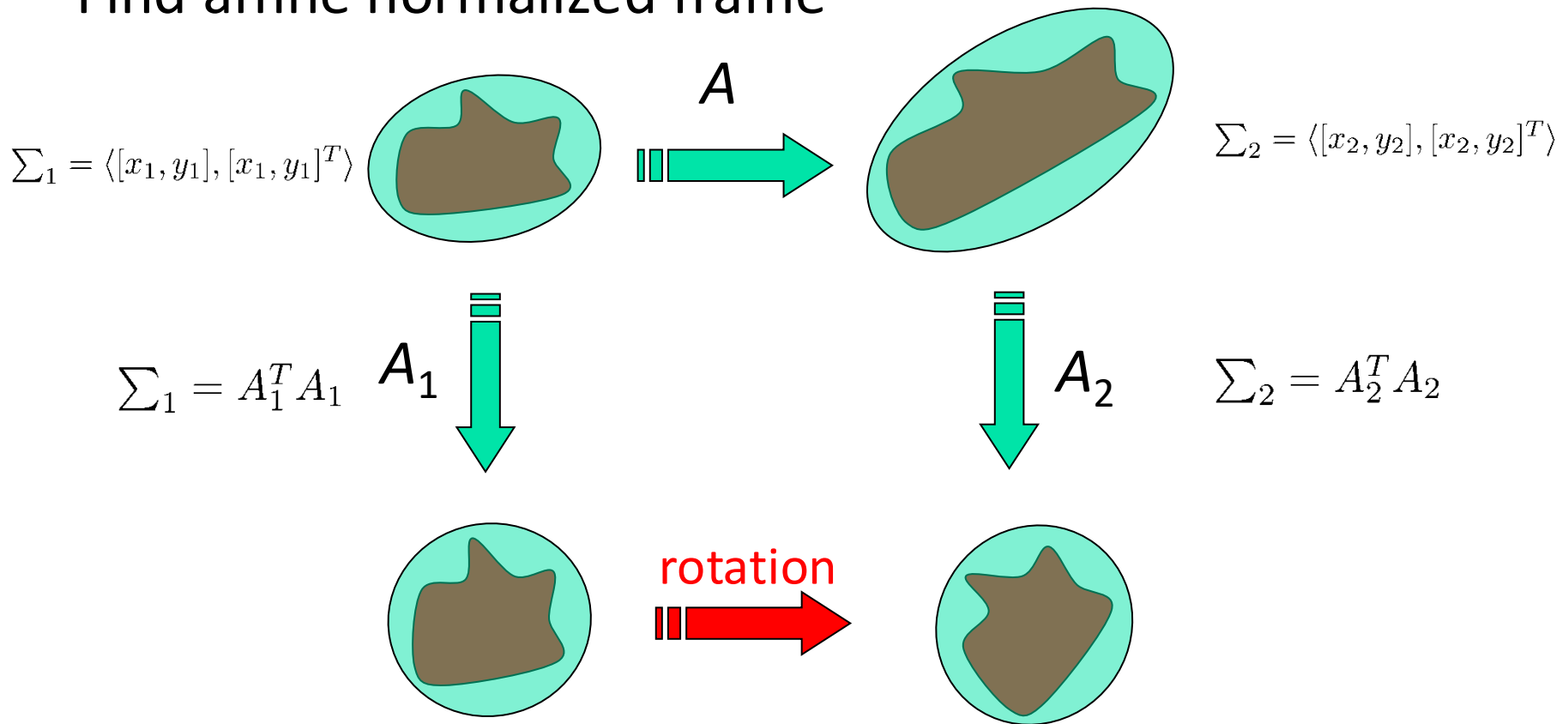
Affine shape adaptation example



After adaptation



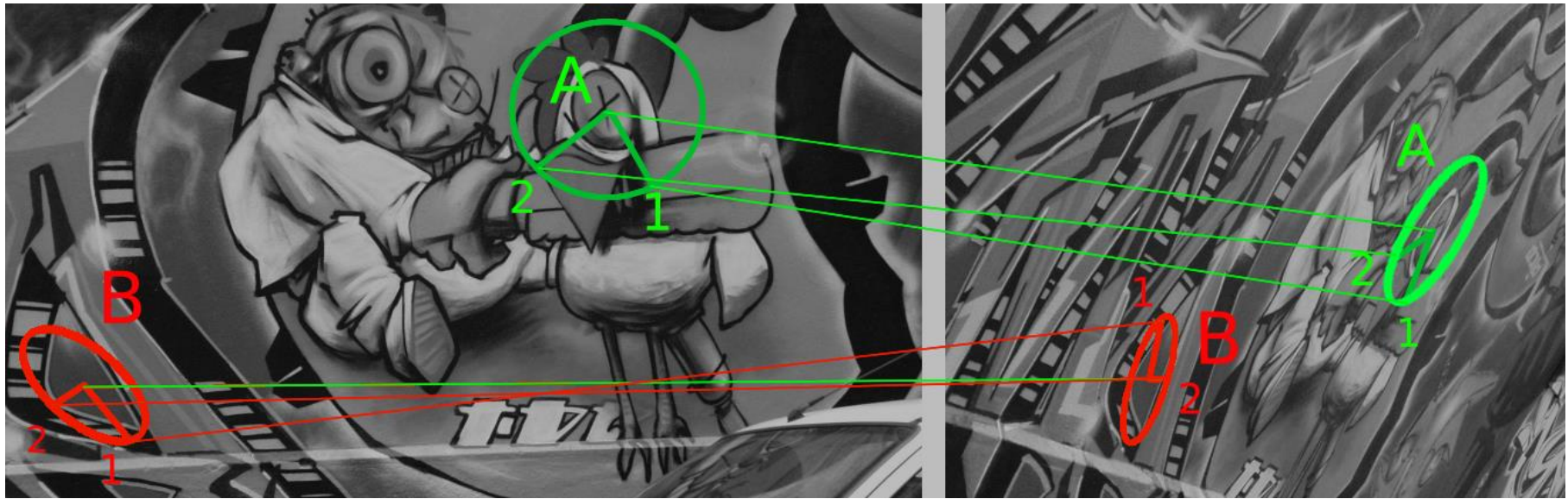
- Find affine normalized frame



- Compute rotational invariant descriptor in this normalized frame, or estimate the rotation

Additional usages for affine regions

Once we have found local affine features (oriented ellipses), we can use them for more than just patch normalization!



Local affine correspondences example.

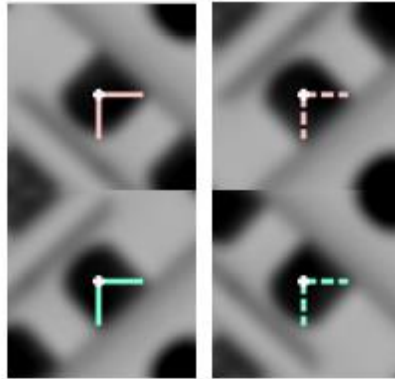
While centers of both regions A and B are correct point matches, only A is a correct affine correspondence.

■ Image repeated patterns for image rectification

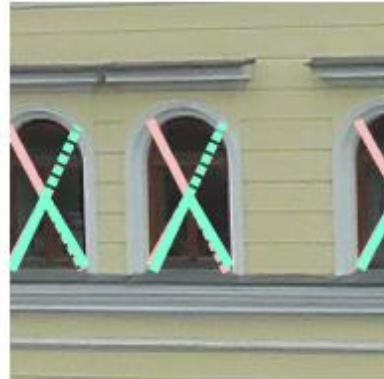
(a) MSER detection



(b) Normalized Frame



(c) LAF representation



Radially-Distorted Conjugate Translations **J. Pritts**, Z. Kukelova, V. Larsson, O. Chum CVPR 2018

■ Surface normals estimation

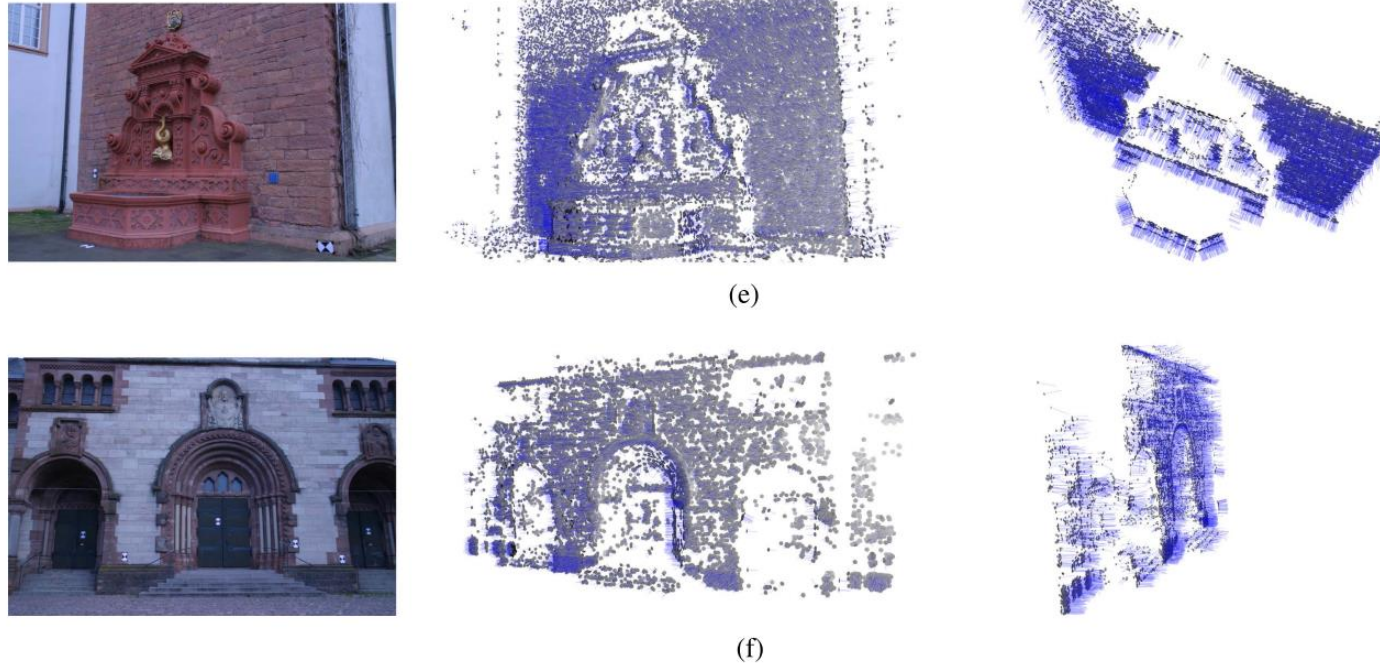
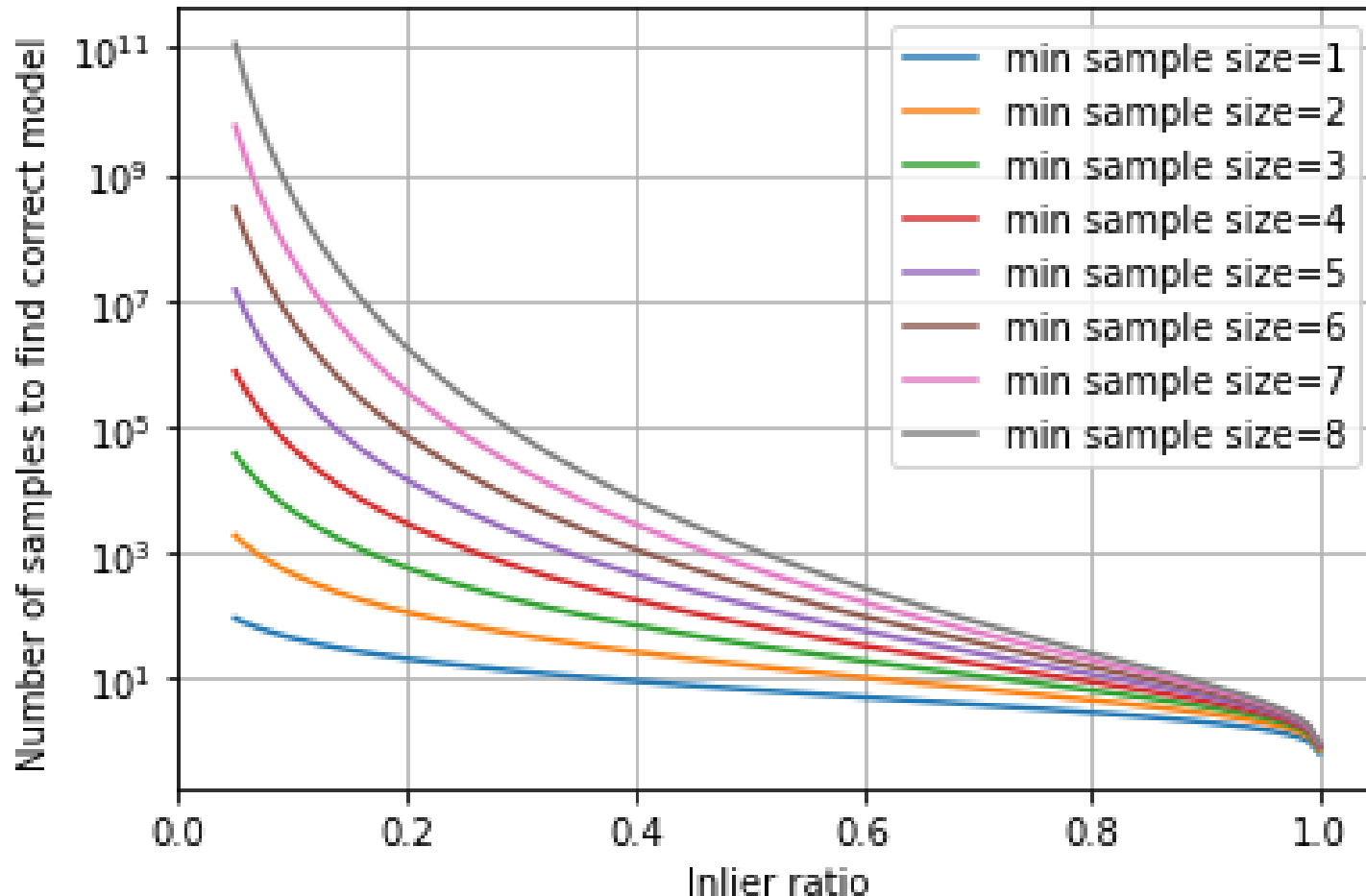


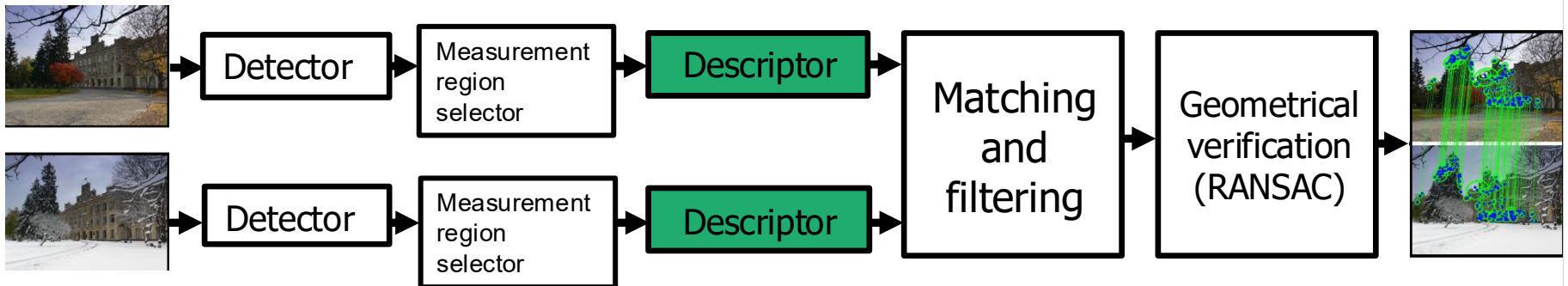
Fig. 6. Example results from each dataset. The first column is an image from the sequence, the remaining ones show the estimated normals (blue lines) and the triangulated points (gray patches) from different view-points. (a) courtyard (ETH3D). (b) relief (ETH3D). (c) books2 (Pusztai). (d) bag (Pusztai). (e) fountain-p11 (Strecha). (f) herz-jesus-p25 (Strecha).

Baráth D., Eichhardt I. and Hajder L., *Optimal Multi-View Surface Normal Estimation Using Affine Correspondences*, IEEE Transactions on Image Processing, 2019

- Speeding-up RANSAC (in details – in the next lectures) for geometry estimation



Local patch descriptors



What do we want from the local descriptor?

Local descriptor encodes a local patch appearance in a compact form.

- Discriminativeness
- Robustness to nuisance factors
- Small and fast to compute

The main reason why we need a local feature descriptor is efficiency:

- Correspondences can be established by finding nearest neighbors by Euclidean (float) or Hamming (binary) distance.
 - Which can be efficiently calculated via approximate NN libraries, e.g. FLANN, FAISS, etc.
- Can be avoided, e.g. the matching done by energy minimization, but more costly.
- See Optical flow (no descriptor, special constraints)

Detector:

- Scale-space peaks of Difference-of-Gaussians filter response (Lindeberg 1995)
- **Similarity frame** from modes of gradient histogram

SIFT Descriptor:

- Local histograms of gradient orientation
- Allows for small misalignments
=> robust to non-similarity transforms

Indexing :

- kD-tree structure

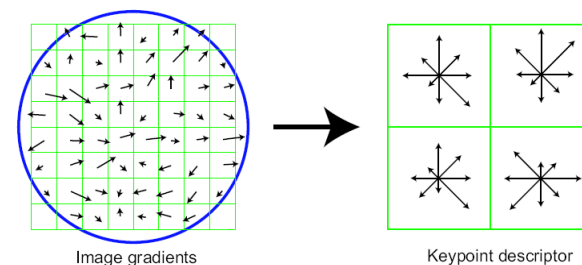
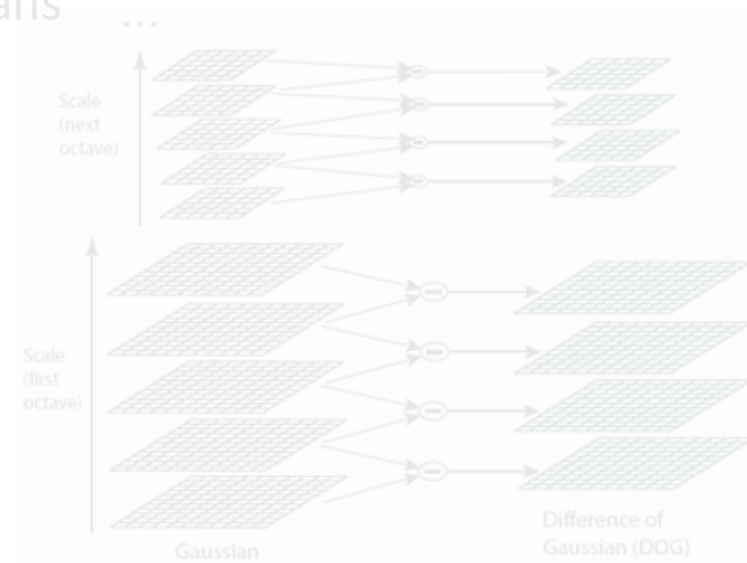
Matching:

- test on euclidean distance of 1st and 2nd match

Verification:

- Hough transform based clustering of correspondences with similar transformations

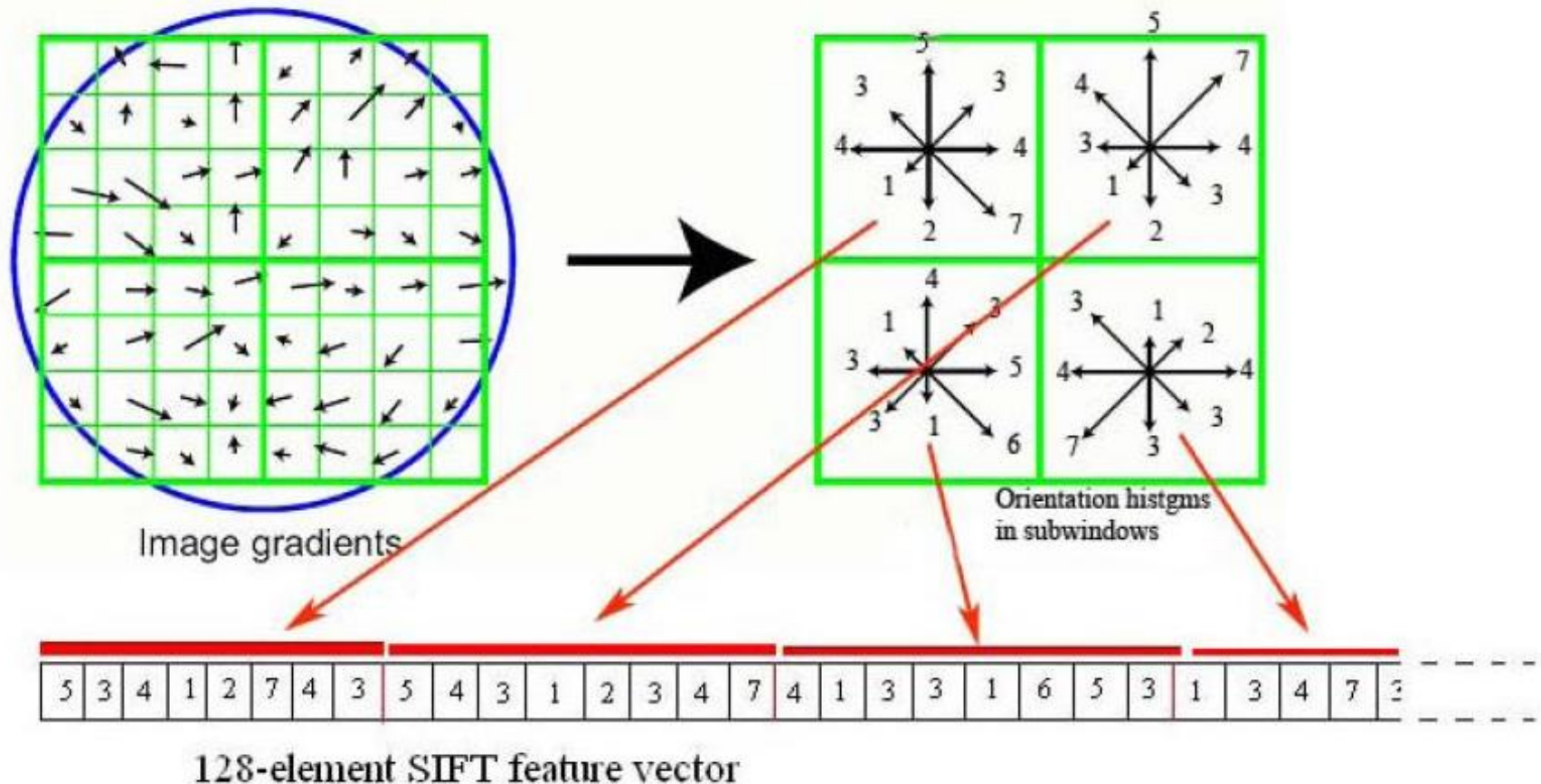
Fast, efficient implementation, **real-time recognition**



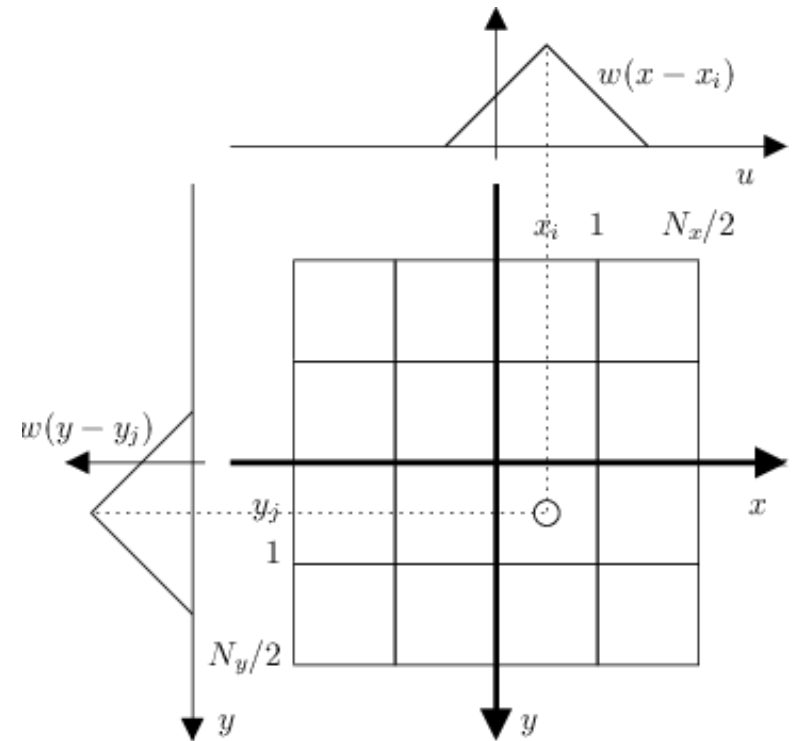
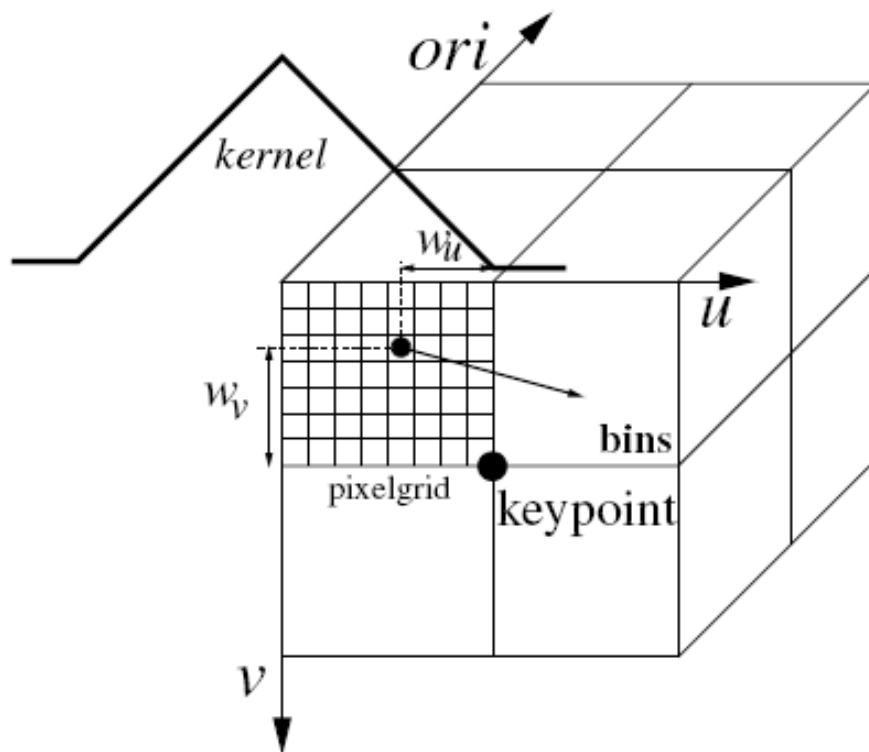
D. G. Lowe: "Distinctive image features from scale-invariant keypoints". IJCV, 2004.

SIFT Descriptor

- A 4x4 histogram lattice of orientation histograms (2x2 shown)
- Orientations quantized (with interpolation) into 8 bins
- Each bin contains a weighted sum of the norms of the image gradients around its center, with complex normalization



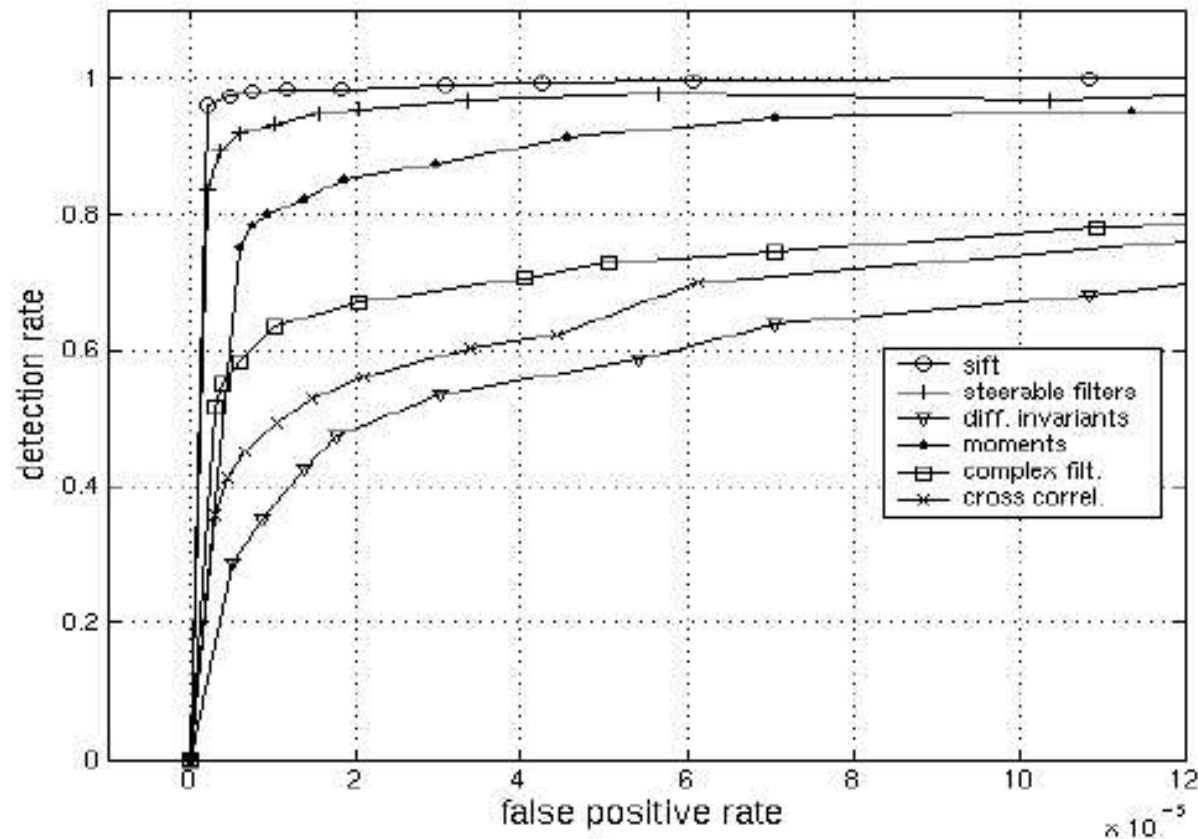
- SIFT descriptor can be viewed as a 3-D histogram:
 - two dimensions (u, v) correspond to image spatial dimensions
 - the other dimension to the image gradient direction – ori – (normally discretized into 8 bins)



■ Empirically shows² very good performance, robust to:

- *image rotation*
- *scale*
- *intensity change*
- *moderate affine transformations*

scale = 2.5, rotation = 45°



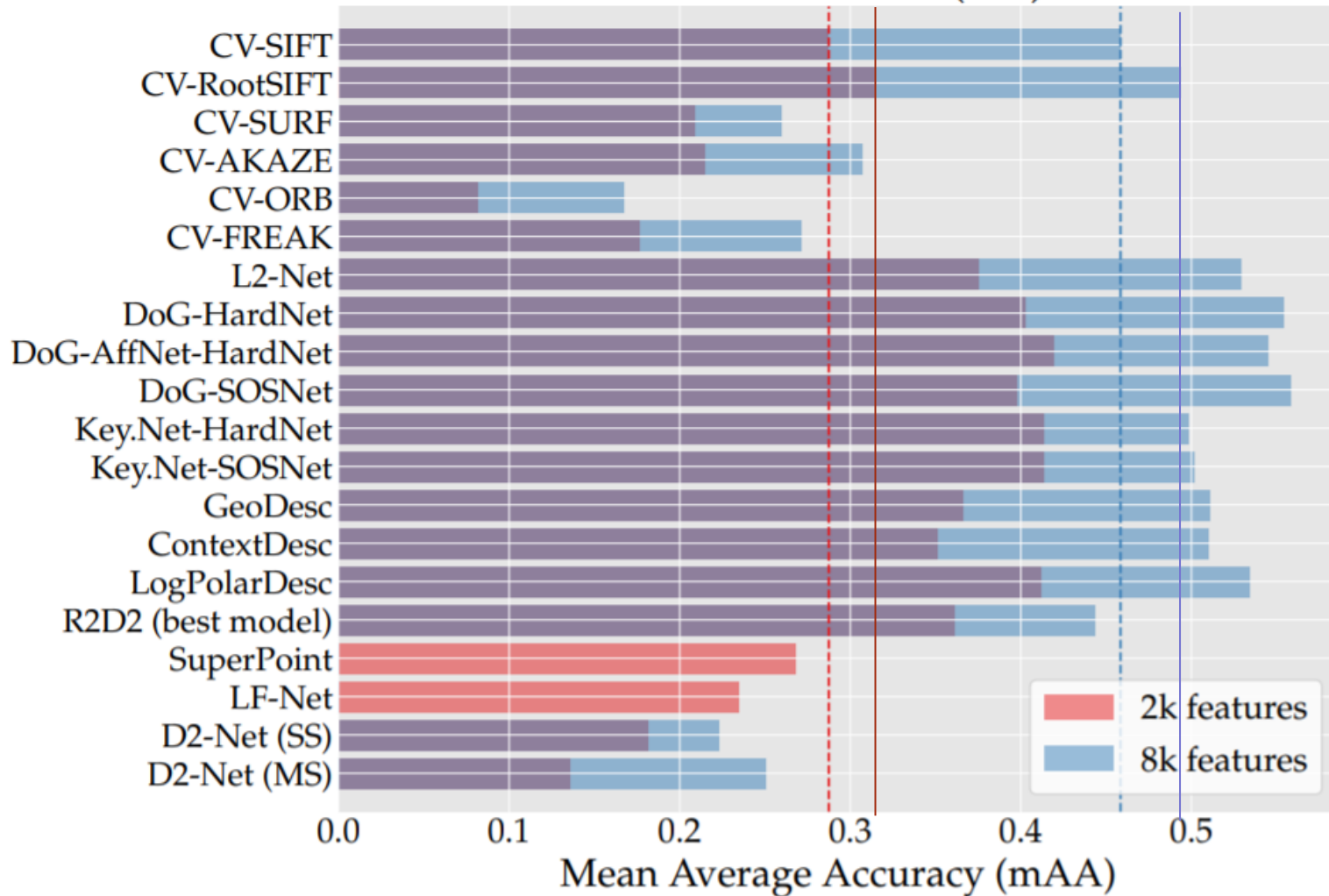
¹ D.Lowe. "Distinctive Image Features from Scale-Invariant Keypoints". IJCV 2004

² K.Mikołajczyk, G.Schmid. "A Performance Evaluation of Local Descriptors". CVPR 2003

- Based on gradient orientations, which are robust to illumination changes
- Spatial binning gives tolerance to small shifts in location and scale, affine change.
- Explicit orientation normalization
- Photometric normalization by making the SIFT vector unit norm
- Orientation histogram gives robustness to small local deformations
- SIFT performance depends on a lot of implementation details

SIFT – Scale Invariant Feature Transform

STEREO: mAA(10^0)



Y. Jin. “Image matching across wide baselines: From paper to practice” IJCV 2020

- By far the most commonly used distinguished region descriptor:
 - fast
 - compact
 - **works for a broad class of scenes**
 - source code available in OpenCV, vlfeat, kornia, etc.
- Large number of ad hoc parameters.
Enormous follow-up literature on both “improvements” and improvements [RootSIFT, HoG, Daisy]
 - GLOH, HoG: different grid, not 4x4, not necessarily a square
 - Daisy: many parameters optimized
 - RootSIFT: square root normalization
- SIFT dominance currently challenged by the CNN-based descriptors

Thank you for attention!

Thank you for
attention!